Moments Of Clarity in Machine Learning for Jet Physics

Rikab Gambhir

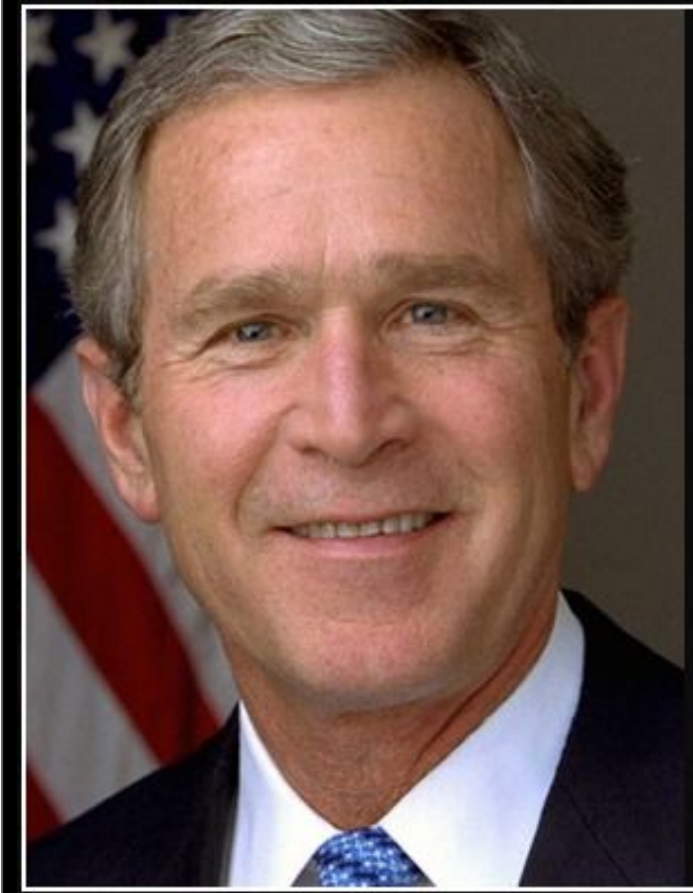
Based on [**RG**, Nachman, Thaler; 2205.03413]
[**RG**, Nachman, Thaler; 2205.05084]
[Ba, Dogra, **RG**, Tasissa, Thaler; 2302.12266]
[**RG**, Thaler, Wu; WIP]
[**RG**, Larkoski, Thaler; 2410.05379]
[**RG**, Osathapan, Thaler; 2403.08854]
[**RG**, Mastandrea; WIP]

Moments Of Clarity in Machine Learning for Jet Physics

Rikab Gambhir

Based on [**RG**, Nachman, Thaler; 2205.03413]
[**RG**, Nachman, Thaler; 2205.05084]
[Ba, Dogra, **RG**, Tasissa, Thaler; 2302.12266]
[**RG**, Thaler, Wu; WIP]
[**RG**, Larkoski, Thaler; 2410.05379]
[**RG**, Osathapan, Thaler; 2403.08854]
[**RG**, Mastandrea; WIP]

Part 0: Introduction



Rarely is the question asked: Is our ~~children~~ learning?

~~machines~~

physicists using machines

— George W. Bush —

AZ QUOTES

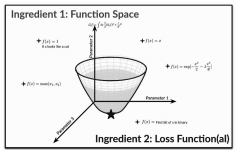
Outline

This talk is going to be **four mini-vignettes** about my work applying Machine Learning to Jet Physics, and the Moments of Clarity I've had along the way.

Key takeaway: Machine learning is *not* a black box, but rather an extremely useful tool for *many different* jet physics problems. All you have to do is design your function space and loss functional to encode *precisely* the physics you want to learn!
You have full control over your physics!

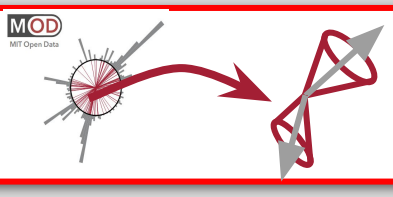
Part 0 (You are here)

Introduction



Part I

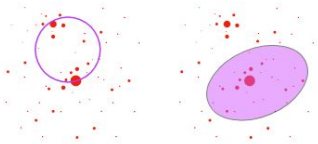
Learning **Uncertainties** the **Frequentist** Way



[**RG**, Nachman, Thaler, 2205.05084]
[**RG**, Nachman, Thaler, 2205.05084]

Part II

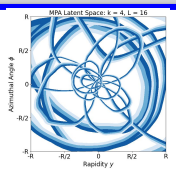
Can you Hear the **Shape** of a Jet?



[Ba, Dogra, **RG**, Tasissa, Thaler, 2302.12266]
[**RG**, Thaler, Wu, WIP]
[**RG**, Larkoski, Thaler, 2410.05379]

Part III

Moment of Clarity:
Streamlining Latent Spaces



[**RG**, Osathapan, Thaler, 2403.08854]

Part IV

Renormalizing Flows:
Taming Perturbation Theory

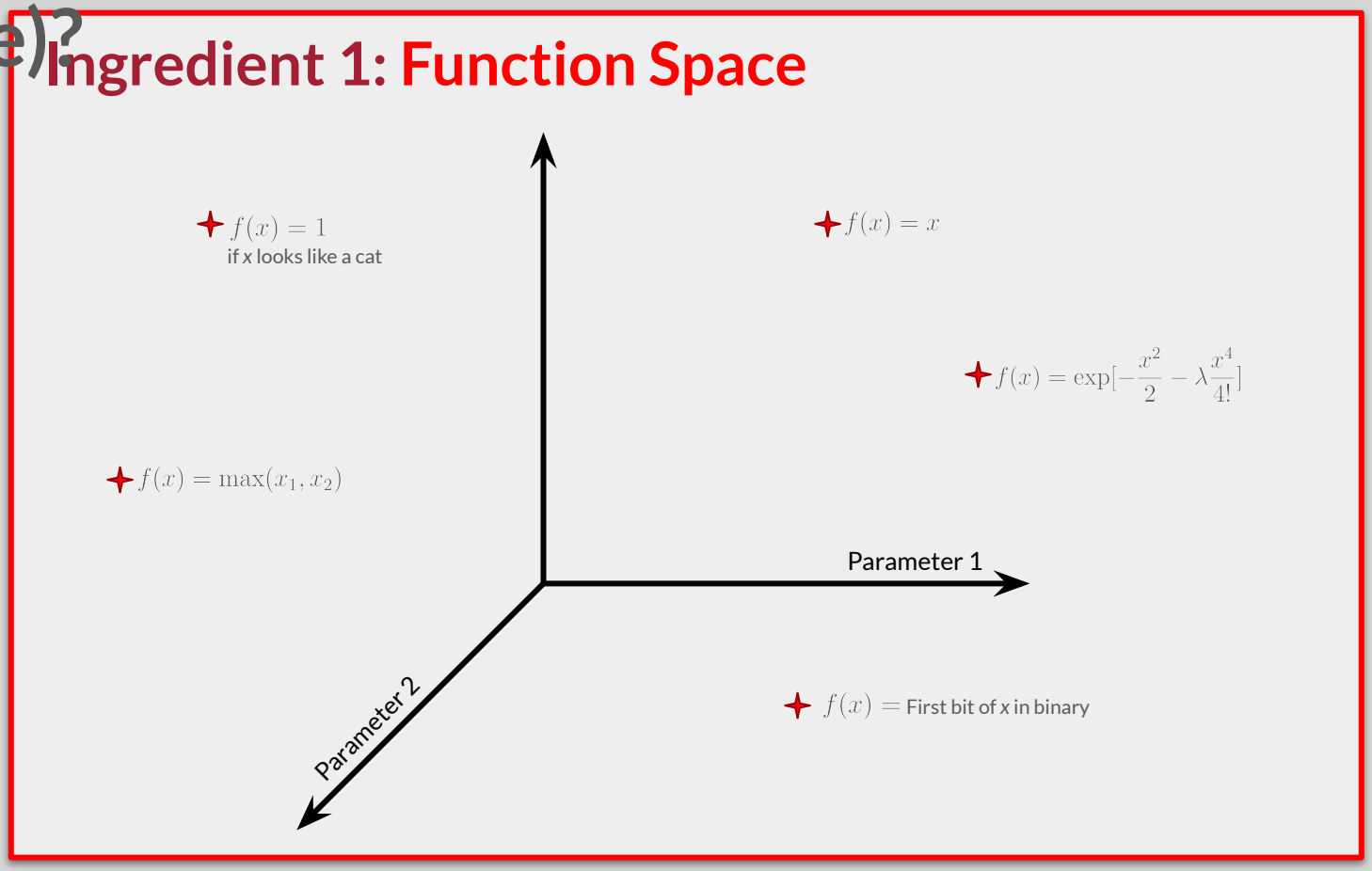
[**RG**, Mastandrea, WIP]

$$\exp[\alpha\frac{d}{d\alpha}]p(x) = p^0(x) + \alpha^1 q^1(x) + \ldots + \alpha^N p^N(x) + \mathcal{O}(\alpha^{N+1})$$
$$\exp[\alpha\frac{d}{d\alpha}]q(x) = q^0(x) + \alpha^1 q^1(x) + \ldots + \alpha^N q^N(x) + \mathcal{O}(\alpha^{N+1})$$

equal!

What even is Machine Learning (Rikab's Take)?

Ingredient 1: Function Space



Machine Learning (ML) = A method to *numerically* explore the space of functions

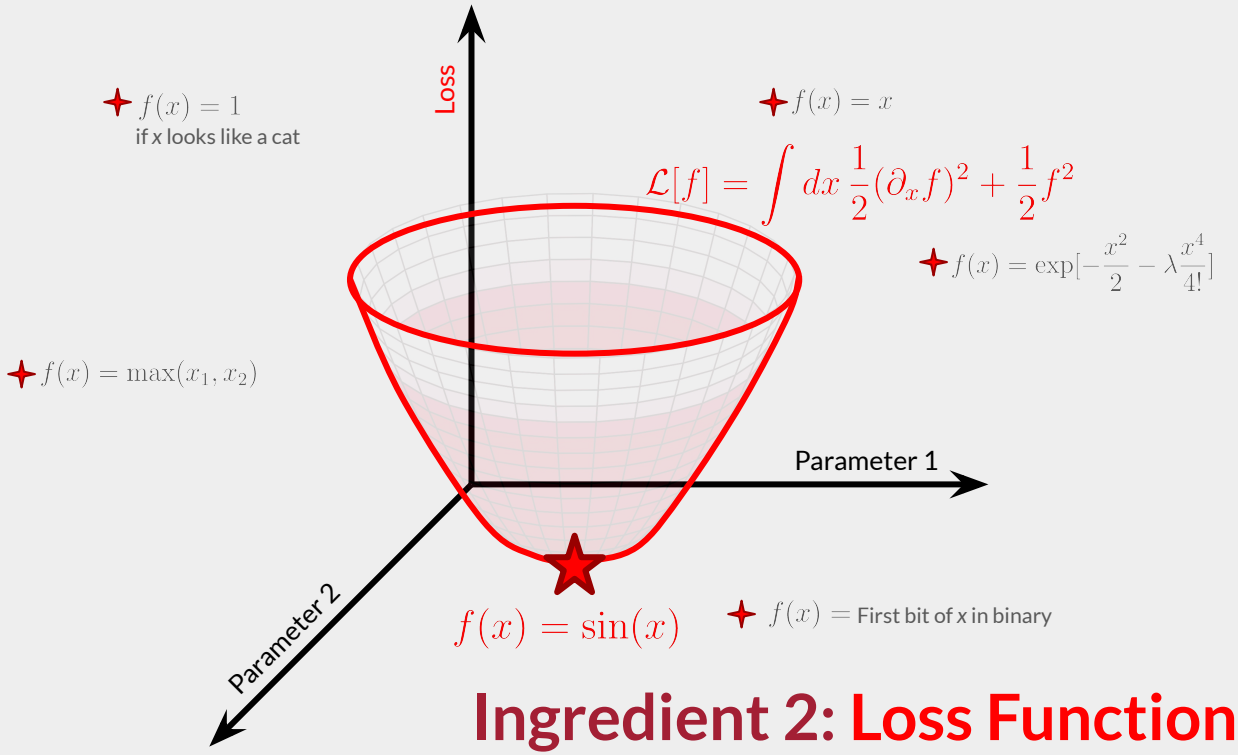
Key workhorse: **Universal Approximation Theorem(s)**

Large classes of functions can be approximated by *Neural Networks* = Parametrized Functions

For this talk, the details of how this approximation is constructed (layers, activations, …) don't matter!

What even is Machine Learning (Rikab's Take)?

Ingredient 1: Function Space



Ingredient 2: Loss Function(al)

Many problems in life (physics and statistics) involve taking **minimums** over function space!

This is *identical* to how we do Lagrangian Mechanics, down to using the Euler-Lagrange equations.

Statistical Inference (Part I), Defining Observables (Part II), Classification (Part III), even Resummation (Part IV)

ML lets us do functional minimization *numerically*!

The details of how we do this (gradient descent, autodifferentiation, ...) are not important for this talk.

Jets 101

Hard Process

The high energy collision that initiates a jet.

Usually perturbative!

Parton Shower

Large-log probability to emit soft or collinear radiation.

Needs to be resummed! Parton showers do this numerically.

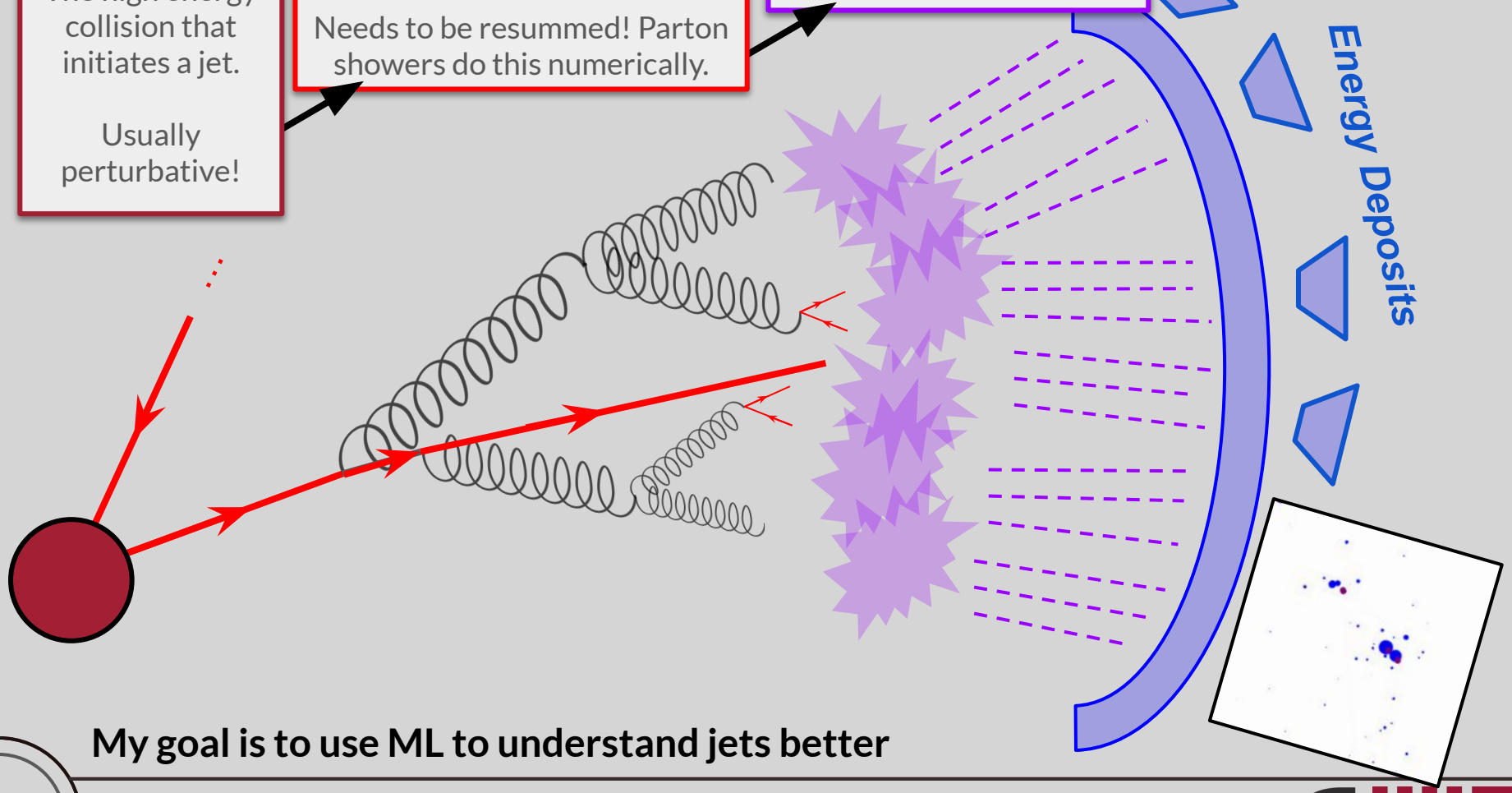
Hadronization

At Λ_{QCD}, non-perturbative physics takes over.

Implemented using pheno models fit to data

Detection

Final energy deposits at the LHC are highly nontrivial and need to be simulated



My goal is to use ML to understand jets better

Jets 101

Hard Process

The high energy collision that initiates a jet.

Usually perturbative!

Parton Shower

Large-log probability to emit soft or collinear radiation.

Needs to be resummed! Parton showers do this numerically.

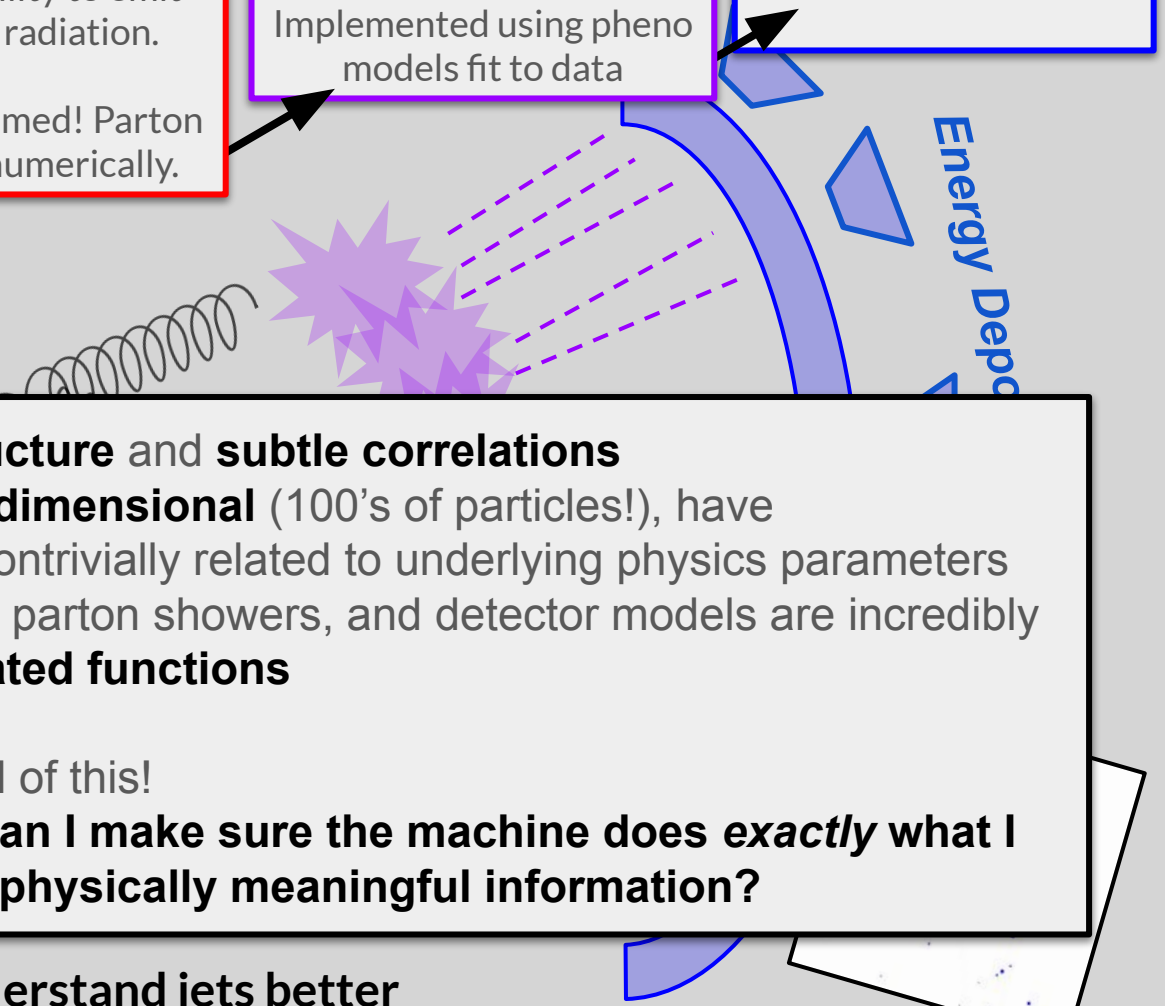
Hadronization

At Λ_{QCD}, non-perturbative physics takes over.

Implemented using pheno models fit to data

Detection

Final energy deposits at the LHC are highly nontrivial and need to be simulated



- Jets have rich **latent structure** and **subtle correlations**
- Measurements are **high dimensional** (100's of particles!), have **uncertainties**, and are nontrivially related to underlying physics parameters
- Perturbative calculations, parton showers, and detector models are incredibly **complex and sophisticated functions**

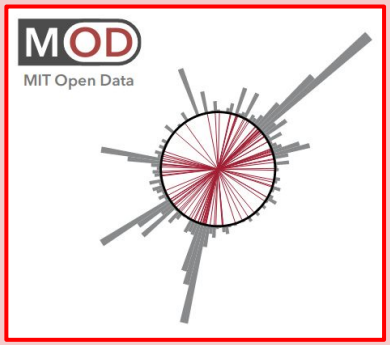
…Machines can learn to do all of this!

But what can *I* learn? How can I make sure the machine does *exactly* what I want, and how can I extract physically meaningful information?

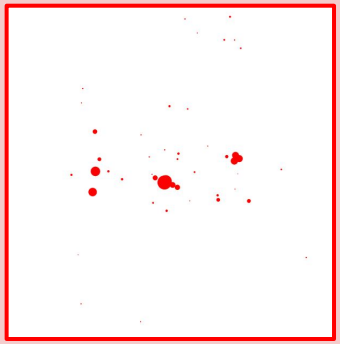
My goal is to use ML to understand jets better

Things a **machine** can understand

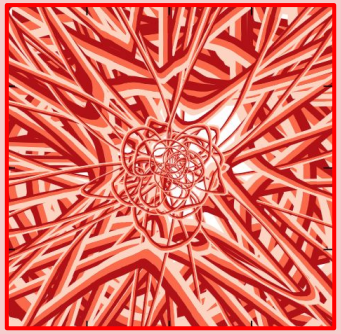
Sophisticated Detector Models



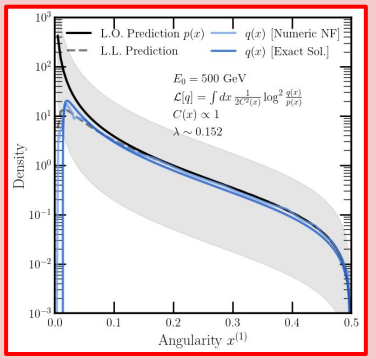
Complicated point clouds



Huge Latent Spaces

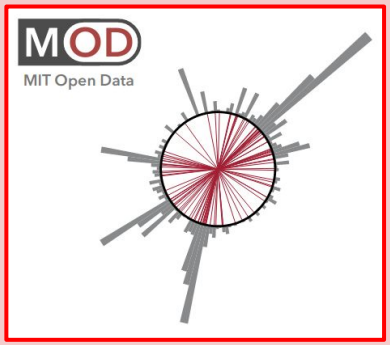


Nontrivially constrained function spaces

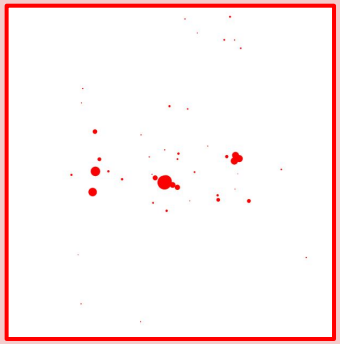


Things a **machine** can understand

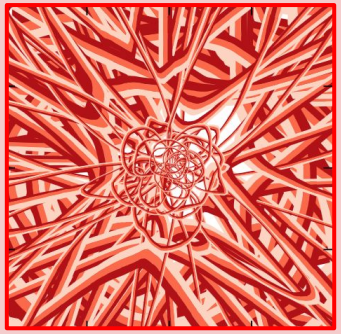
Sophisticated Detector Models



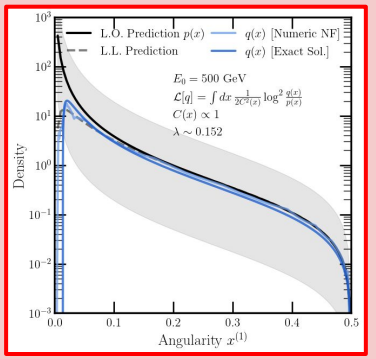
Complicated point clouds

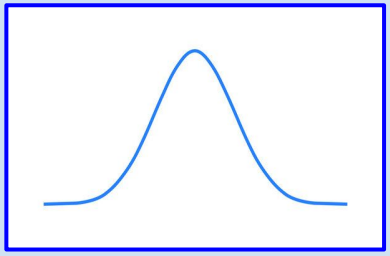


Huge Latent Spaces

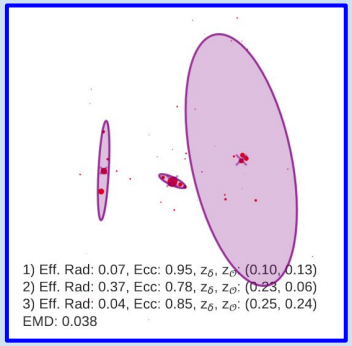


Nontrivially constrained function spaces

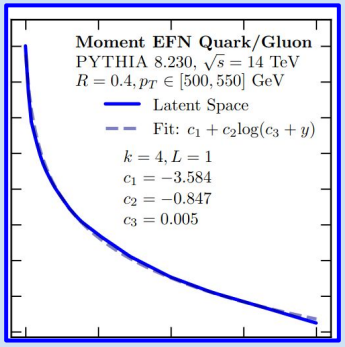




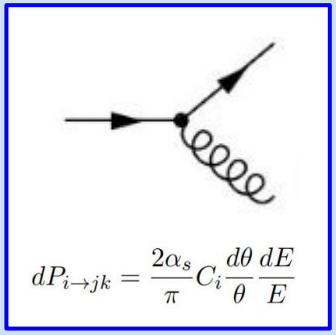
Gaussians



Basic Kindergarten Shapes



Addition, multiplication, and a few elementary functions

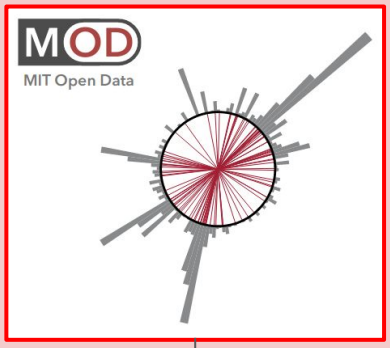


Tree-level Feynman Diagrams

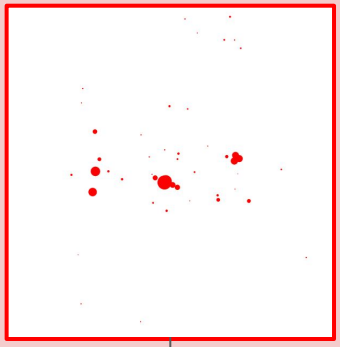
Things my **mortal physicist brain** can understand

Things a **machine** can understand

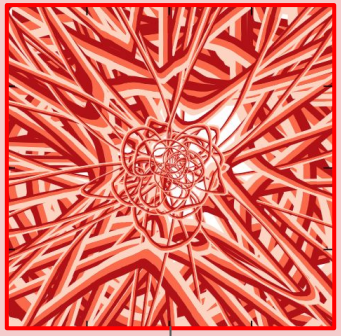
Sophisticated Detector Models



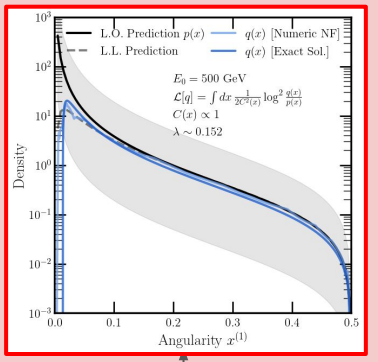
Complicated point clouds



Huge Latent Spaces

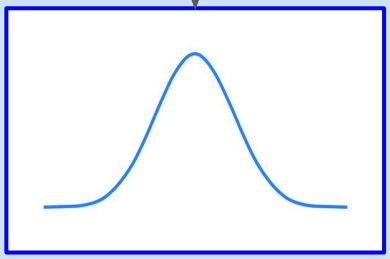


Nontrivially constrained function spaces



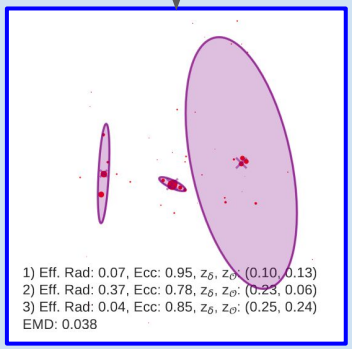
Interface: *Targeted* and *goal-motivated* ML function spaces and loss functionals!

Using the Gaussian Ansatz and DV Loss



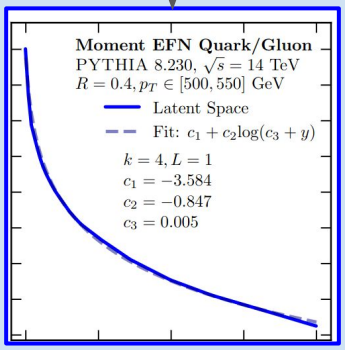
Gaussians

Using faithful optimal transport



Basic Kindergarten Shapes

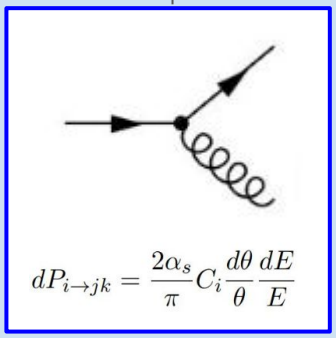
Using Moment Pooling



Addition, multiplication, and a few elementary functions

Using Normalizing Flows

And a perturbative matching Loss



Tree-level Feynman Diagrams

Things my **mortal physicist brain** can understand

Based on: [**RG**, Nachman, Thaler, 2205.03413]
[**RG**, Nachman, Thaler, 2205.05084]
[**RG**, Nachman, Thaler, 2402.14067]

Part I

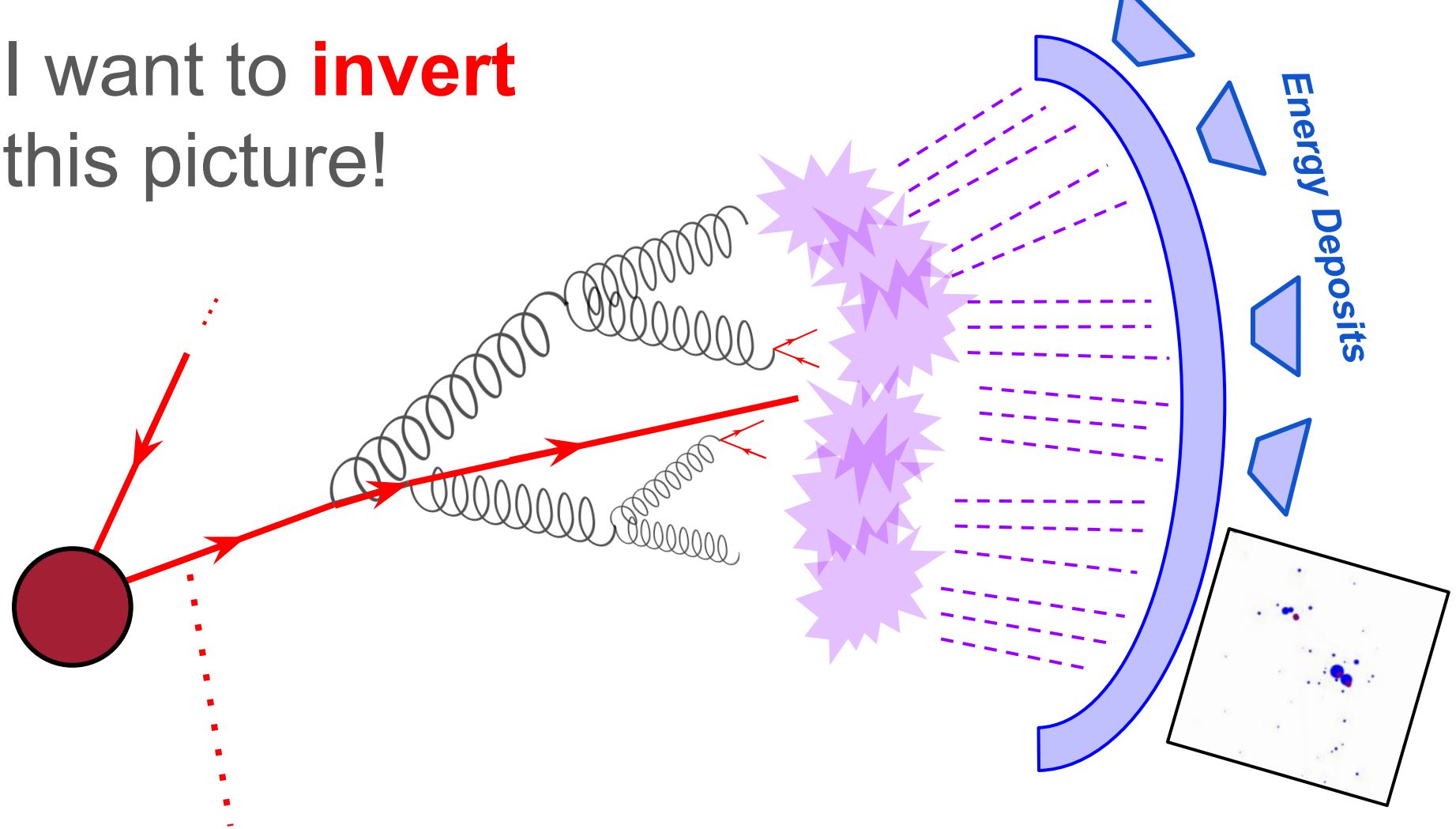
Learning **Uncertainties** the **Frequentist** Way: Calibration and Correlation

Download our repo!

Try *pip install GaussianAnsatz*



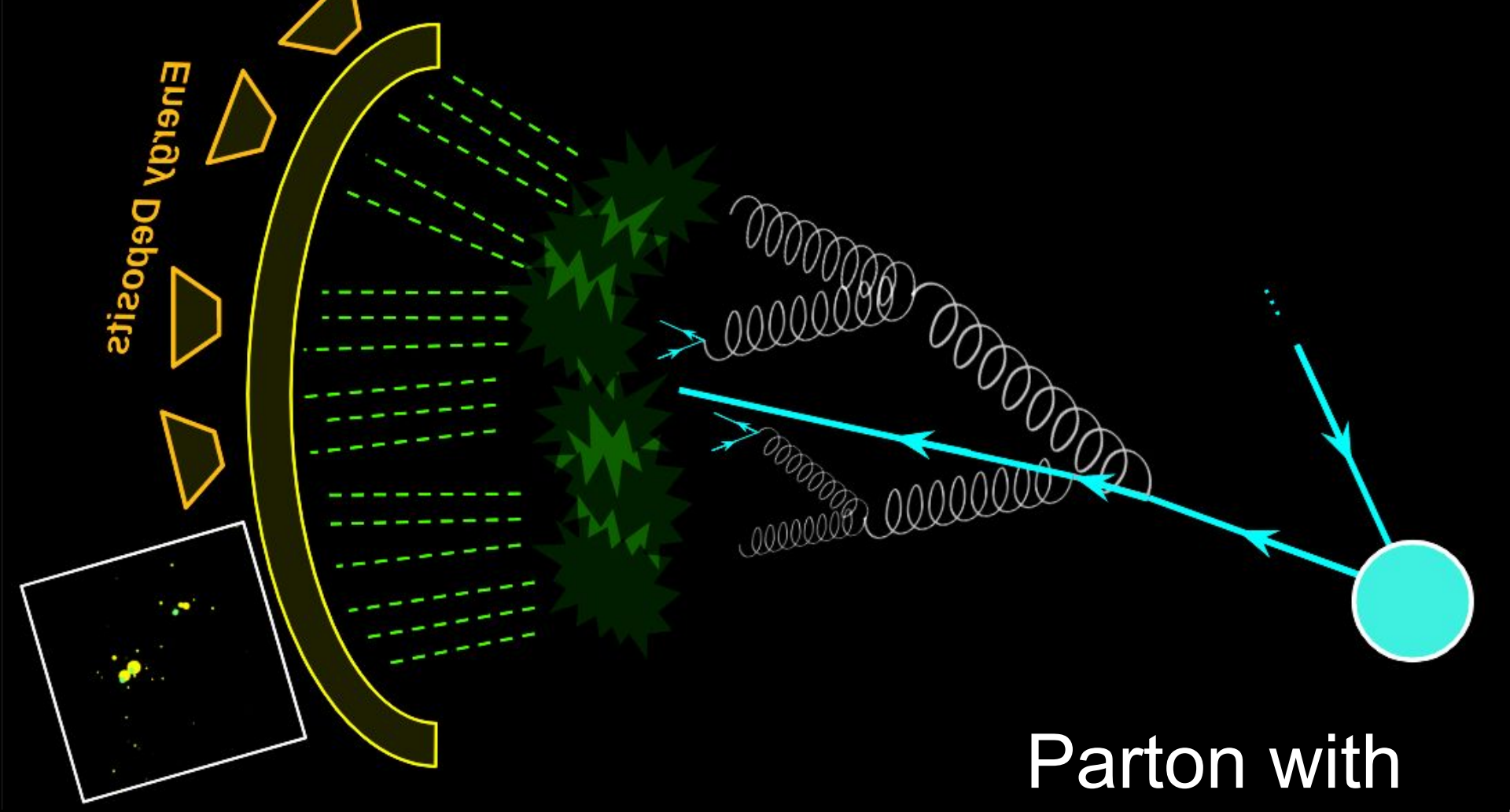
I want to **invert** this picture!



Parton with **true** energy ***z***

Model: Pythia + Geant4

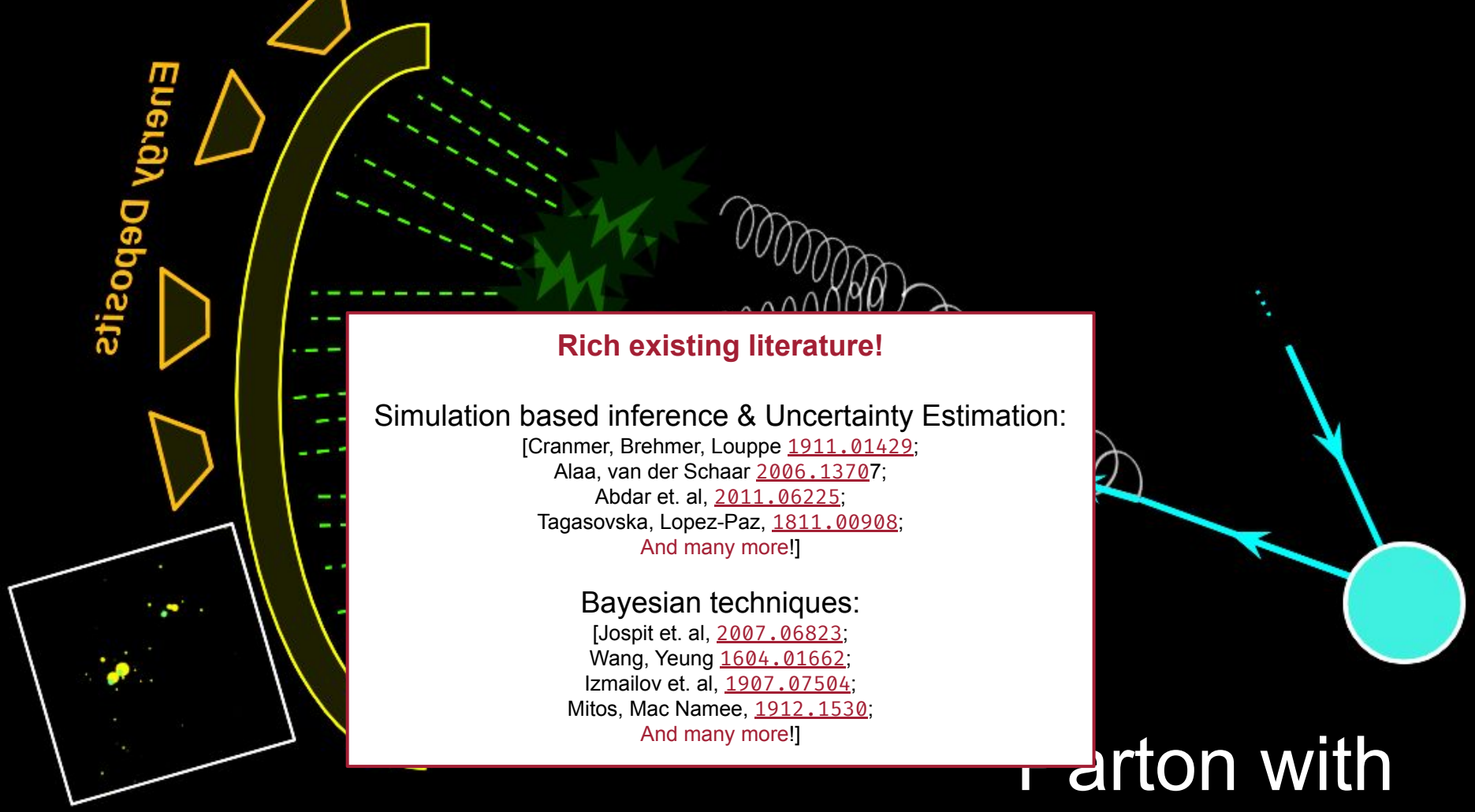
Measured particles ***x***



Measured particles x

What function does this? Machine learn it!

Parton with **estimated** energy $\hat{z} \pm \sigma_z$



Rich existing literature!

Simulation based inference & Uncertainty Estimation:
[Cranmer, Brehmer, Louppe 1911.01429;
Alaa, van der Schaar 2006.13707;
Abdar et. al, 2011.06225;
Tagasovska, Lopez-Paz, 1811.00908;
And many more!]

Bayesian techniques:
[Jospit et. al, 2007.06823;
Wang, Yeung 1604.01662;
Izmailov et. al, 1907.07504;
Mitos, Mac Namee, 1912.1530;
And many more!]

Measured particles x

What function does this?
Machine learn it!

Parton with **estimated** energy $\hat{z} \pm \sigma_z$

Problem Statement: Calibration

Given a training set of (x, z) pairs, can we machine learn a function f such that $f(x) = z$? With uncertainties?

Let's agree on two properties that make a good **Calibration**. If these two properties are satisfied, we can trust that our ML was sensible and extract information!

1. **Closure**: On average, $f(x)$ should be correct for each x! That is, f is **unbiased**.

$$b(z) = \mathbb{E}_{\text{test}}[f(X) - z | Z = z] = 0$$

2. **Universality**: $f(x)$ should not depend on the choice of sampling for z. That is, f is **prior-independent**.

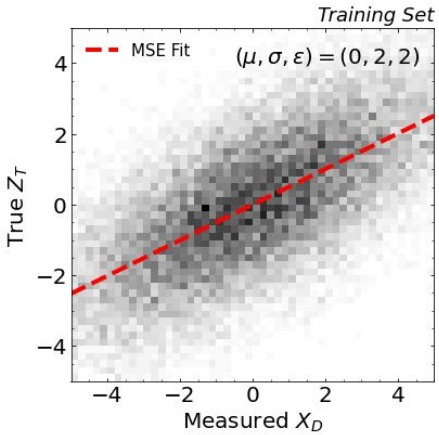
Likelihood: Detector simulation, noise model, etc

$$f \text{ depends only on } p(x|z), \text{ and not } p(z)$$

Naive ML Approach

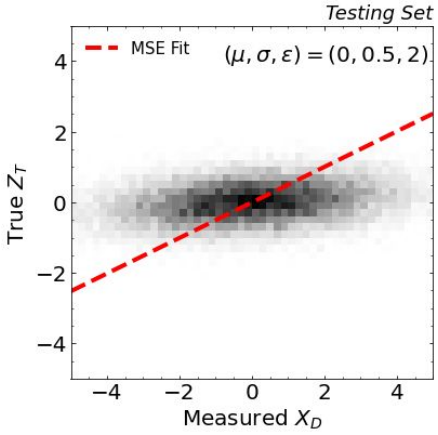
Naive Approach: Let *f*(x) be parameterized by an ordinary Neural Network. Then minimize the "Mean Squared Error" (MSE) – this is what usual curve fitting does!

$$\mathcal{L}[f] = \sum_i [f(x_i) - z_i]^2$$



Same "detector" sim *p*(*x*|*z*), only different priors *p*(*z*)!

We can't apply our calibration universally.



Using Euler-Lagrange, we can *prove* that the function minimizing the MSE will *always* be both prior dependent and biased. We need to choose a smarter function space and/or a smarter loss functional!

[Aside, if we have time]
Maximum Likelihood "Frequentist" Inference

Some statistics facts:

Fact 1: Given data *x*, the inference $\hat{z}(x) = \underset{z}{\text{argmax}}\ T(x,z)$re *T(x,z)* is any monotonic function of the (log) likelihood *p(x|z)*, is always prior-independent.

Fact 2: If the likelihood *p(x|z)* is approximately Gaussian with mean *z*, then the above inference is unbiased, up to subleading corrections.

Fact 3: The Gaussian (dominant) component of the uncertainty* on the inference is given by the (inverse) second derivative of the (log) likelihood:

$$\left[\hat{\sigma}_z^2(x)\right]_{ij} = -\left[\frac{\partial^2 T(x,z)}{\partial z_i\, \partial z_j}\right]^{-1}\Bigg|_{z=\hat{z}}$$

Note that higher derivative terms exist, but these are subleading.

Really, a pseudo-Frequentist confidence interval

[Belghazi, Baratin, Rajeswar, Ozair, Bengio, Courville, Hjelm, 1801.04062;
Le, Nguyen, Phung, 1711.01744
Nowozin; Cseke, 1606.00709]

Moment of Clarity: The Gaussian Ansatz

Given the properties of calibration we want to enforce, the properties we want to extract, and Statistics Facts 1-3, we can design an ML algorithm that does *exactly* what we want!

I claim the **Donsker-Varadhan (DVR)** loss functional solves our problem!

$$\mathcal{L}_{\mathrm{DVR}}[T] = -\left(\mathbb{E}_{P_{XZ}}[T] - \log\left(\mathbb{E}_{P_X \otimes P_Z}\left[e^T\right]\right)\right)$$

Euler Lagrange

What we want!

$$T(x,z) = \log \frac{p(x|z)}{p(x)} + c$$

Unimportant

Nice bonus: DVR is a strict bound on mutual information

Maxima and second-derivatives are usually really hard to estimate – but not for **Gaussians**! Parameterize *T* as:

$$T(x,z) = A(x) + \left(z - B(x)\right) \cdot D(x) + \frac{1}{2}\left(z - B(x)\right)^T \cdot C(x,z) \cdot \left(z - B(x)\right)$$

Take *D* to 0

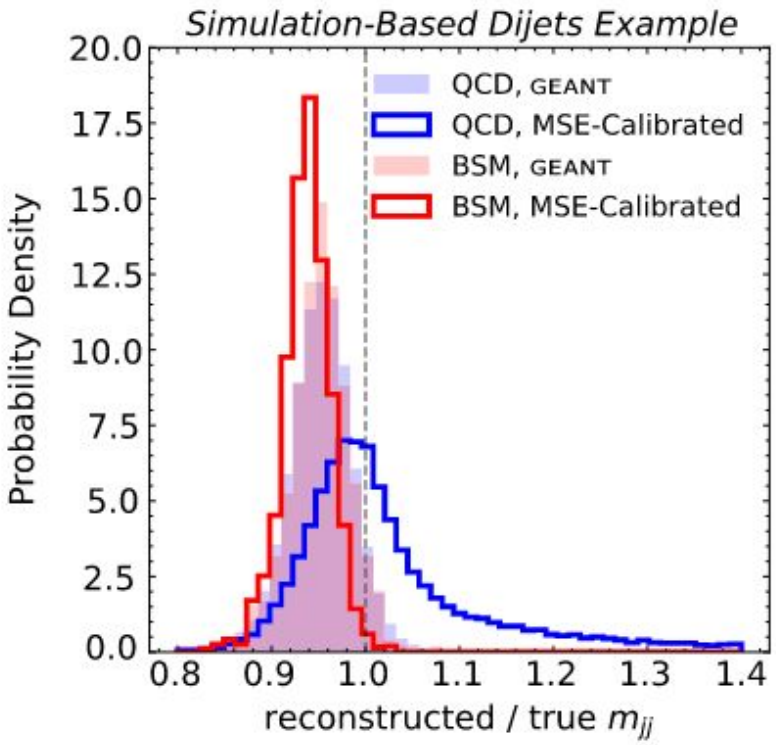
$$\hat{z}(x) = B(x) \qquad \hat{\sigma}_z^2(x) = -\left[C(x, B(x))\right]^{-1}$$

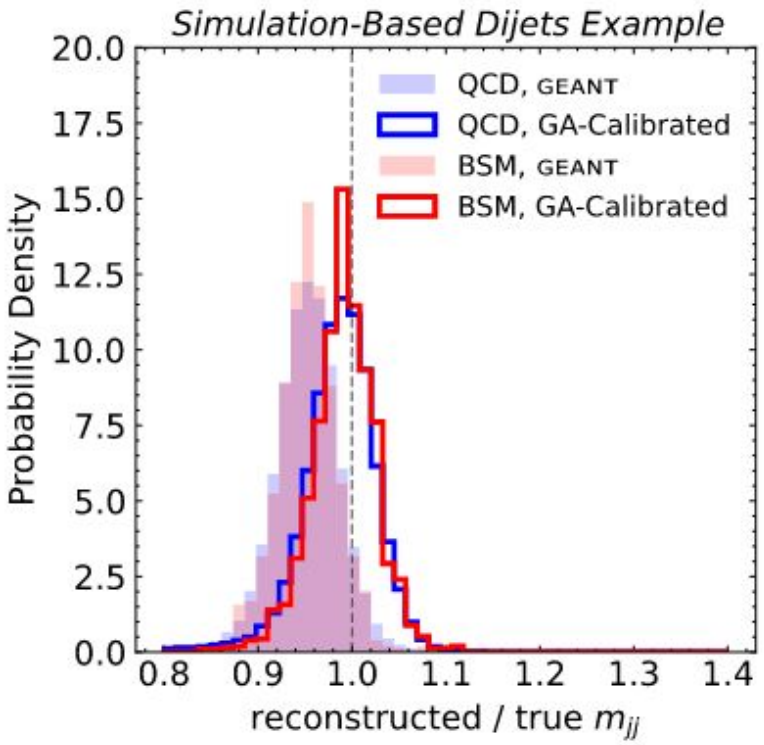
Our Maximum! Our Second Derivative!

Where *A(x), B(x),* and *C(x,z)* are learned functions (and $D \to 0$). This is a universal function approximator!

Lots of other losses also work, but DVR has very nice convergence properties - ask me later!

Example 1: QCD and BSM Dijets



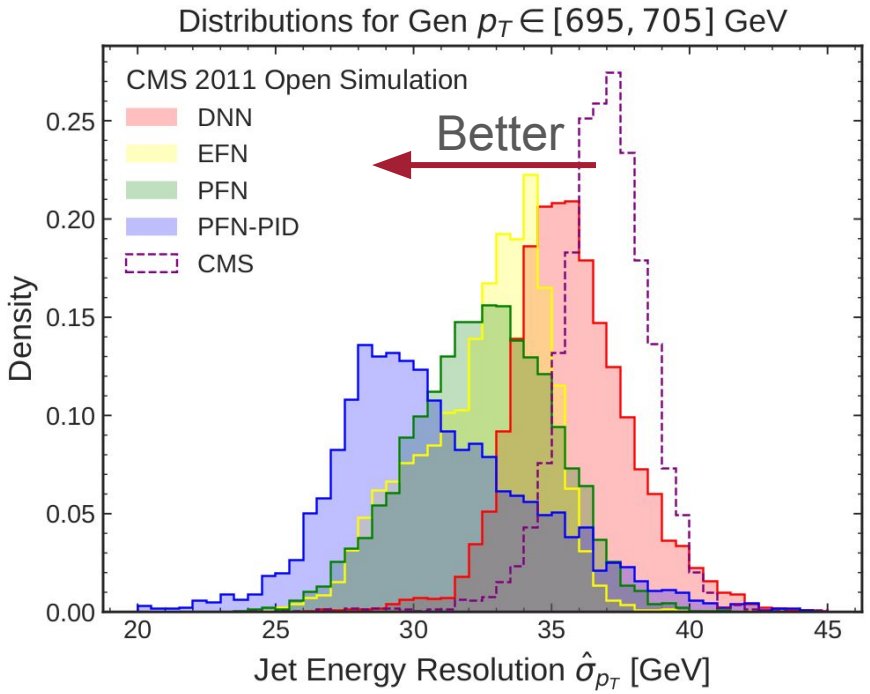


(Left) MSE-fitted network.

(Right) Gaussian Ansatz-fitted network

Clever loss function choice: *Prior dependence is built-in!*

Example 2: Jet Energy Resolution

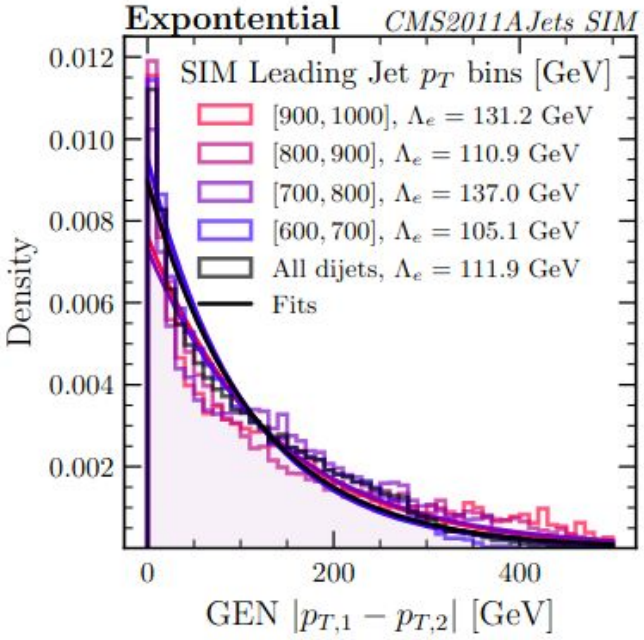


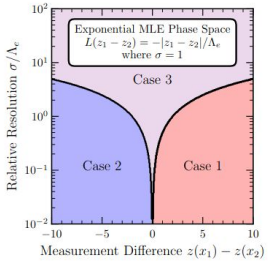
Best of both worlds: Using ML to extract more information* than hand-crafted features, while still being able to extract resolutions in a prior-independent and unbiased way!

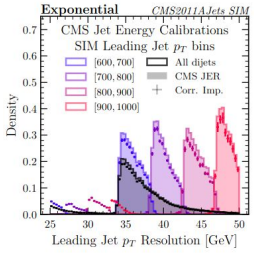
*I mean "information" literally. Ask me later about the cool information theory of the DV loss!

[RG, Nachman, 2402.14067]

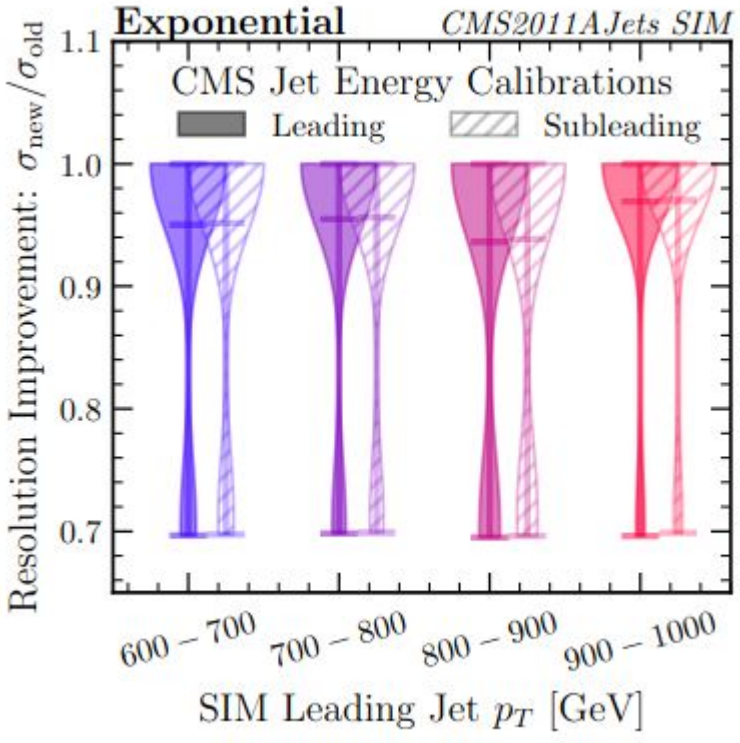
Bonus Example: Seeing Double



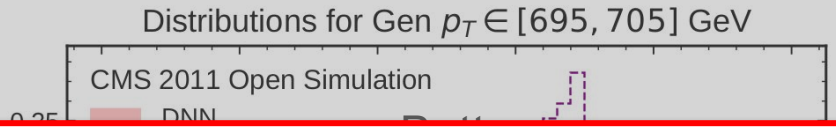




Even *without* ML, we can use multiple features to improve resolutions by using a trick similar to L1/LASSO regularization!



Example 2: Jet Energy Resolution



End of **Part I** The **Moment of Clarity**

By choosing:

1. A Gaussian-like functional space
2. A loss that guarantees prior independence

We can perform inference and uncertainty estimation in a manifestly robust way, while taking advantage of the power of machines to crunch a lot of high-dimensional data!

Best of both worlds: Using ML to extract more information* than hand-crafted features, while still being able to extract resolutions in a prior-independent and unbiased way!

*I mean "information" literally. Ask me later about the cool information theory of the DV loss!

Based on: [Ba, Dogra, **RG**, Tasissa, Thaler, 2302.12266]
[**RG**, Thaler, Wu, WIP]
[**RG**, Larkoski, Thaler, 2410.05379]

Part II

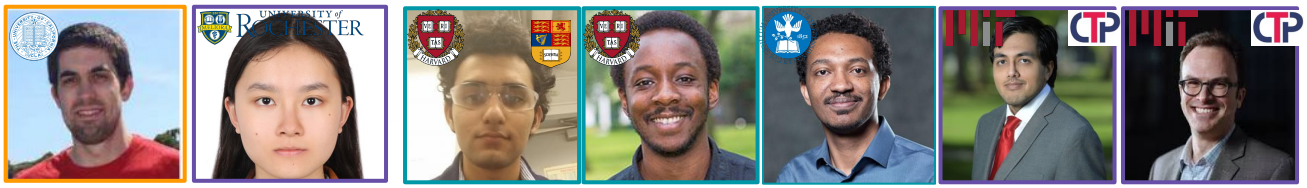
Can you Hear the **Shape** of a Jet?

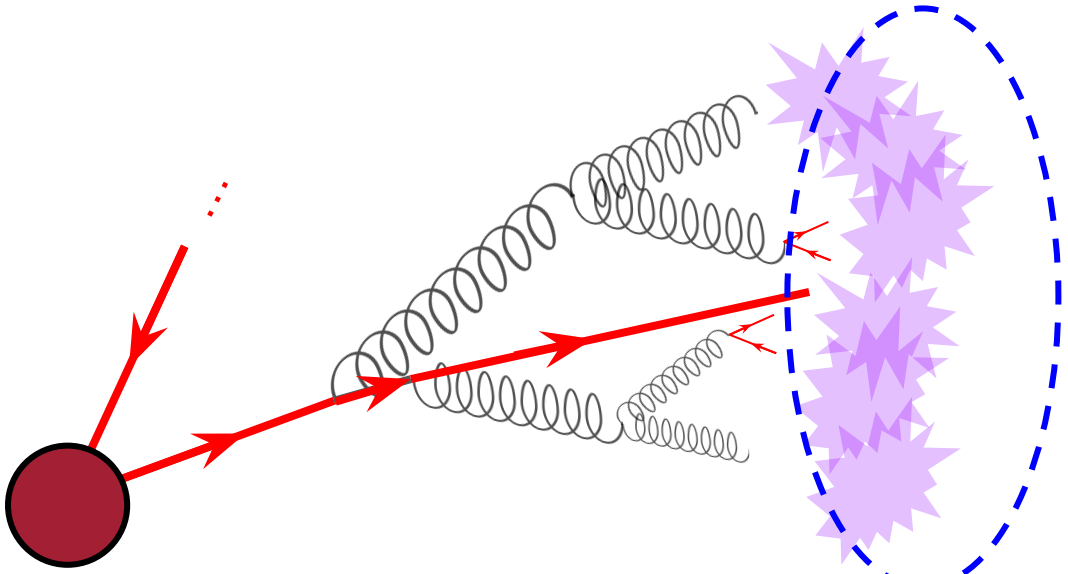
Download SHAPER!

Try `pip install pyshaper`

Download SPECTER!

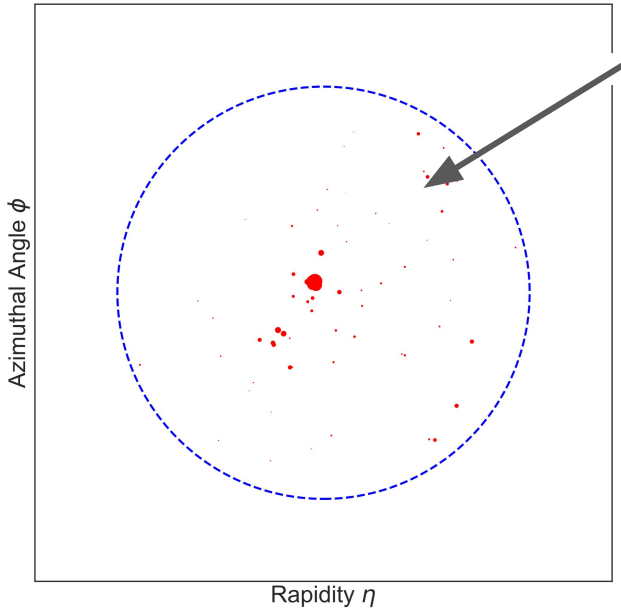
(New!) `pip install specterpy`





How “*big*” is this jet?

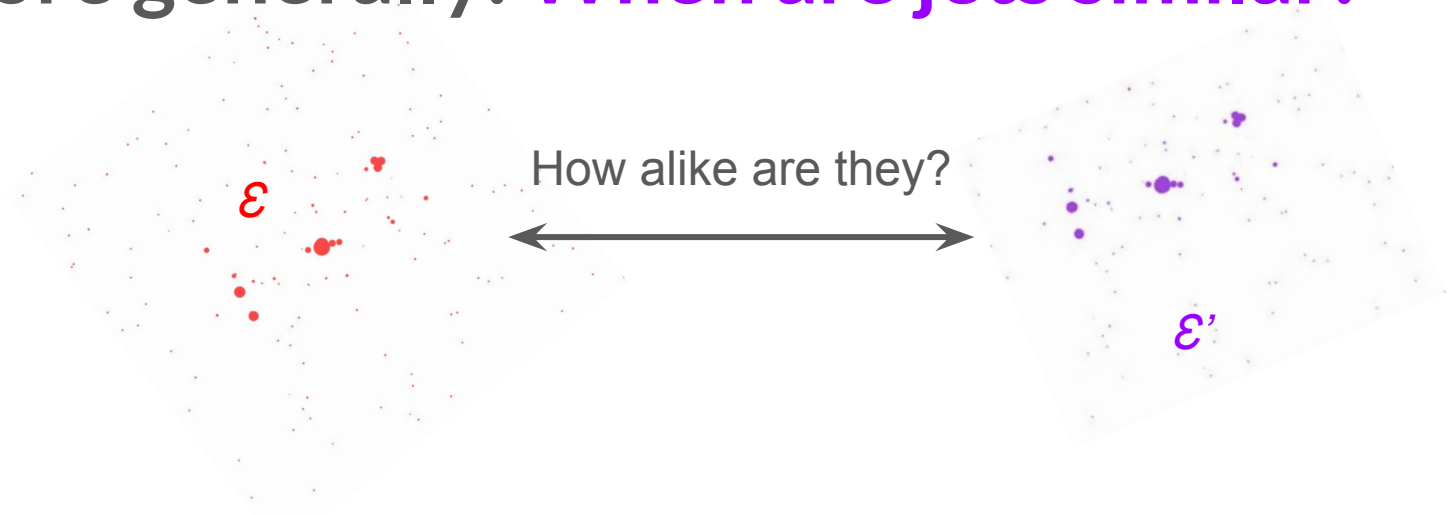
… What do we even mean by the “size” of a jet?



By following this line of thinking, we’ll be able to come up with a new class of QCD observables!

I don’t mean the fixed size of a jet algorithm, e.g. AK4, but rather a dynamical notion of a jet size!

More generally: When are jets similar?

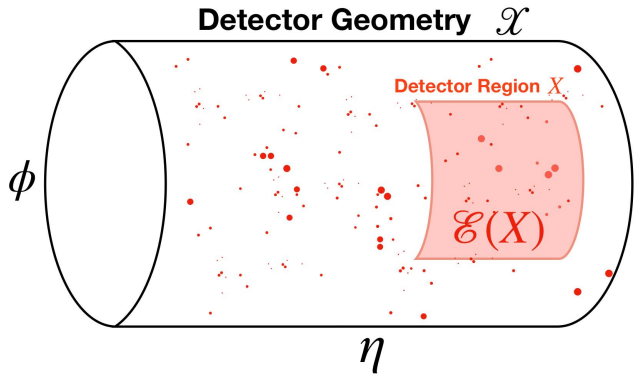


Jets can be represented as **point clouds** – let's scour through the ML and computer vision literature for a metric on point clouds we like![*]

Detector Coordinate (η, ϕ)

$$\mathcal{E}(y) = \sum_i z_i \delta(y - y_i)$$

Energy Fraction E_i / E_{Tot}



[*]Or better yet, construct one!

More generally: When are jets similar?

How alike are they?

The **moment of clarity**:

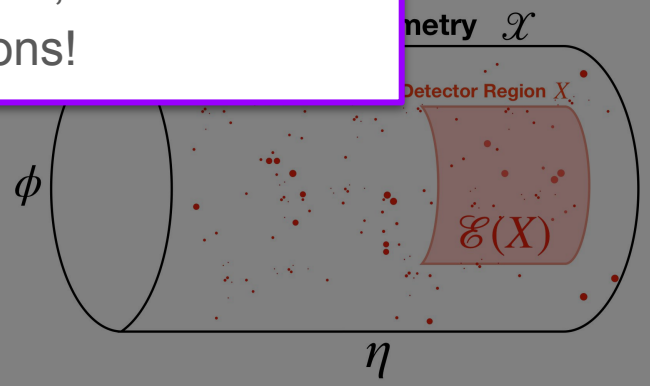
Let's figure out what properties we want our similarity metric to satisfy, and then simply construct one satisfying all the properties!

This is just like a loss functional, because our metric is between *distributions*, which are themselves functions!

Jets can be
comp
he ML and
like![*]

$$\mathcal{E}(y) = \sum_i z_i \delta(y - y_i)$$

Energy Fraction E_i / E_{Tot}



[*]Or better yet, construct one!

Loss: The Wasserstein Metric

Let's demand the following reasonable properties of our metric on point clouds:

1. … is a genuine metric: nonnegative, nondegenerate, finite
2. … is **IRC-safe**[*] (Calculable and robust)
3. … is translationally invariant
4. … is invariant to particle labeling
5. respects the detector metric ***faithfully***[**]

[*]Ask me for more details on this offline!
[**] Preserves distances between *extended* objects, not just points

*Keep nondegeneracy in mind, we'll break it later!

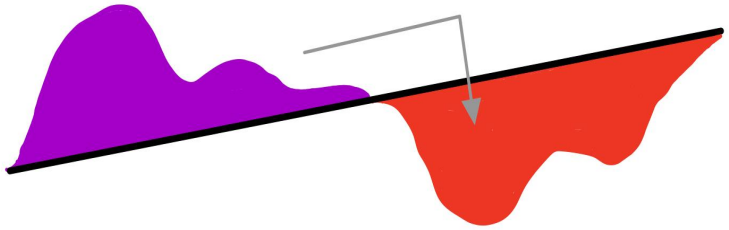
Proved in [Ba, Dogra, **RG**, Tasissa, Thaler, 2302.12266]
Ask me later for proof details!

See also: [Komiske, Metodiev, Thaler, 1902.02346,
Komiske, Metodiev, Thaler, 2004.04159]

Loss: The Wasserstein Metric

Let's demand the following reasonable properties of our metric on point clouds:

1. … is a genuine metric: nonnegative, nondegenerate*, finite
2. … is **IRC-safe*** (Calculable and robust)
3. … is translationally invariant
4. … is invariant to particle labeling
5. respects the detector metric ***faithfully*****



EMD = Work done to move "dirt" optimally

I can prove that the *only* metric that satisfies this is the **Wasserstein Metric / EMD**!

$$\mathrm{EMD}^{(\beta,R)}(\mathcal{E},\mathcal{E}') = \min_{\pi\in\mathcal{M}(\mathcal{X}\times\mathcal{X})}\left[\frac{1}{\beta R^\beta}\left\langle \pi, d(x,y)^\beta\right\rangle\right] + |\Delta E_{\mathrm{tot}}|$$

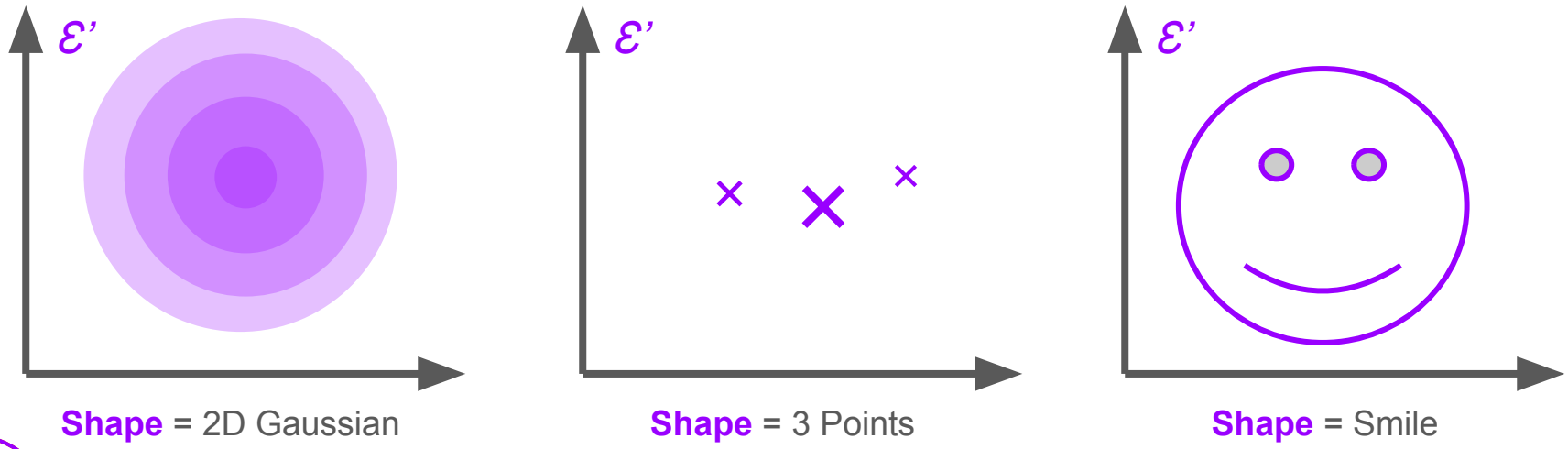
$$\pi(\mathcal{X},Y)\le\mathcal{E}'(Y),\quad \pi(X,\mathcal{X})\le\mathcal{E}(X),\quad \pi(\mathcal{X},\mathcal{X}) = \min(E_{\mathrm{tot}},E'_{\mathrm{tot}})$$

A staple of computer vision ML is also useful for jets!
Other common ML metrics, like Chamfer, MMDs, or Kernel metrics arent *faithful*.

Shapes as Energy Flows

Energy flows don't have to be real events – they can be *any* energy distribution in detector space, or **shape**. In our framework, these shapes will later become our **Function Space**

Can make anything you want! Even continuous or complicated shapes.
(Or, something you calculate in perturbative QCD)



Shape = 2D Gaussian

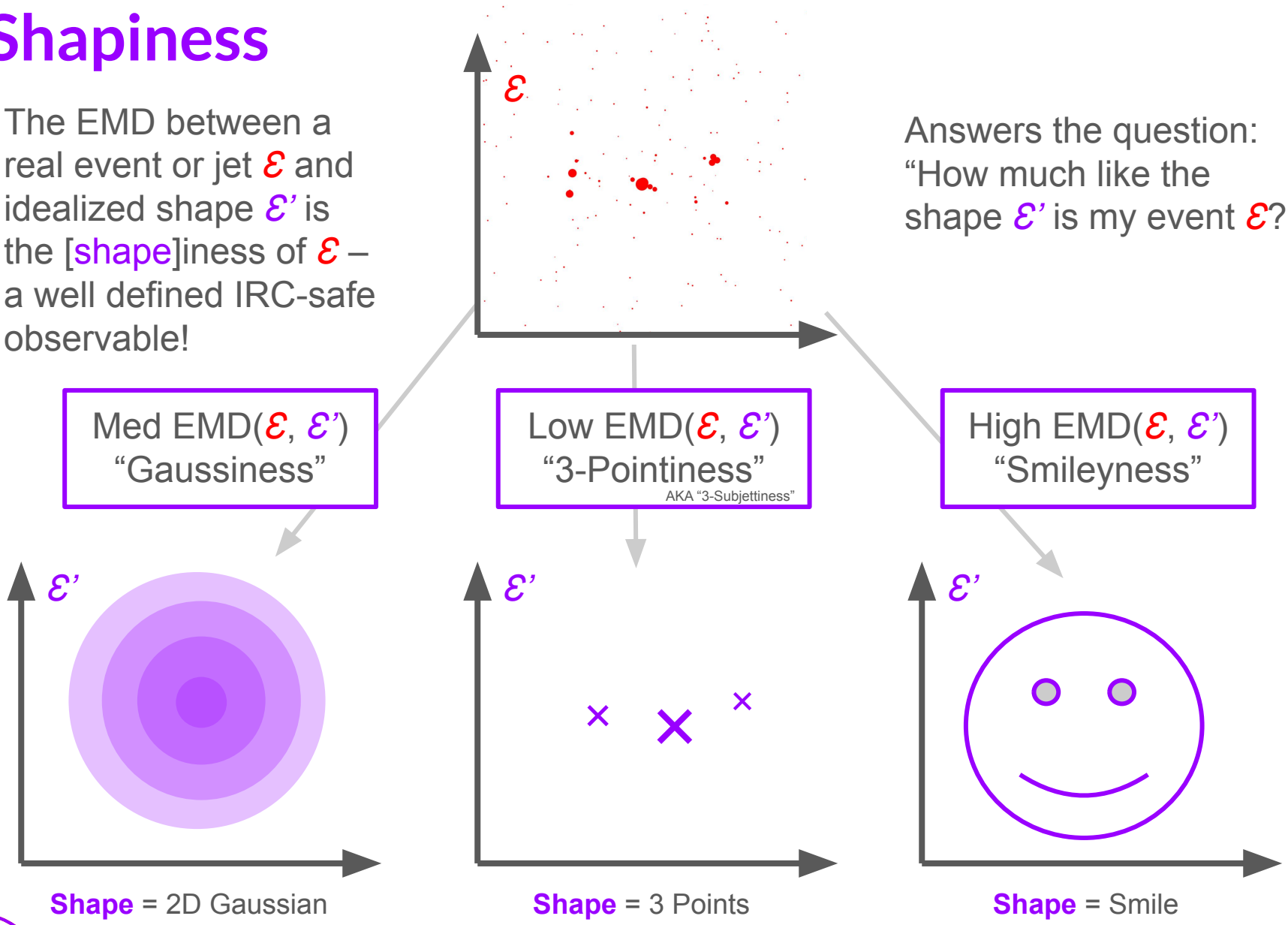
Shape = 3 Points

Shape = Smile

Shapiness

The EMD between a real event or jet $\mathcal{E}$ and idealized shape $\mathcal{E}'$ is the [shape]iness of $\mathcal{E}$ – a well defined IRC-safe observable!

Answers the question: "How much like the shape $\mathcal{E}'$ is my event $\mathcal{E}$?"

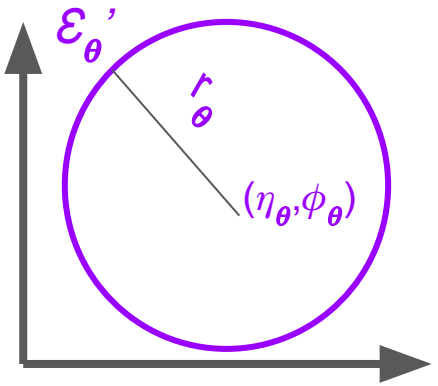


Mathematical Details - Shapiness

Rather than a single shape, consider a **parameterized manifold** $\mathcal{M}$ of **energy flows**.

e.g. The manifold of uniform circle energy flows:

$$\mathcal{E}_\theta'(y) = \begin{cases} \frac{1}{2\pi r_\theta} & |\vec{y} - \vec{y}_\theta| = r_\theta \\ 0 & |\vec{y} - \vec{y}_\theta| \neq r_\theta \end{cases}$$



Then, for an event $\mathcal{E}$, define the **shapiness** $\mathcal{O}_\mathcal{M}$ and **shape parameters** $\theta_\mathcal{M}$, given by:

$$\mathcal{O}_\mathcal{M}(\mathcal{E}) \equiv \min_{\mathcal{E}_\theta \in \mathcal{M}} \text{EMD}^{(\beta,R)}(\mathcal{E}, \mathcal{E}_\theta)$$

$$\theta_\mathcal{M}(\mathcal{E}) \equiv \operatorname{argmin}_{\mathcal{E}_\theta \in \mathcal{M}} \text{EMD}^{(\beta,R)}(\mathcal{E}, \mathcal{E}_\theta)$$

Unlike our previous function space in the last Part, we have only a few parameters

[P. Komiske, E. Metodiev, and J. Thaler, 2004.04159;
J. Thaler, and K. Van Tilburg, 1011.2268;
I. W. Stewart, F. J. Tackmann, and W. J. Waalewijn, 1004.2489.;
S. Brandt, C. Peyrou, R. Sosnowski and A. Wroblewski, PRL 12 (1964) 57-61;
C. Cesarotti, and J. Thaler, 2004.06125]

Observables ⇔ Manifolds of Shapes

Observables can be specified solely by defir
a **manifold of shapes**:

$$\mathcal{O}_{\mathcal{M}}(\mathcal{E}) \equiv \min_{\mathcal{E}_\theta \in \mathcal{M}} \mathrm{EMD}^{(\beta,R)}(\mathcal{E}, \mathcal{E}_\theta),$$

$$\theta_{\mathcal{M}}(\mathcal{E}) \equiv \operatorname*{argmin}_{\mathcal{E}_\theta \in \mathcal{M}} \mathrm{EMD}^{(\beta,R)}(\mathcal{E}, \mathcal{E}_\theta),$$

Functional Minimization! Many well-known observables* already have this form!

Observable	Manifold of Shapes
N-Subjettiness	Manifold of *N*-point events
N-Jettiness	Manifold of *N*-point events with floating total energy
Thrust	Manifold of back-to-back point events
Event / Jet Isotropy	Manifold of the single uniform event

… and more!

All of the form “How much like **[shape]** does my **event** look like?”
Generalize to *any* shape.

*These observables are usually called event shapes or jet shapes in the literature – we are making this literal!

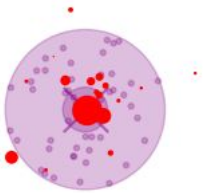
Hearing Jets Ring

(and Disk, and Ellipse)

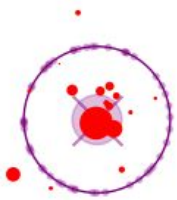
$\mathcal{O}_{\mathcal{M}}(\mathcal{E})$ answers: "How much like a shape in $\mathcal{M}$ does my event $\mathcal{E}$ look like?"

$\theta_{\mathcal{M}}(\mathcal{E})$ answers: "Which shape in $\mathcal{M}$ does my event $\mathcal{E}$ look like?"

Can define complex manifolds to probe increasingly subtle geometric structure!

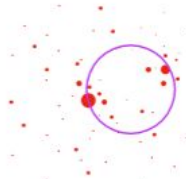
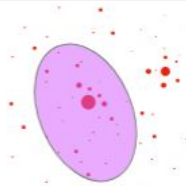


Disk + δ-function



Ring + δ-function

Probing **collinear** (δ-function) and **soft** (shape) structure!

Shape	Specification	Illustration
Ringiness $\mathcal{O}_R$	**Manifold of Rings** $\mathcal{E}_{x_0,R_0}(x) = \frac{1}{2\pi R_0}$ for $\lvert x - x_0\rvert = R_0$ x_0 = Center, R_0 = Radius	
Diskiness $\mathcal{O}_D$	**Manifold of Disks** $\mathcal{E}_{x_0,R_0}(x) = \frac{1}{\pi R_0^2}$ for $\lvert x - x_0\rvert \leq R_0$ x_0 = Center, R_0 = Radius	
Ellipsiness $\mathcal{O}_E$	**Manifold of Ellipses** $\mathcal{E}_{x_0,a,b,\varphi}(x) = \frac{1}{\pi ab}$ for $x \in \text{Ellipse}_{x_0,a,b,\varphi}$ x_0 = Center, a, b = Semi-axes, φ = Tilt	
(Ellipse Plus Point)iness	**Composite Shape** $\mathcal{O}_E \oplus \tau_1$ Fixed to same center x_0	
N-(Ellipse Plus Point)iness Plus Pileup	**Composite Shape** $N \times (\mathcal{O}_E \oplus \tau_1) \oplus \mathcal{I}$	

Some examples of new shapes you can define!

[see also L., B. Nachman, A. Schwartzman, C. Stansbury, 1509.02216;
see also B. Nachman, P. Nef, A. Schwartzman, M. Swiatlowski, C. Wanotayaroj, 1407.2922;
see also M. Cacciari, G. Salam, 0707.1378]

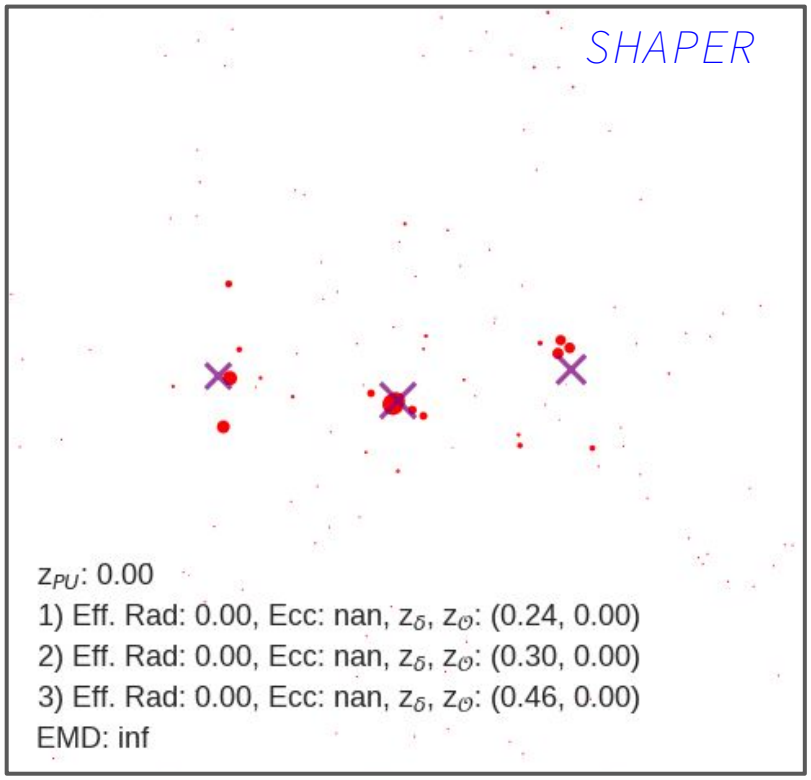
New IRC-Safe Observables

The *SHAPER* framework makes it easy to algorithmically invent new jet observables!

e.g. ***N-(Ellipse+Point)iness+Pileup*** as a jet algorithm:

- Learn jet centers + collinear radiation
- Dynamic jet radii (no *R* parameter!)
- Dynamic eccentricities and angles
- Dynamic jet energies
- Dynamic pileup (no z_{cut} parameter!!!)
- Learned parameters for discrimination

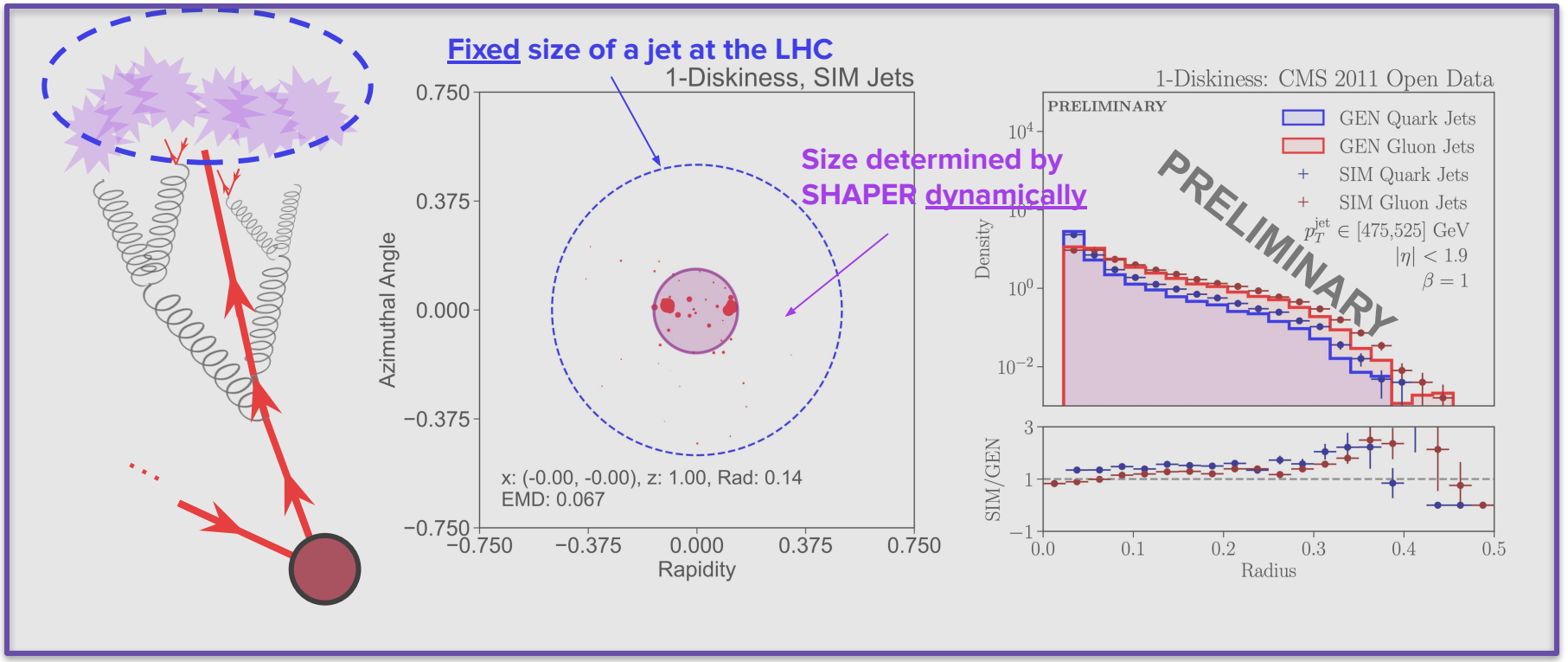
Can design custom specialized jet algorithms to learn jet substructure!



[RG, Thaler, Wu; WIP]

Back to our question: How Big are Jets?

Slide images from [Wu; APS 2023]



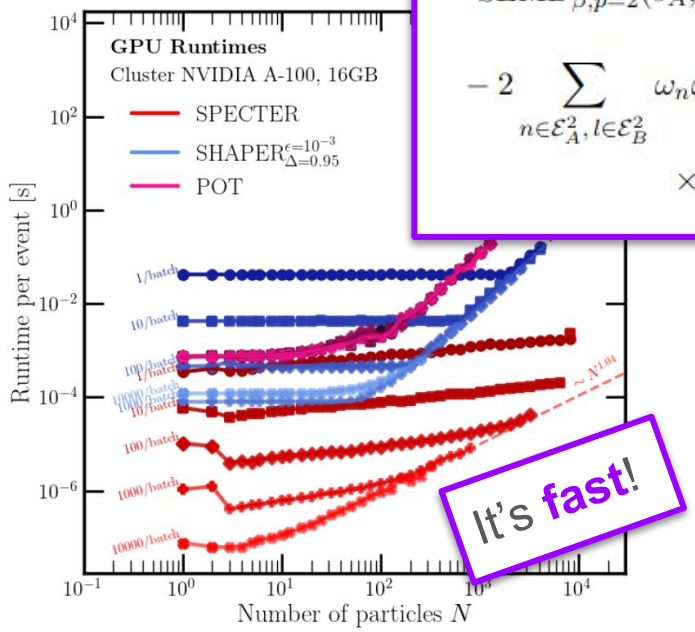
We can now answer this question in a precise way!

[RG, Larkoski, Thaler; 2410.05379]

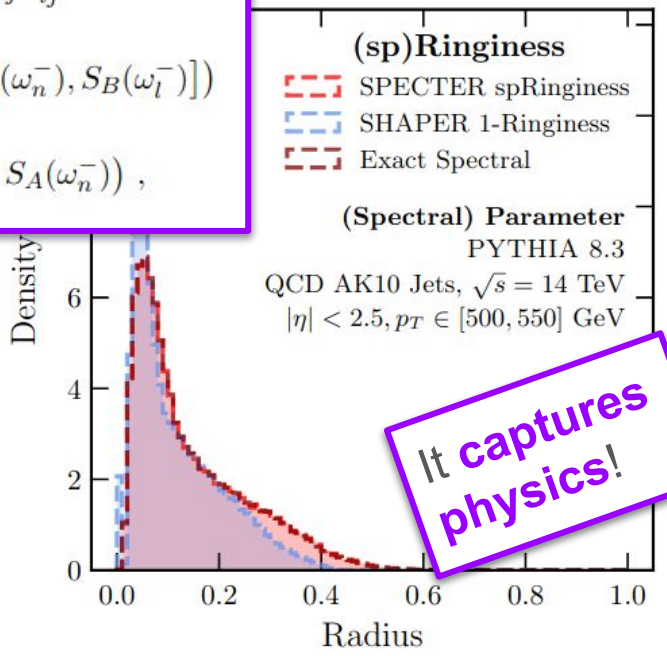
SPECTER: We can do more with OT!

By demanding our loss is only *almost** always a metric on events rather than always a metric, we can build a new loss that is *completely and exactly solvable*!

$$\mathrm{SEMD}_{\beta,p=2}(s_A, s_B) = \sum_{i<j\in\mathcal{E}_A} 2E_iE_j\omega_{ij}^2 + \sum_{i<j\in\mathcal{E}_B} 2E_iE_j\omega_{ij}^2$$
$$- 2 \sum_{n\in\mathcal{E}_A^2, l\in\mathcal{E}_B^2} \omega_n\omega_l \left(\min\left[S_A(\omega_n^+), S_B(\omega_l^+)\right] - \max\left[S_A(\omega_n^-), S_B(\omega_l^-)\right]\right)$$
$$\times\, \Theta\left(S_A(\omega_n^+) - S_B(\omega_l^-)\right)\Theta\left(S_B(\omega_l^+) - S_A(\omega_n^-)\right) ,$$



It's **fast**!



It **captures physics**!

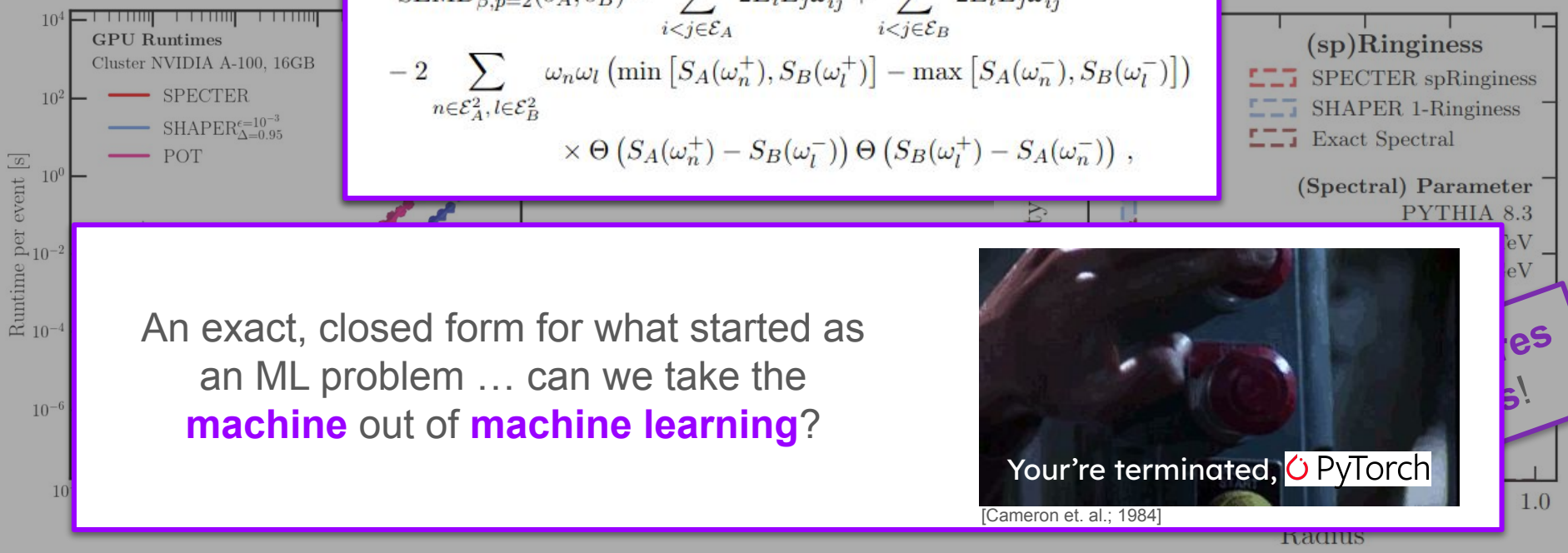
*By *almost*, I mean that there is a measure-0 set of events for which $d(x,y) = 0$ does not imply $x = y$
Ask me later about connections to EECs!

[RG, Larkoski, Thaler; 2410.05379]

SPECTER: We can do more with OT!

By demanding our loss is only *almost*[*] always a metric on events rather than always a metric, we can build a new loss that is *completely solvable*!

$$\mathrm{SEMD}_{\beta,p=2}(s_A, s_B) = \sum_{i<j\in\mathcal{E}_A} 2E_iE_j\omega_{ij}^2 + \sum_{i<j\in\mathcal{E}_B} 2E_iE_j\omega_{ij}^2$$
$$- 2 \sum_{n\in\mathcal{E}_A^2, l\in\mathcal{E}_B^2} \omega_n\omega_l \left(\min\left[S_A(\omega_n^+), S_B(\omega_l^+)\right] - \max\left[S_A(\omega_n^-), S_B(\omega_l^-)\right]\right)$$
$$\times\, \Theta\left(S_A(\omega_n^+) - S_B(\omega_l^-)\right)\Theta\left(S_B(\omega_l^+) - S_A(\omega_n^-)\right) ,$$



An exact, closed form for what started as an ML problem … can we take the **machine** out of **machine learning**?

[Cameron et. al.; 1984]

[*]By *almost*, I mean that there is a measure-0 set of events for which $d(x,y) = 0$ does not imply $x = y$
Ask me later about connections to EECs!

[RG, Larkoski, T

***SPECTER*: We can do more with OT!**

By demanding our loss is only *almost*[*] always a metric on events rather than always

End of **Part II**

The **Moment of Clarity**

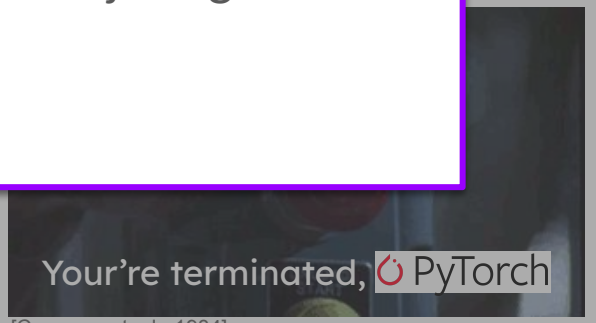
By choosing:

1. A loss that guarantees IRC safety and respects geometry
2. Parametrized distribution spaces (shapes)

We can define new meaningful observables and jet algorithms that are (in-principle) calculable in QCD

[*]Ask me later about theory space and beta functions!

machine out of **machine learning**?



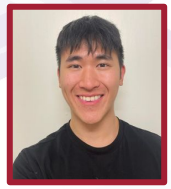
[Cameron et. al.; 1984]

[*]By *almost*, I mean that there is a measure-0 set of events for which $d(x,y) = 0$ does not imply $x = y$
Ask me later about connections to EECs!

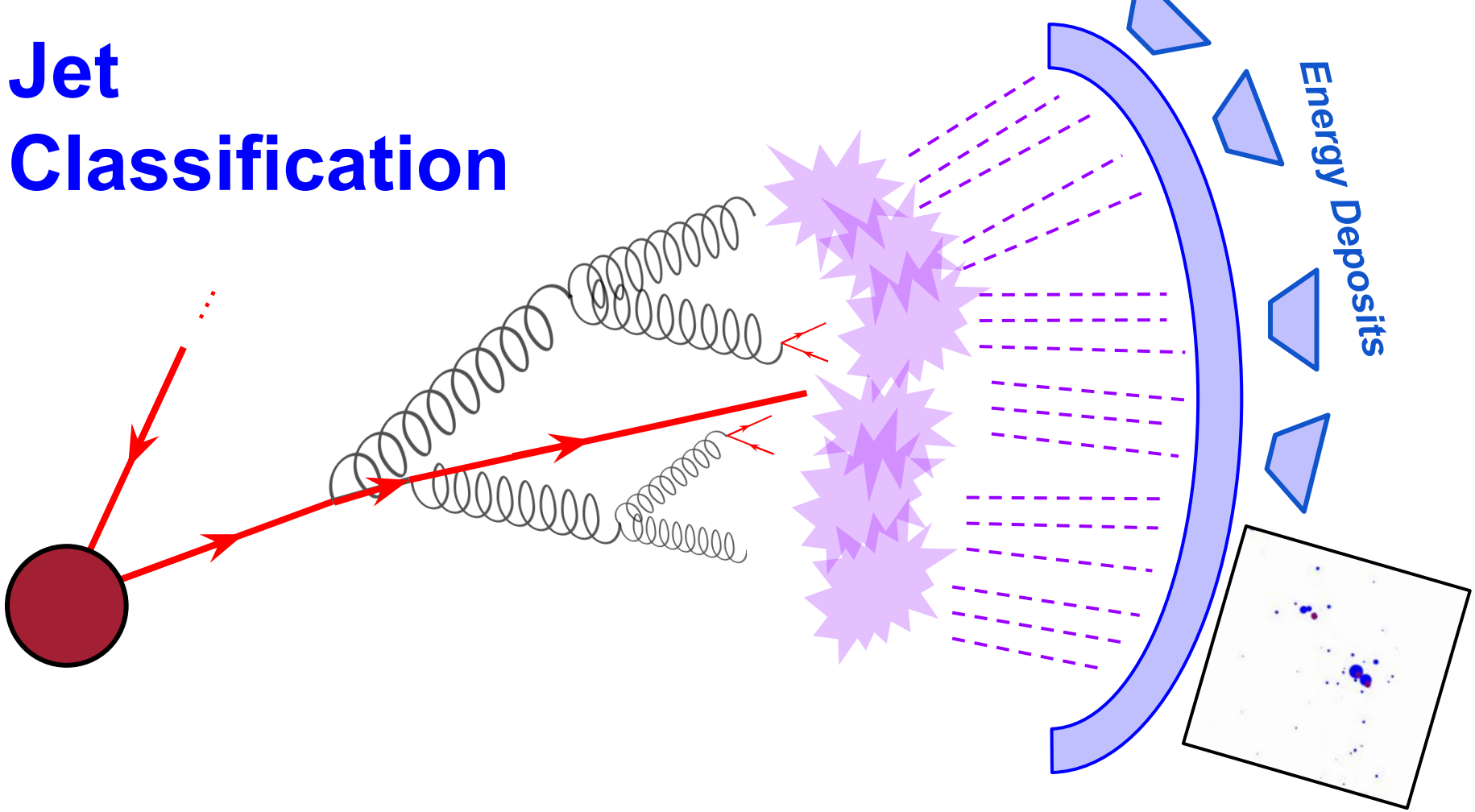
Part III

Moments of Clarity: Streamlining Latent Spaces

Download
Moment EFNs!


Jet Classification

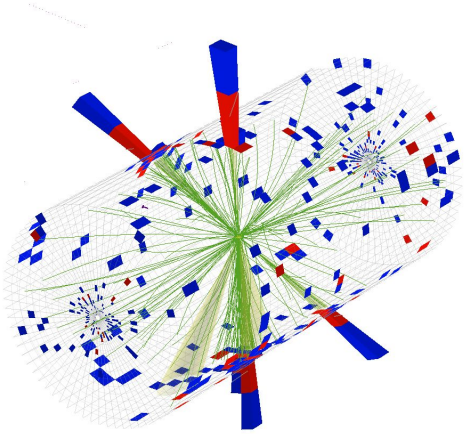


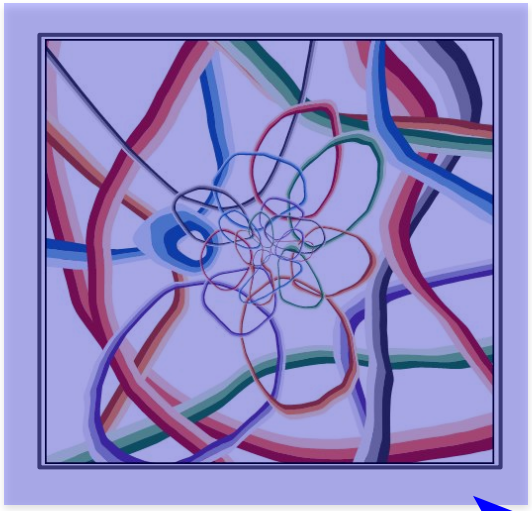
Given the final measured **point cloud**, was the **initiating parton** a **quark** or a **gluon**?

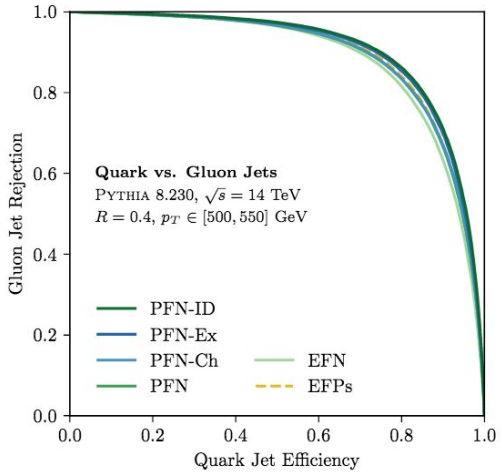
A staple machine learning task!

[Komiske, Metodiev, Thaler, 1810.05165]

Typical Machine Learning Setup







Collider Data → **Latent Space** → **Observables**

~1000 Dimensional | ~10-100 Dimensional | ~1-10 Dimensional

Pictured: An Energy Flow Network (**EFN**):

$$\mathcal{O}(\{p_1, \ldots, p_M\}) = F\left(\sum_{i=1}^{M} z_i \Phi(\hat{p}_i)\right)$$

(How) can we understand and constrain this?

(How) can we be more efficient?

Deep Learning on Point Clouds

The **Deep Sets Theorem** tells us how to parameterize functions on point clouds*:

Set of momenta

a = 1 … *L*, the *Latent Dimension* index

$$\mathcal{O}(\mathcal{P}) = F\left(\left\langle \phi^a \right\rangle_{\mathcal{P}}\right)$$

$$\langle \phi \rangle_{\mathcal{P}} \equiv \sum_i z_i \phi(\hat{p}_i)$$

The Deep Sets Theorem guarantees that any function on sets $\mathscr{P}$ can be written this way, for "sufficiently complex" *F* and ϕ and large enough *L*.

For this talk, I will focus primarily on Energy Flow Networks (EFNs) where the sums are energy-weighted.

*More sophisticated architectures still reduce to Deep Sets in some limit, e.g. graph networks

Deep Learning on Point Clouds

The **Deep Sets Theorem** tells us how to parameterize functions on point clouds*:

Set of momenta

a = 1 … *L*, the *Latent Dimension* index

$$\mathcal{O}(\mathcal{P}) = F\left(\left\langle \phi^a \right\rangle_{\mathcal{P}}\right)$$

$$\langle \phi \rangle_{\mathcal{P}} \equiv \sum_i z_i \phi(\hat{p}_i)$$

The Deep Sets Theorem guarantees that any function on sets $\mathscr{P}$ can be written this way, for "sufficiently complex" *F* and ϕ and large enough *L*.

But how complex do *F* and ϕ need to be? Can I reduce the number of latent dimensions?

For this talk, I will focus primarily on Energy Flow Networks (EFNs) where the sums are energy-weighted.

*More sophisticated architectures still reduce to Deep Sets in some limit, e.g. graph networks

Moment Pooling

$$\mathcal{O}(\mathcal{P}) = F\left(\langle \phi^a \rangle_{\mathcal{P}}\right)$$

Generalize to more moments! "**Moment Pooling**"

k = Highest order moment considered

$$\mathcal{O}_k(\mathcal{P}) = F_k\left(\underbrace{\langle \phi^a \rangle_{\mathcal{P}}}_{1^{\text{st}}\text{ Moment}}, \underbrace{\langle \phi^{a_1}\phi^{a_2} \rangle_{\mathcal{P}}}_{2^{\text{nd}}\text{ Moment}}, \ldots, \underbrace{\langle \phi^{a_1}\ldots\phi^{a_k} \rangle_{\mathcal{P}}}_{k^{\text{th}}\text{ Moment}}\right)$$

$$L_{\text{eff}} = \binom{L+k}{k}$$

Moment Pooling – Why?

k = Highest order moment considered

$$\mathcal{O}_k(\mathcal{P}) = F_k\left(\underbrace{\langle \phi^a \rangle_{\mathcal{P}}}_{1^{\text{st}}\text{ Moment}}, \underbrace{\langle \phi^{a_1}\phi^{a_2} \rangle_{\mathcal{P}}}_{2^{\text{nd}}\text{ Moment}}, \ldots, \underbrace{\langle \phi^{a_1}\ldots\phi^{a_k} \rangle_{\mathcal{P}}}_{k^{\text{th}}\text{ Moment}}\right)$$

$$L_{\text{eff}} = \binom{L+k}{k}$$

- **Explicit Multiplication**: Neural nets are mostly piecewise linear! But most functions we learn aren't. Moments are just multiplication, but for distributions!
- **Latent Space Compression**: For large L, there are effectively L^k latent dimensions due to combinatorics, but still made of only L functions! Fewer latent dimensions means easier analysis!

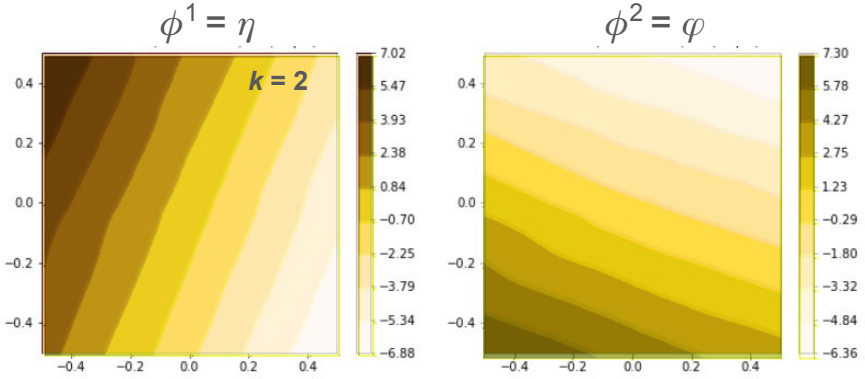
This is actually a much simpler version of the attention mechanism. It turns out ***you don't even need attention*** – ask me later!

Multiplication is all You Need

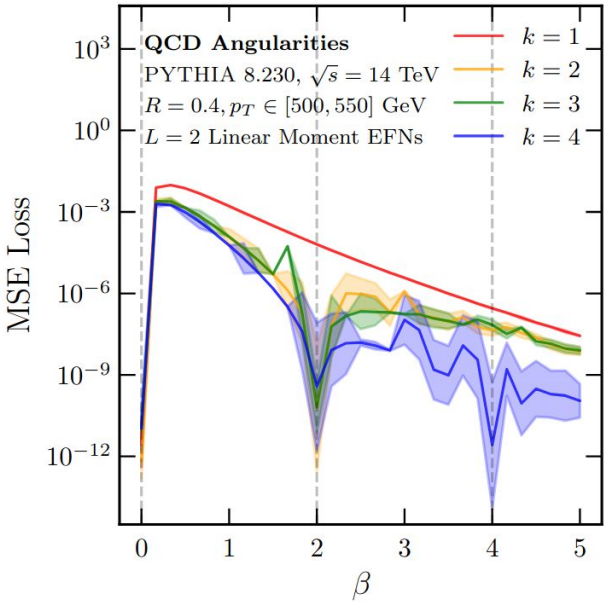
In the moment language, even integer* β jet angularities ⇔ $k = \beta^{\text{th}}$ moments!

$$\lambda^{(\beta)}(\mathcal{P}) = \sum_i z_i \left(\eta_i^2 + \phi_i^2\right)^{\beta/2}$$
$$= \left\langle \eta^\beta \right\rangle_{\mathcal{P}} + \left\langle \phi^\beta \right\rangle_{\mathcal{P}} + \text{ Cross Moments}$$

Learns the simplest latent representations!

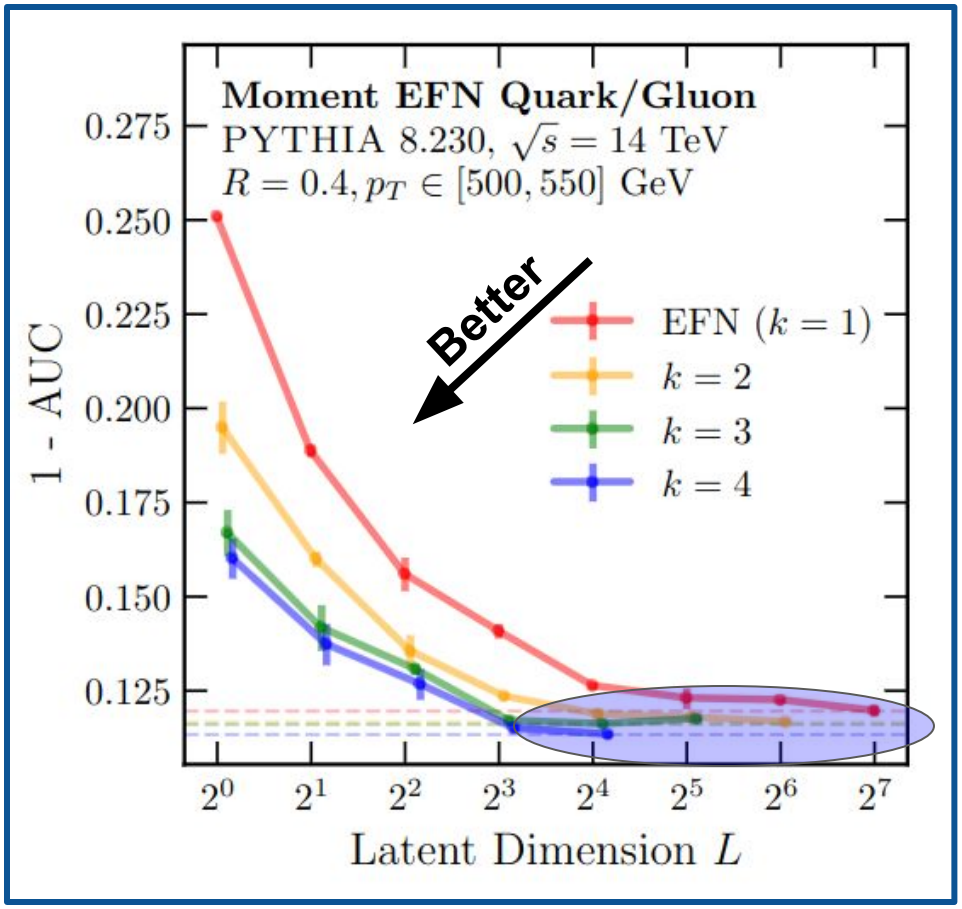


Encourage learning a diagonal basis in (η,φ) using L1 Regularization on *F*



The lesson: Basic observables are naturally represented with multiplication, so build it in!

Moment Pooling Results

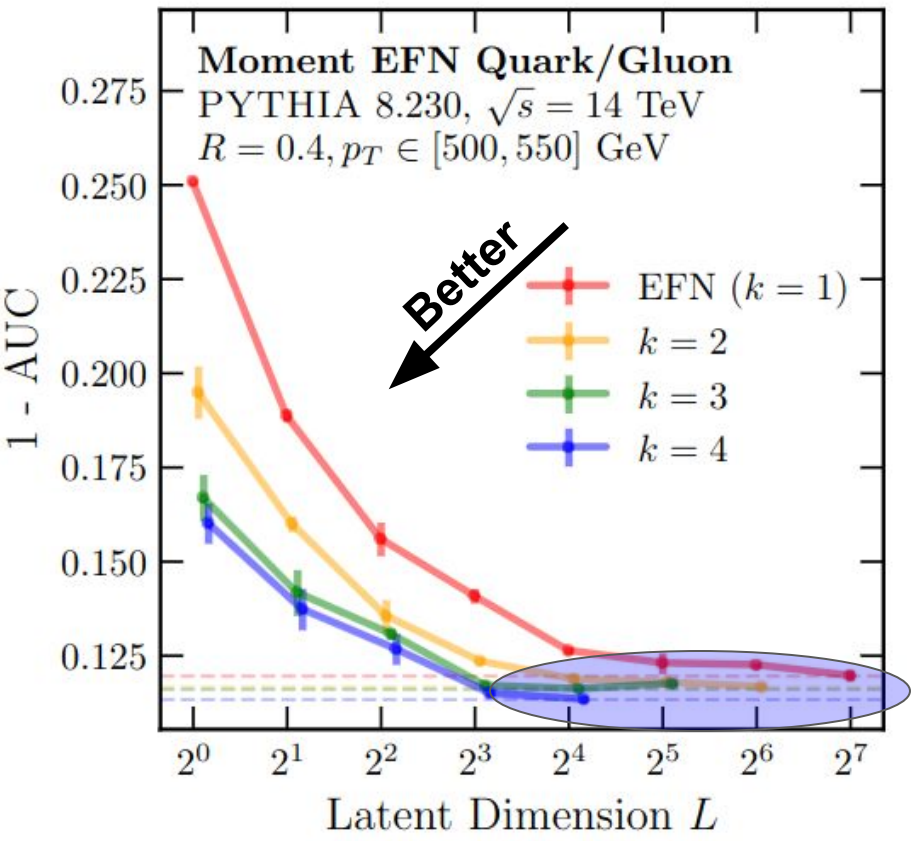


A high order **Moment EFN** can achieve the same top performance on quark/gluon classification as an **ordinary EFN**, but with far fewer latent dimensions!*

A Moment EFN is a more efficient encoding of the same data!

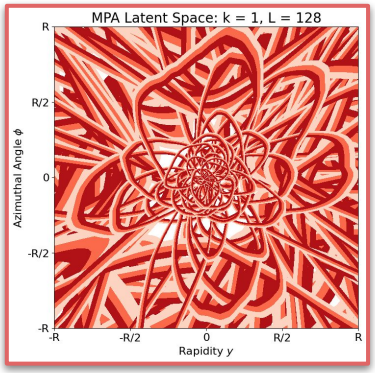
*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

The Black Box

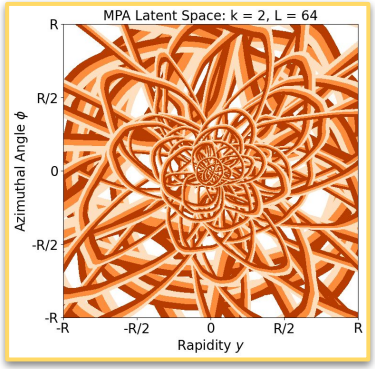


Latent Spaces

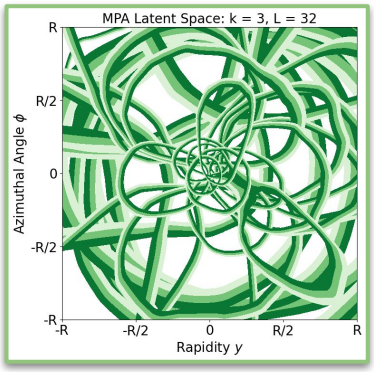
EFN (*k = 1*)
L = 128



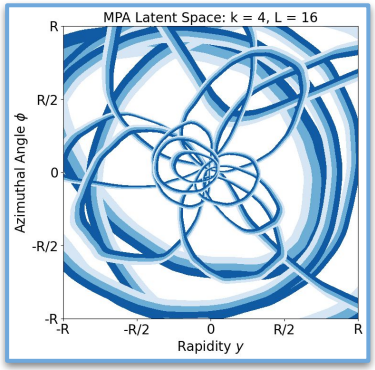
k = 2
L = 64



k = 3
L = 32



k = 4
L = 16



The 45% - 55% contours of each of the *L* different ϕ functions

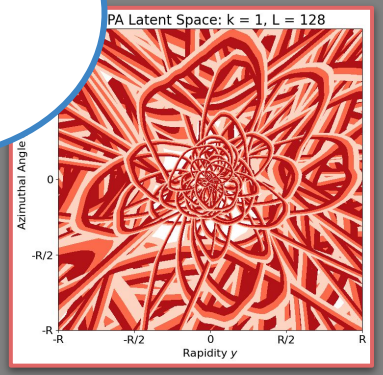
*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

Latent Spaces

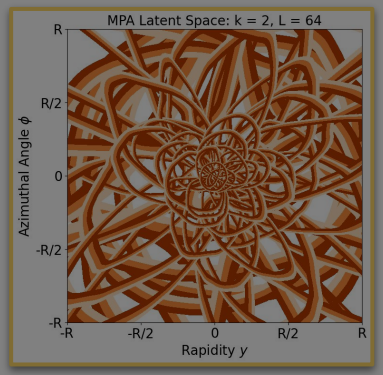
So what? The information is just being shuffled around. Is 16 latent dimensions any more interpretable than 128?

The machine probably learned something, but I didn't!

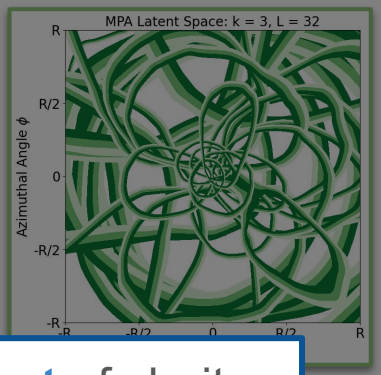
EFN ($k = 1$)
$L = 128$



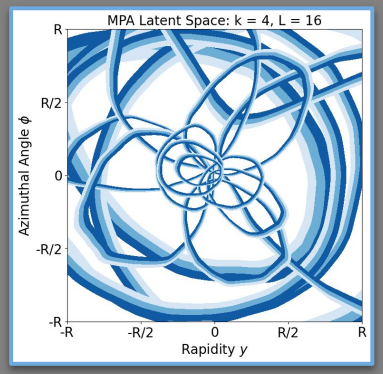
$k = 2$
$L = 64$



$k = 3$
$L = 32$



$k = 4$
$L = 16$



Me, just before the **moment** of clarity.

...rs of each of the L different ϕ functions

*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

Latent Spaces

So what? The information is just being shuffled around. Is 16 latent dimensions any more interpretable than 128?

The machine probably learned something, but I didn't!

I'm allowed to control the ML function space!

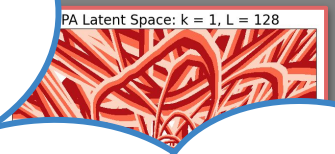
Let me just choose one that's easy for me to understand: **One dimensional functions**

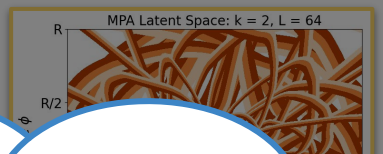
Because of latent space compression, even one dimension can contain a lot of info!

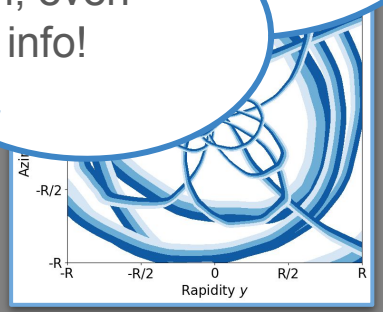
Me, just before the **moment** of clarity.

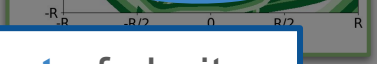
EFN ($k = 1$)
$L = 128$

$k = 2$
$L = 64$

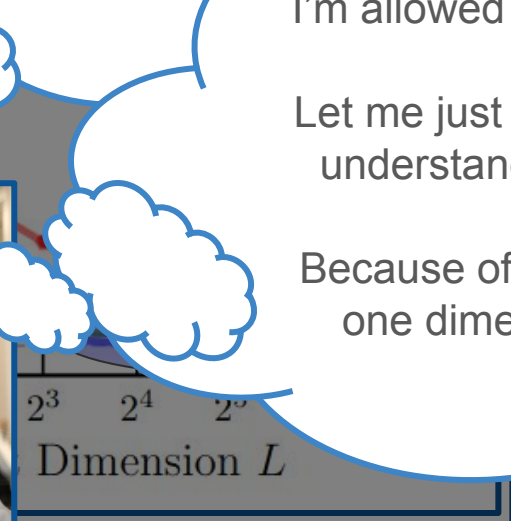






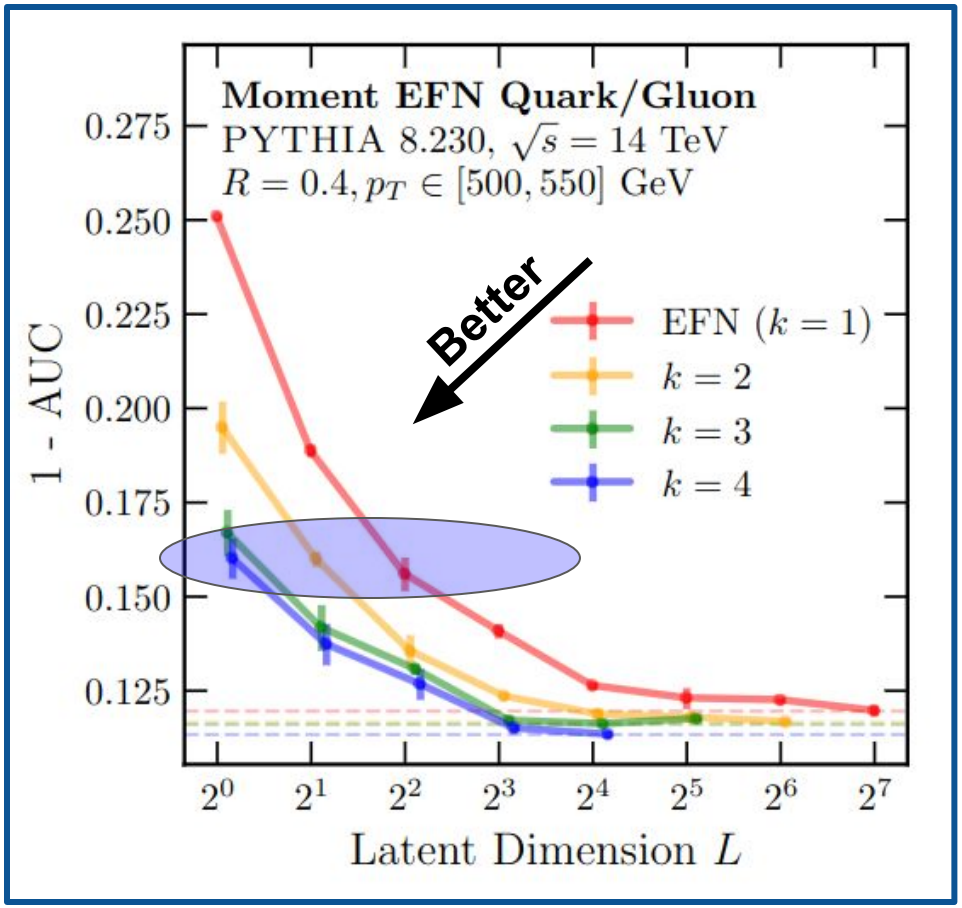


rs of each of the L different ϕ functions



*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

Moment Pooling Results (Cont'd)



Let's instead look at networks with just a *single* latent dimension.

A ***k = 4* Moment EFN** with *just one* latent dimension does just as good as an **ordinary EFN** with *four* latent dimensions!

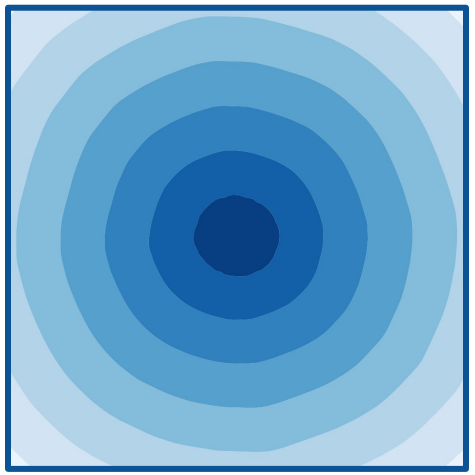
Going down to *L = 1*

The more efficient encoding is extremely useful when we can get to *L = 1*!
Look how much simpler allowing neural networks to multiply makes things!

EFN (*k = 1*)
L = 4

Same information!

Order *k = 4*
L = 1



- 4 Different Functions
- Combined in a complicated way
- No symmetry
- **???**

- Single function
- Expected to be, and is, radially symmetric
- We can **interpret** this!

The Moment of Truth

We can extract physics from this!

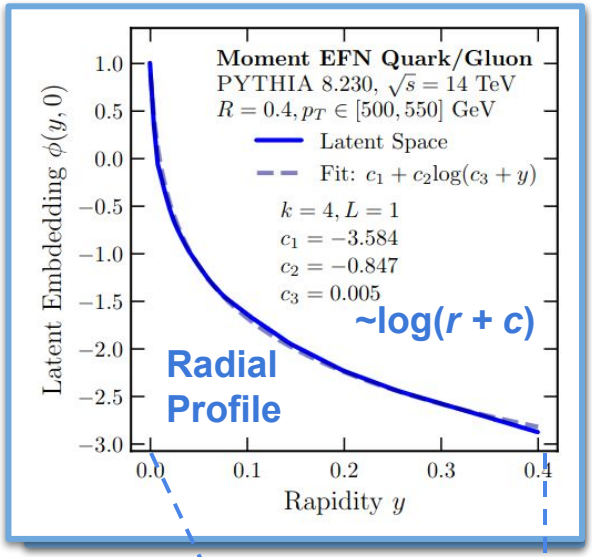
$$\Phi_{\mathcal{L}}(r) = c_1 + c_2 \log(c_3 + r)$$

This defines a **log angularity** observable, related to the $\square \to 0$ limit of ordinary angularities

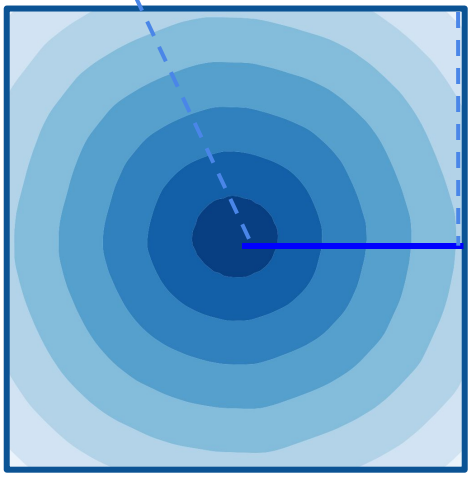
We can even see non-perturbative physics!

$$c_3 \sim \frac{\Lambda_{\mathrm{QCD}}}{p_T R} \sim 0.001$$

Just this alone is enough to get an IRC-safe quark-gluon classifier with an AUC ~ 0.83!



Order *k* = 4 *L* = 1

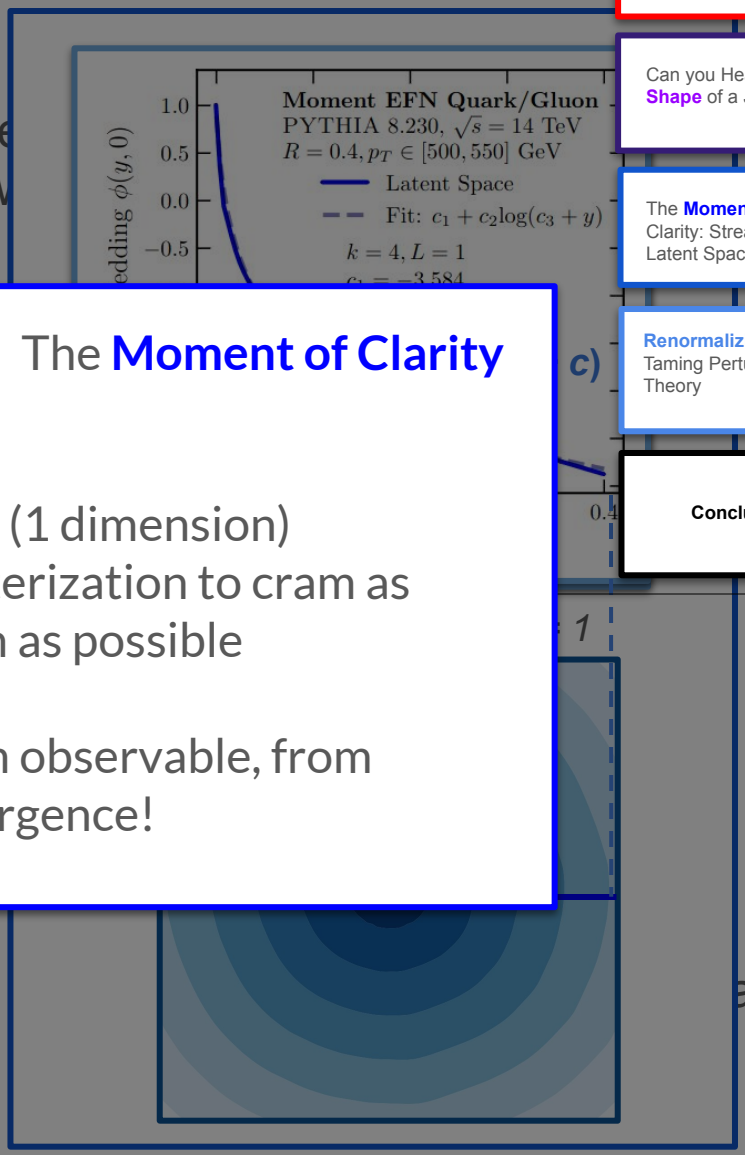


The Moment of Truth

We can extract physics from this!

$$\Phi_C(r) = c_1 + c_2 \log(c_3 + r)$$

Just this alone is enough to get an IRC-safe quark-gluon classifier with an AUC ~ 0.83!



End of **Part III**

The **Moment of Clarity**

By choosing:

1. A human-readable function space (1 dimension)
2. A multiplication-based reparameterization to cram as much info into that one dimension as possible

We can construct a simple closed-form observable, from which we can even read off the IR divergence!

Part IV: Re-Normalizing Flows

Taming Logs and Perturbation Theory in QCD

Perturbation Theory in QCD

Problem: Fixed-order perturbation theory in QCD generically suffers from non-physical issues that do not occur in real life!

1. Non-finite
2. Non-smooth
3. Non-positive
4. Non-normalizable
5. Non-converging

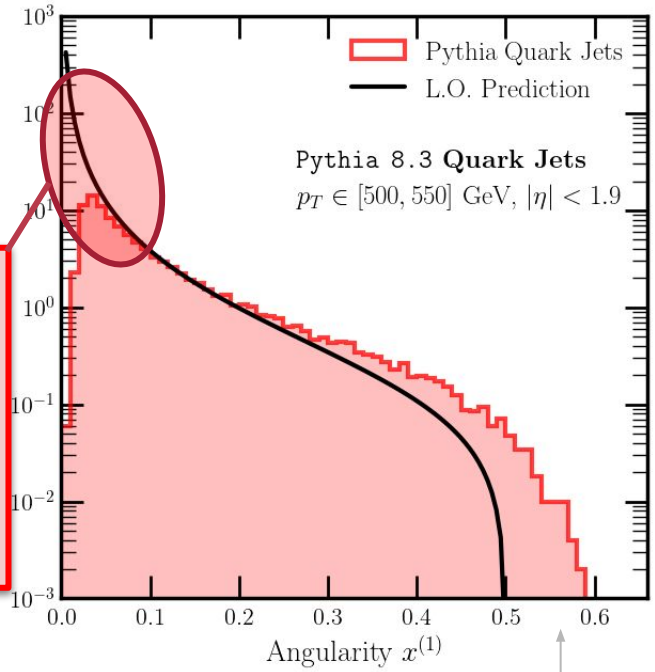
Logs blow up! This distribution is not normalized*, and convergence is spoiled when *x* is small!

Jet Angularity: $x^{(\beta)} = \sum_i z_i \left(\frac{\theta_i}{R}\right)^\beta$

$$p(x^{(1)}) = \frac{\alpha_s(xE_0)C_F}{2\pi}\int_0^R \frac{d\theta}{\theta}\int_0^1 dz\, P_{q\to qg}(z)\delta(x - z\frac{\theta}{R})$$

$$\sim \frac{\alpha_s(xE_0)C_F}{2\pi}\frac{R\log\left(\frac{R}{x}\right)}{x}$$

Fixed Order vs. Real*



This difference is a genuine N.L.O effect, due to the definition of jet axis through the WTA algorithm.

*Technically, these are *plus Distributions*, which are normalized. However, these are invisible to any real value of *x*, are highly non-smooth, and can be negative.

Perturbation Theory in QCD

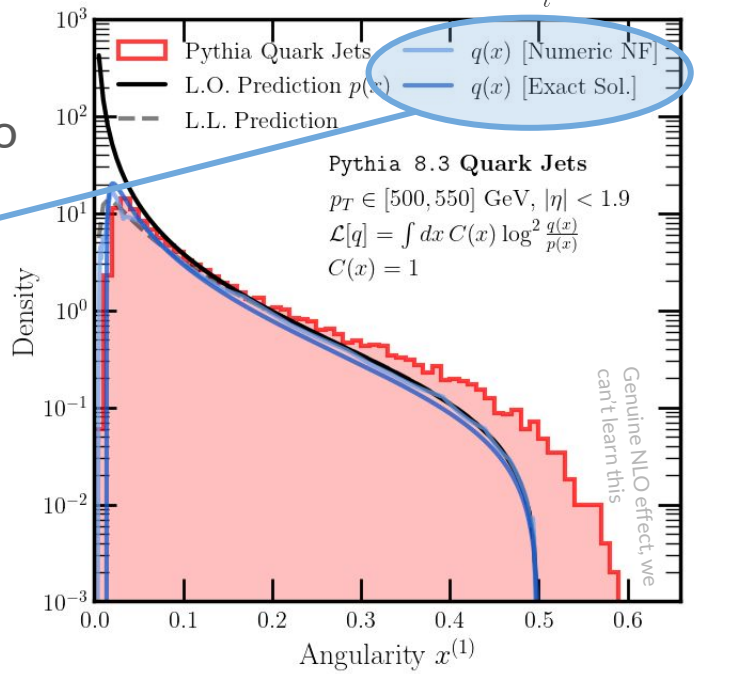
Jet Angularity: $x^{(\beta)} = \sum_i z_i \left(\frac{\theta_i}{R}\right)^{\beta}$

Problem: Fixed-order perturbation theory in QCD generically suffers from non-physical issues that do not occur in real life!

1. Non-finite → **Finite**!
2. Non-smooth → **Smooth**!
3. Non-positive → **Positive**!
4. Non-normalizable → **Normalized**!
5. Non-converging

I won't solve this one today, sorry!

Guaranteed by NF



Solution: *Force* (1)-(4) by matching the perturbation theory to a **Normalizing Flow (NF)**, which parametrizes the set of *all* finite, smooth, positive, normalizable functions, *while preserving the perturbative structure in* α_s!

Original, badly behaved prediction $p(x)$

New, well behaved prediction $q(x)$

Parametrized by NF

"Matching" Loss Functional $\mathcal{L}[q]$

The large logs have been tamed!

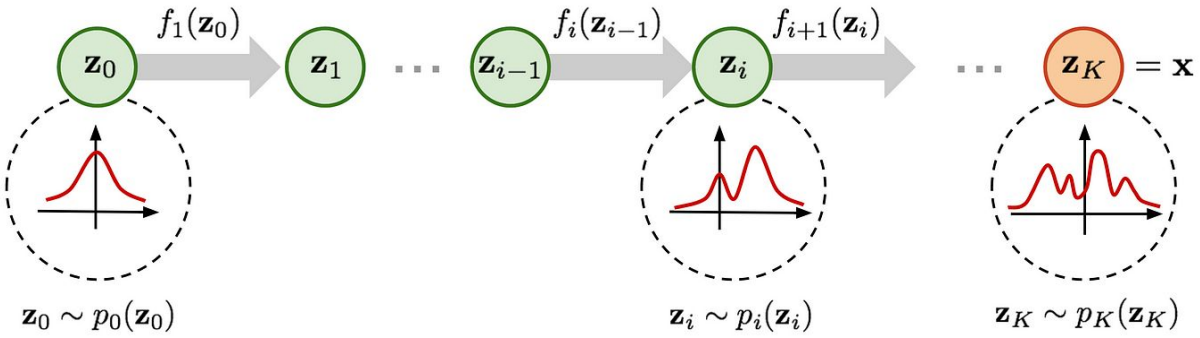
[Beate Sick, Hothorn, Dürr; 2004.00464]
[Ramasinghe, Fernando, Khan, Barnes; 2102.03509]
[Image from: Medium]

[Aside, if there's time] Normalizing Flows

A **Normalizing Flow** is a parameterized function *q(x)* that is *guaranteed* to be a valid probability distribution and can be easily sampled from.

This is accomplished by taking an *already known* probability distribution, $q_0(z)$, then parameterizing a transformation $x = f(z)$ that is invertible, has tractible Jacobians, and is easy to compose:

$$p_X(x) = p_Z(f^{-1}(x))\left|\det \frac{\partial f}{\partial z}\right|^{-1}_{z=f^{-1}(x)}$$



We choose to use *Bernstein-Polynomial Flows (BPFs)*, which we have found to be a nice, stable parameterization for low-dimensional flows.

Really, any method of learning distributions would work for our purposes! But NFs and in particular BPFs are easy.

Loss Matching

Given *p*, we want to find a *q* that still retains the salient physics of *p* – for example, matching the perturbative expansion of *p*, or matching the regions of *p* where the logs are not too large.

Encode this in a *Loss Functional*:

f is a choice. Here, we will pick *f* = *linear* or *log*.
This tells us in what space our "errorbars" are Gaussian in!

The dagger matters; *f* might be complex! See Backup Slides.

$$\mathcal{L}[q] = \int dxdy\,[f(q(x)) - f(p(x))]^\dagger \left[\frac{1}{2C^2(x,y)}\right] [f(q(y)) - f(p(y))]$$

Gaussian Kernel: Tells us our tolerable "Gaussian Error" on matching!
Choose this to regulate divergences.

$\frac{1}{2C^2(x,y)}$ can be anything, *especially* a differential operator, but for convenience we will often choose a diagonal and real kernel:

$$\mathcal{L}[q] = \int dx\,\frac{|f(q) - f(p)|^2}{2C^2(x)}$$

Why this loss? Most other losses can be encoded this way! Non-uniform sampling can also be built into *C*

*Almost everywhere.

Loss Matching – Exact Solutions

For *f* = log or linear and real-valued choices of *C*, we can directly solve for the optimal normalized, smooth*, finite, positive *q* by using Euler-Lagrange to minimize the MSE loss!

f = linear

$$q(x) = \mathrm{ReLU}\left(p(x) - \lambda C^2(x)\right)$$

f = log

$$q(x) = p(x)\exp[-W(\lambda p(x) C^2(x))]$$

Such that λ solves $\int dx\, q(x) = 1$

q(x) is a second-order correction to *p(x)*, especially if $C^2 \sim c(x)\alpha^2 + \ldots$

q(x) is an all-orders resummation of *p(x)*, especially if $C^2 \sim c_0(x) + c_1(x)\alpha + c_2(x)\alpha^2 + \ldots$

For general *f*: $$q(x) = \mathrm{ReLU}\left[f^{-1}\left(f(p(x)) - \frac{\lambda C^2(x)}{f'(p(x))}\right)\right]$$

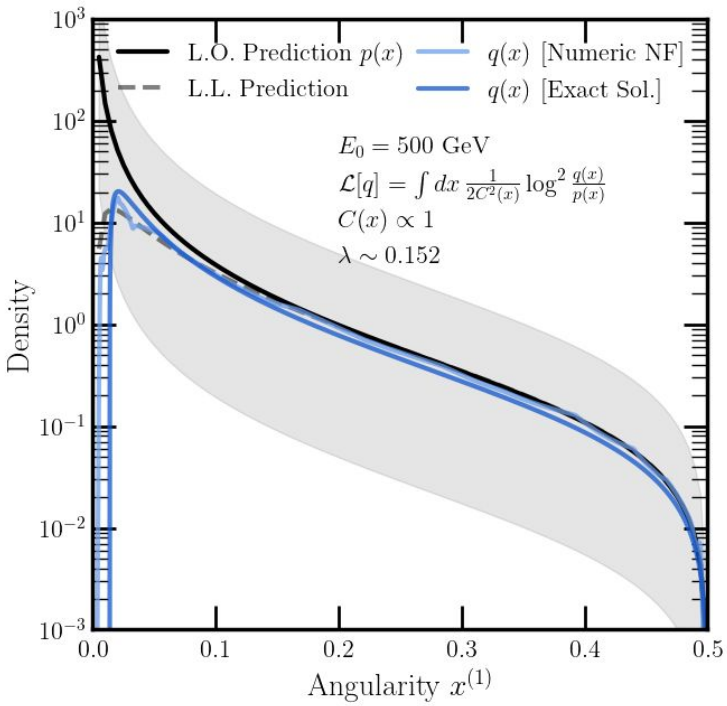
If we have exact solutions, why bother with *NF*s? We might not have closed forms for *q*, we might be working in many dimensions, and solving for λ is numerically hard.

Taming Large Logarithms – Example

Let's pick f = log, $C(x)$ = 1 for the L.O. jet angularity!

$$
\begin{aligned}
q(x) &= p(x)\exp[-W(\lambda p(x) C^2(x))] \\
&= \left[\frac{\alpha_s C_F}{\pi}\frac{\log(1/x)}{x}\right]\exp\left[-W(\lambda\left[\frac{\alpha_s C_F}{\pi}\frac{\log(1/x)}{x}\right])\right] \\
&\approx \left[\frac{\alpha_s C_F}{\pi}\frac{\log(1/x)}{x}\right]\exp\left[-\frac{\lambda\alpha_s C_F}{\pi}\frac{\log(1/x)}{x} + \mathcal{O}(\alpha_s^2)\right] \\
&= p(x) + \mathcal{O}(\alpha_s^2)
\end{aligned}
$$

"Sudakov"-Like



Perturbative structure preserved at α!
Exponential suppression of the divergence!
Sudakov-like!

Due to running

Sudakov

$$
q^{LL}(x) = \left[\frac{\alpha_s C_F}{\pi}\frac{\log(1/x)}{x} + \frac{\beta_\alpha(\alpha)}{2\pi x}\log^2(1/x)\right]\exp\left[-\frac{\alpha_s C_F}{2\pi}\log^2(1/x) + \mathcal{O}(\alpha_s^2)\right]
$$

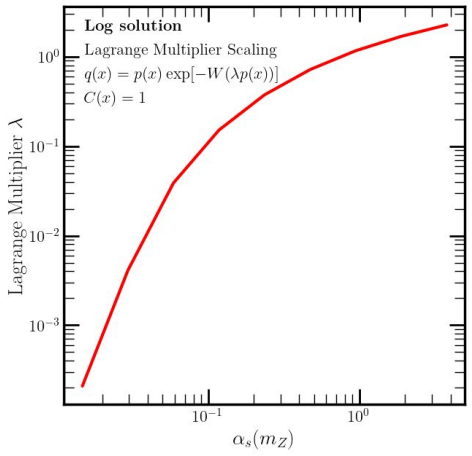
For comparison, the full L.L.

$C = 1$ is OK for f = log, but not linear. Ask me why later!

Taming Large Logarithms – Details

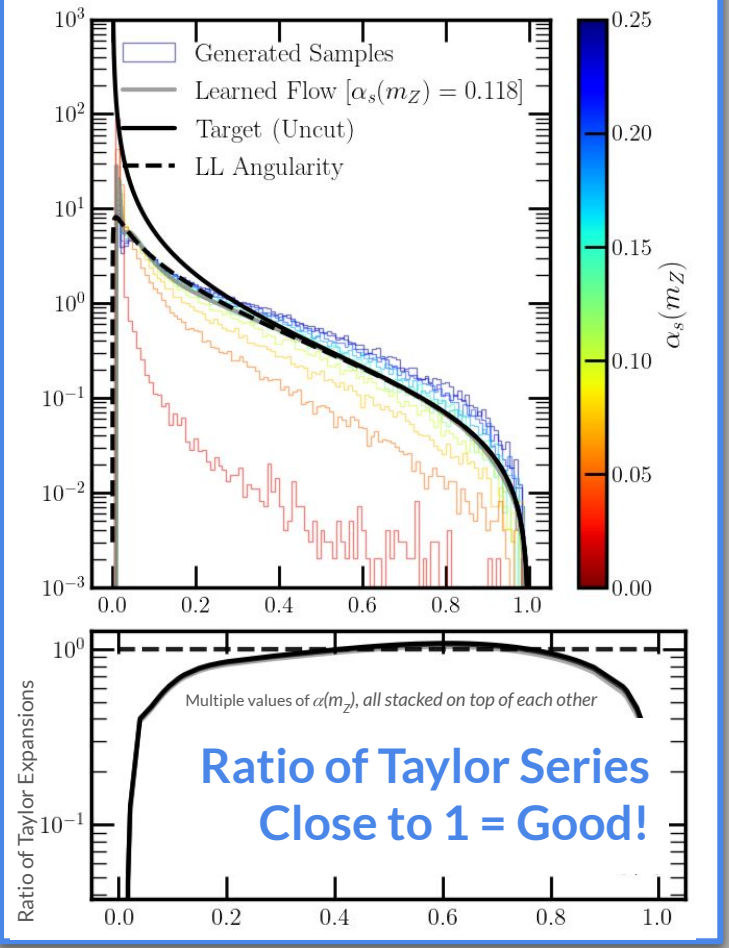
A catch! If the Lagrange Multiplier λ scales as α^{-1}, the perturative structure is doomed! Let's check.

$$
\begin{aligned}
q(x) &= p(x)\exp[-W(\lambda p(x) C^2(x))] \\
&= \left[\frac{\alpha_s C_F}{\pi}\frac{\log(1/x)}{x}\right]\exp\left[-W(\lambda\left[\frac{\alpha_s C_F}{\pi}\frac{\log(1/x)}{x}\right])\right] \\
&\approx \left[\frac{\alpha_s C_F}{\pi}\frac{\log(1/x)}{x}\right]\exp\left[-\frac{\lambda\alpha_s C_F}{\pi}\frac{\log(1/x)}{x} + \mathcal{O}(\alpha_s^2)\right] \\
&= p(x) + \mathcal{O}(\alpha_s^2)
\end{aligned}
$$



Numerically, we are *ok*!
We can say *numerically* that $C(x) = 1$ preserves perturbative structure at α^1!

If λ itself scales as α^1, we could go even further and say $C(x) = 1$ preserves α^2 for some *p*, but this numerically does not seem to be the case, it seems to be a nontrivial power law



For comparison, the full L.L.

Can we do All Orders?

Yes. Just use the Taylor Series Operator: Extremely numerically difficult (WIP), but progress has been made.

$$[2C(x)]^{-1} = \exp_N[\alpha \frac{d}{d\alpha}|_{\alpha=0}] = \sum_{n=0}^{N}[\frac{1}{n!}\alpha^n \frac{d^n}{d^n\alpha}|_{\alpha=0}]$$

You, the user, give us:

1. An expression $p^N(x|\alpha)$ representing a fixed-order expansion to order α^N.
   Or, samples x with weights auto-differentiable in α
2. A *promise* that $p^N(x|\alpha)$ is a fixed-order expansion of some "platonic" $p(x|\alpha)$ which is a non-pathological distribution.
   e.g. $p^N(x|\alpha)$ is an approximation to $p(x|\alpha)$ computed from the full QCD path integral

Ideally, we give you back:

A new distribution $q(x|\alpha)$ that is ***guaranteed*** to be normalized, smooth, positive, and finite, *and* **numerically** matches the perturbative expansion of $p^N(x|\alpha)$ to order N:

$$\exp[\alpha\frac{d}{d\alpha}]p(x) = p^0(x) + \alpha^1 q^1(x) + \ldots + \alpha^N p^N(x) + \mathcal{O}(\alpha^{N+1})$$
$$\exp[\alpha\frac{d}{d\alpha}]q(x) = q^0(x) + \alpha^1 q^1(x) + \ldots + \alpha^N q^N(x) + \mathcal{O}(\alpha^{N+1})$$

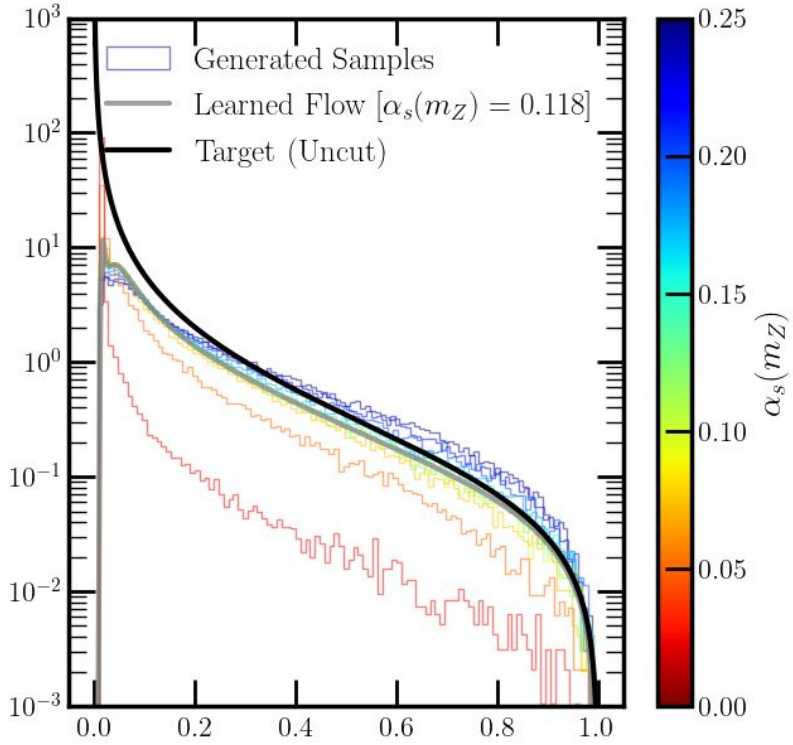
equal!

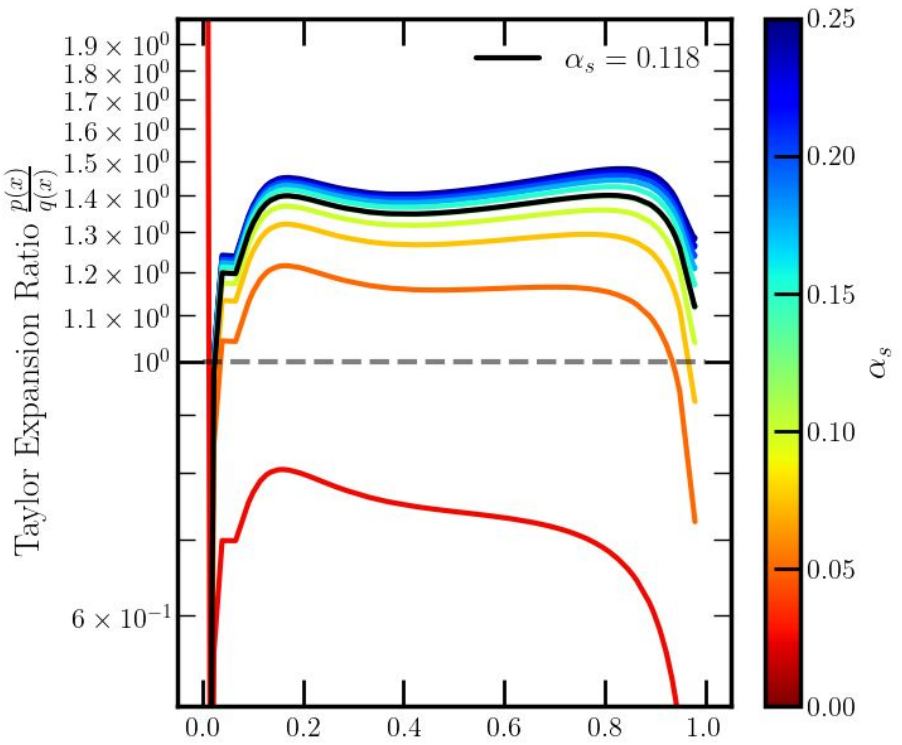
All of the information in $p(x)$ is preserved!

Why is requirement [2], the promise, necessary? Ask me later!

Proof of Concept

By playing lots of games with numeric regularization, we can get the Generic **Taylor Series Operator** to work to first order (at least, to 50% ...)





Proof of Concept

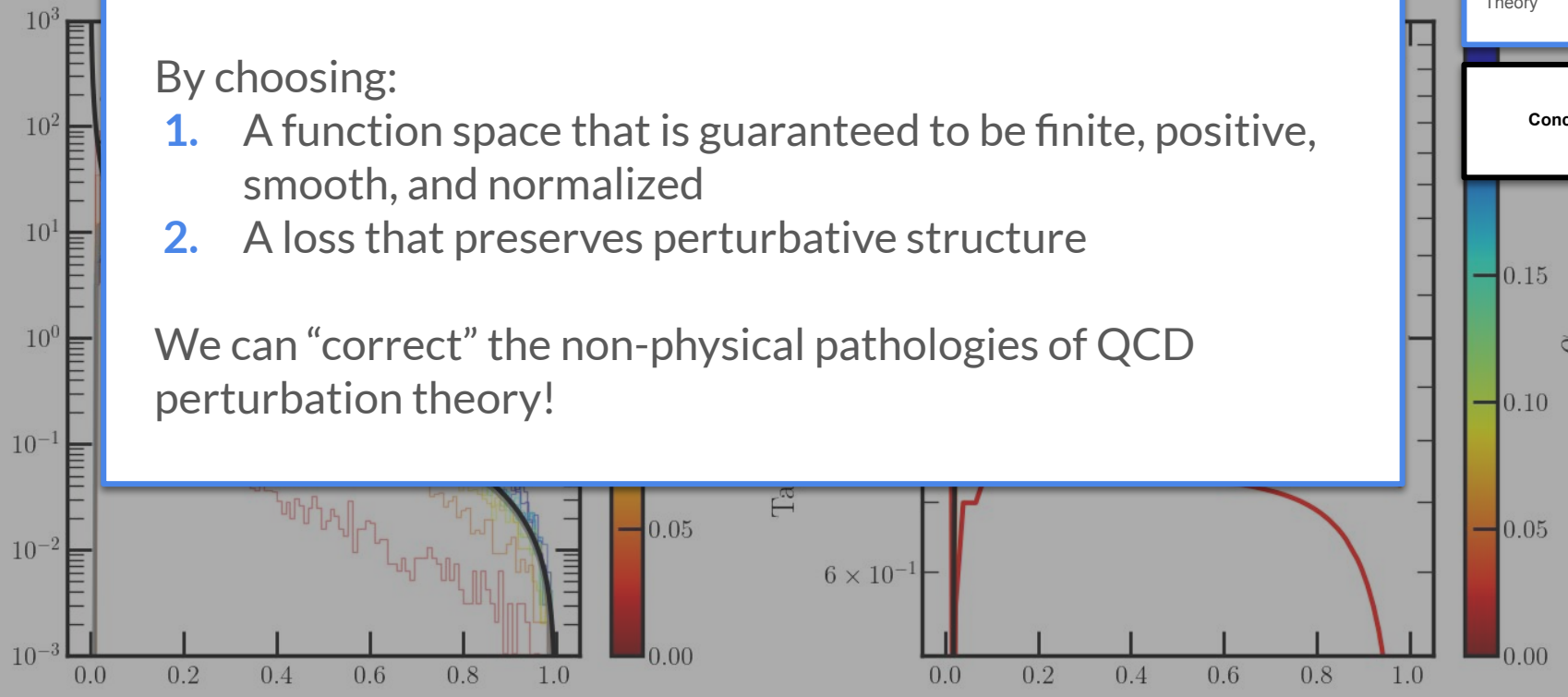
By playing lots of games with numeric regularization, we can get the Generic Taylor

End of **Part IV** The **Moment of Clarity**

By choosing:

1. A function space that is guaranteed to be finite, positive, smooth, and normalized
2. A loss that preserves perturbative structure

We can "correct" the non-physical pathologies of QCD perturbation theory!

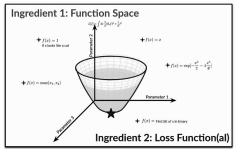


Part V: Conclusion

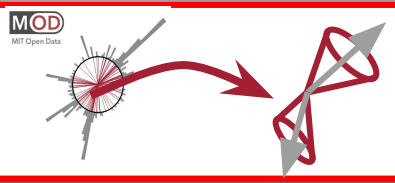
Even if you missed some details from each of the four vignettes I went over, I hope you took away:

- ML is *not* a black box, it's just numerical functional analysis.
- We can ***completely*** control what the ML does through the function space and loss functional – so physically meaningful results can be ***guaranteed***.
- We can use this control to solve jet physics problems involving subtle correlations, high dimensional structures, complicated simulations or nontrivial constraints *that we could not have solved (as optimally) before.*

Introduction



Learning **Uncertainties** the **Frequentist** Way

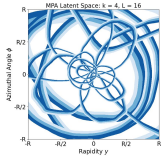


[**RG**, Nachman, Thaler, 2205.05084]
[**RG**, Nachman, Thaler, 2205.05084]

Can you Hear the **Shape** of a Jet?

[Ba, Dogra, **RG**, Tasissa, Thaler, 2302.12266]
[**RG**, Thaler, Wu, WIP]
[**RG**, Larkoski, Thaler, 2410.05379]

Moment of Clarity:
Streamlining Latent Spaces



[**RG**, Osathapan, Thaler, 2403.08854]

Renormalizing Flows:
Taming Perturbation Theory

[**RG**, Mastandrea, WIP]

$$\exp[\alpha\frac{d}{d\alpha}]p(x) = p^0(x) + \alpha^1 q^1(x) + \ldots + \alpha^N p^N(x) + \mathcal{O}(\alpha^{N+1})$$
$$\exp[\alpha\frac{d}{d\alpha}]q(x) = q^0(x) + \alpha^1 q^1(x) + \ldots + \alpha^N q^N(x) + \mathcal{O}(\alpha^{N+1})$$

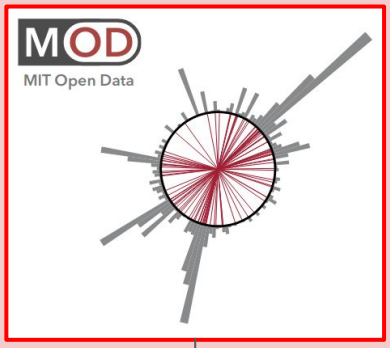
equal!

Part V [You are here!]

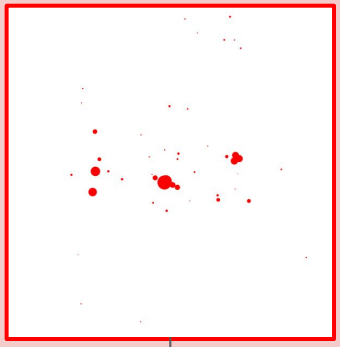
Conclusion

Things a **machine** can understand

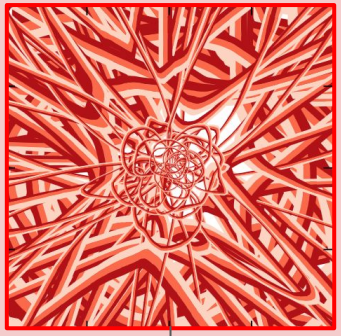
Sophisticated Detector Models



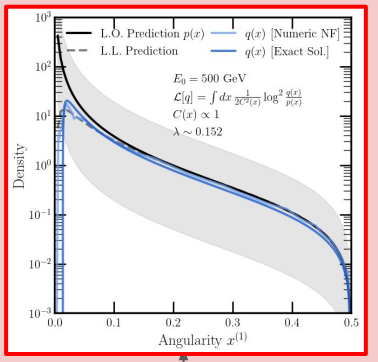
Complicated point clouds



Huge Latent Spaces

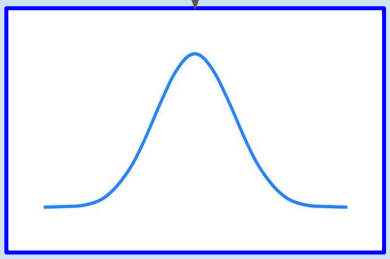


Nontrivially constrained function spaces



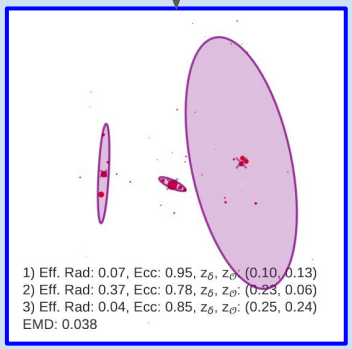
Interface: *Targeted* and *goal-motivated* ML function spaces and loss functionals!

Using the Gaussian Ansatz and DV Loss



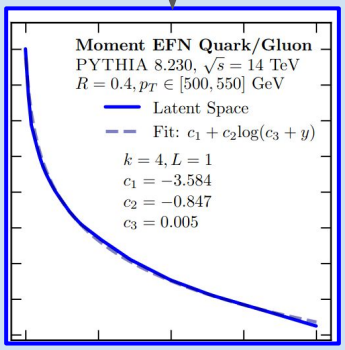
Gaussians

Using faithful optimal transport



Basic Kindergarten Shapes

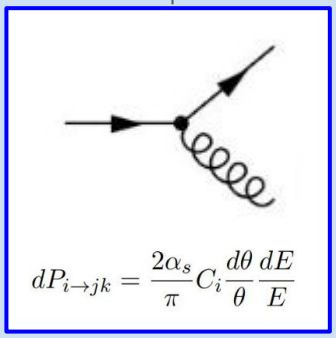
Using Moment Pooling



Addition, multiplication, and a few elementary functions

Using Normalizing Flows

And a perturbative matching Loss



Tree-level Feynman Diagrams

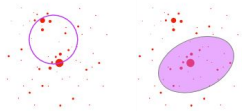
Things my **mortal physicist brain** can understand

Email me questions at rikab@mit.edu!

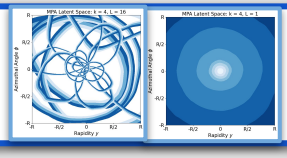
[**RG**, Nachman, Thaler; 2205.03413]
[**RG**, Nachman, Thaler; 2205.05084]
[Ba, Dogra, **RG**, Tasissa, Thaler; 2302.12266]
[**RG**, Thaler, Wu; WIP]
[**RG**, Larkoski, Thaler; 2410.05379]
[**RG**, Osathapan, Thaler; 2403.08854]
[**RG**, Mastandrea; WIP]

Learning **Uncertainties** the **Frequentist** Way

Can you Hear the **Shape** of a Jet?



The **Moments** of Clarity: Streamlining Latent Spaces

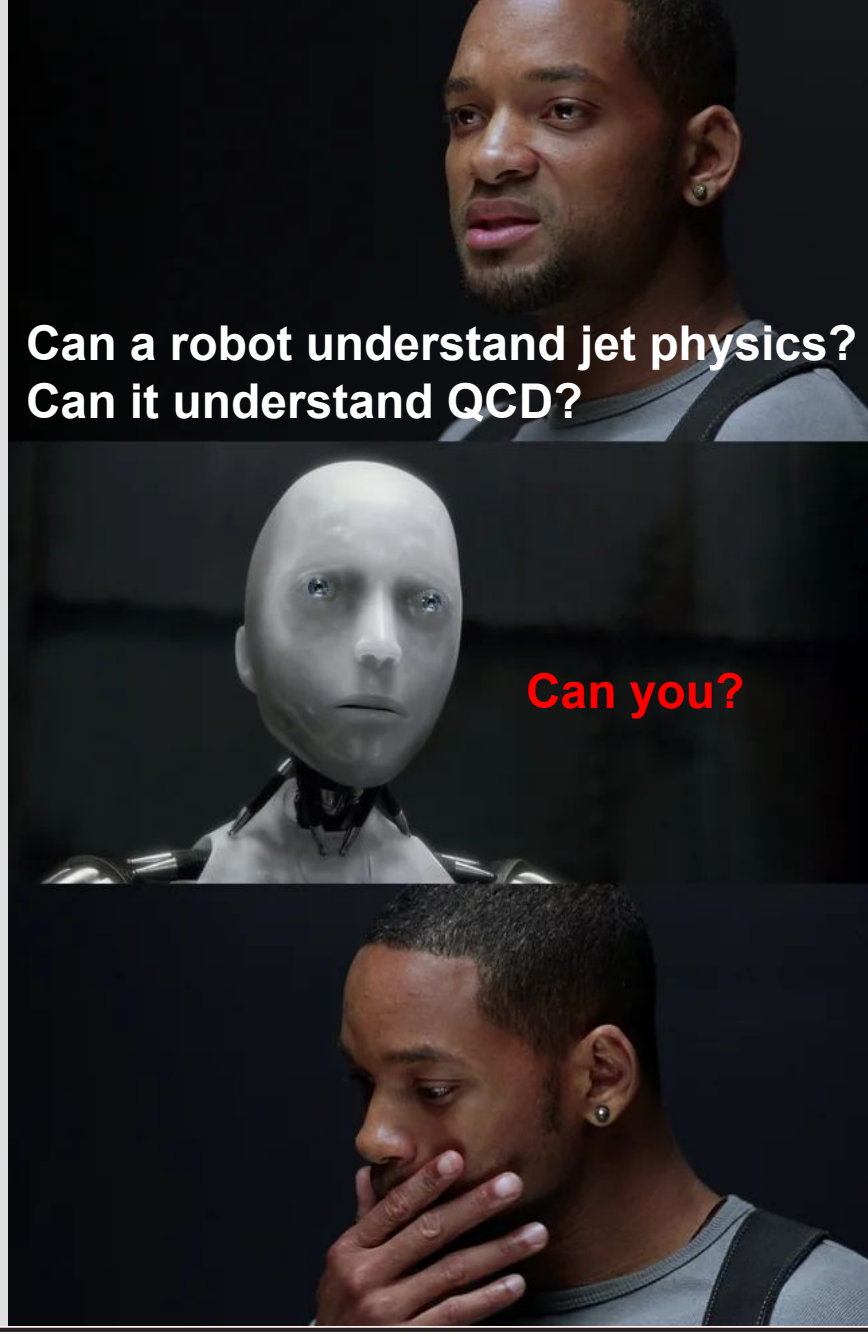


Renormalizing Flows: Taming Perturbation Theory

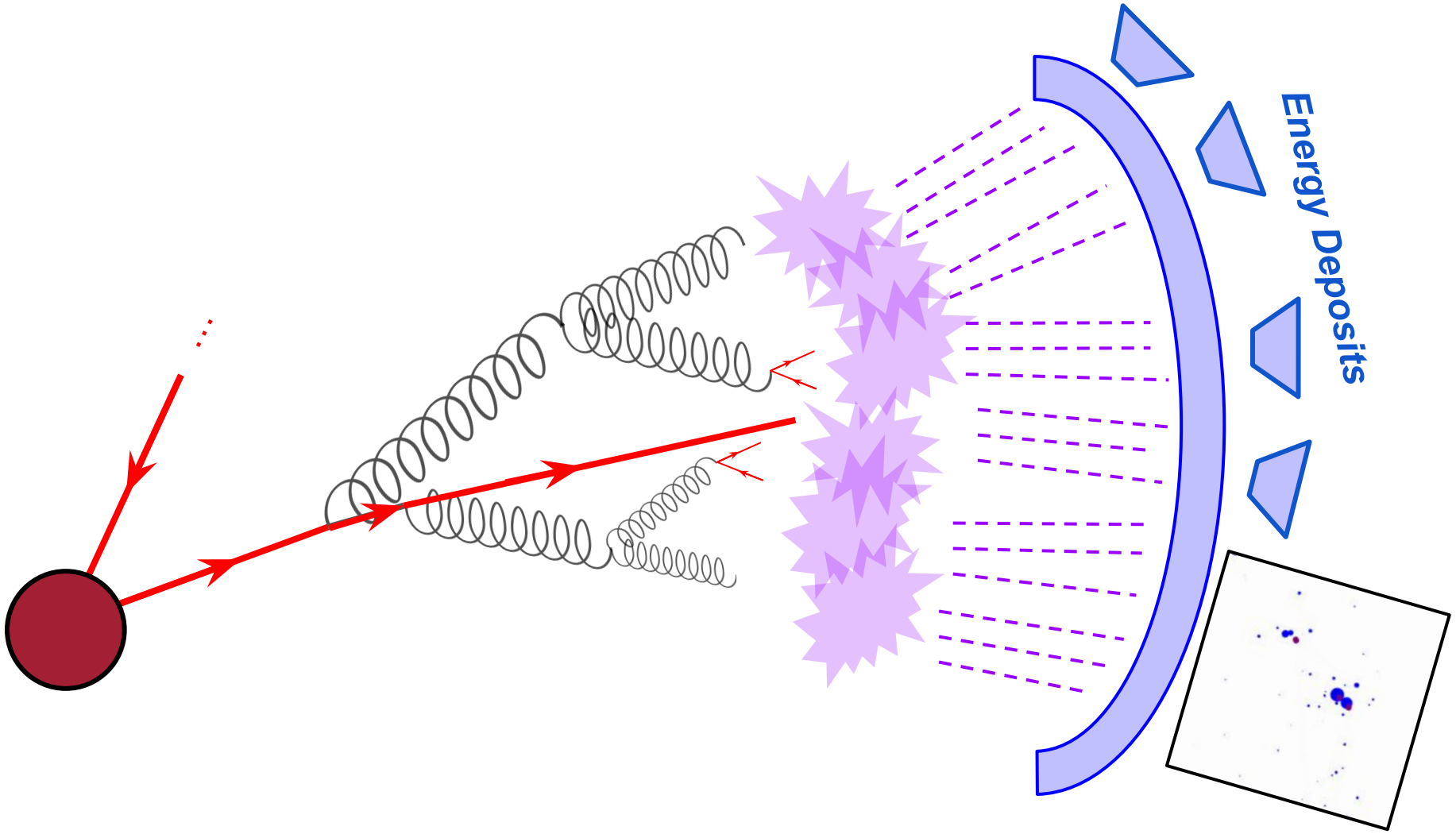
$$\exp[\alpha \frac{d}{d\alpha}]q(x) = q^0(x) + \alpha^1 q^1(x) + \ldots + \alpha^N q^N(x) + \mathcal{O}(\alpha^{N+1})$$
$$\exp[\alpha \frac{d}{d\alpha}]q(x) = q^0(x) + \alpha^1 q^1(x) + \ldots + \alpha^N q^N(x) + \mathcal{O}(\alpha^{N+1})$$

equal!

Any Questions?



Backup


Energy Deposits

Learning MLC

How do we calculate *f*?

$$f_{\mathrm{MLC}}(x) = \operatorname*{argmax}_z p_{\mathrm{train}}(x|z)$$

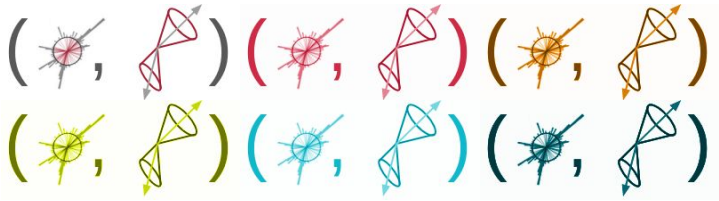
$$= \operatorname*{argmax}_z \underbrace{\log \frac{p_{\mathrm{train}}(x,z)}{p_{\mathrm{train}}(x)p_{\mathrm{train}}(z)}}_{T(x,z)}$$

The function *T* is the likelihood ratio between *p(x,z)* and *p(x)p(z)*.

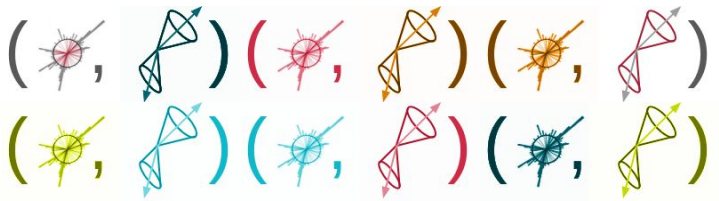
↓ Neyman–Pearson

T is the **optimal classifier** between (*x*,*z*) pairs and shuffled (*x*,*z*) pairs!

Class P



Class Q



Classify between ***P*** and ***Q***!

Aside: Mutual information

A measure for non-linear interdependence is the **Mutual Information**:

$$I(X;Z) = \int \mathrm{d}x\,\mathrm{d}z\, p(x,z) \log \frac{p(x,z)}{p(x)\,p(z)}$$
$$= \mathbb{E}_{\text{train}} T(X,Z)$$

Answers the question: How much information, in terms of bits, do you learn about *Z* when you measure *X* (or vice versa)?

When doing calibration this way, we get a measure of the **correlation** between *X* and *Z*, *for free*.

The general strategy for *all* of these problems will be:

1. Identify what properties we want our solution function *f* to have: *e.g. rior-independent, nice Gaussian limit, contains particular statistical information, IRC-safety, certain geometrical properties, easy function composition, easily plottable limits, physical constraints like smoothness or positivity …*

2. Design a space of parameterized functions *F* that manifestly satisfy the properties you want. *e.g. If I want Gaussian functions in z, $\log(T(x,z)) = A(x) + (B(x) - z)C^{-1}(x)(B(x) - z)$ parametrizes all of them!*

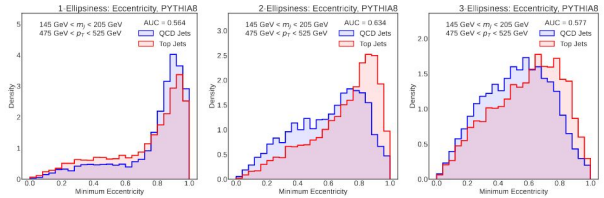
3. In conjunction, design a "Loss Functional" $\mathscr{L}[f]$ over *F* whose minimum satisfies the properties you want. *e.g the Wasserstein Metric guarantees IRC-safety! The DV-loss is prior-free! We know how to do Euler-Lagrange, so just check!*

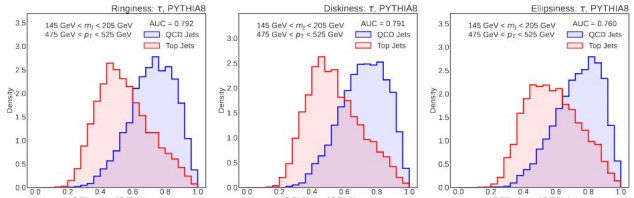
4. Perform the minimization! *This is where the actual ML comes in – but these are just incidental numeric details!*

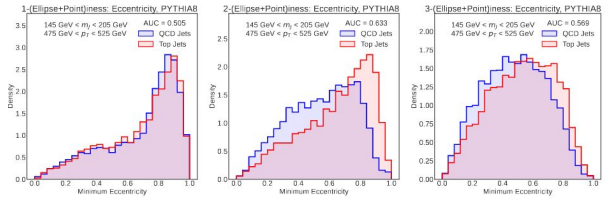
If you don't remember the details of each physics example I show in this talk, I want you to remember this overall theme:

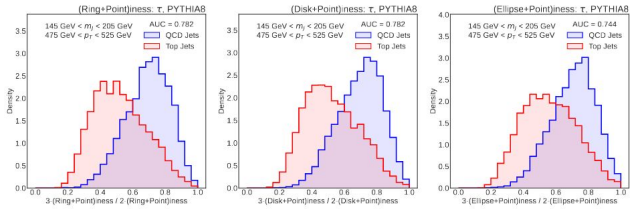
Key Takeaway: This is *all you need* for learning physics from machines!

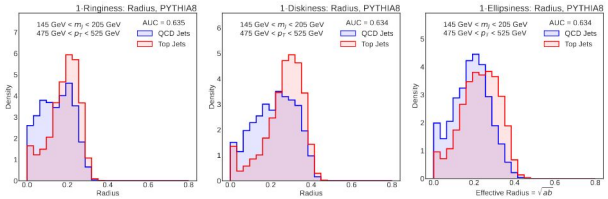
New IRC-Safe Observables

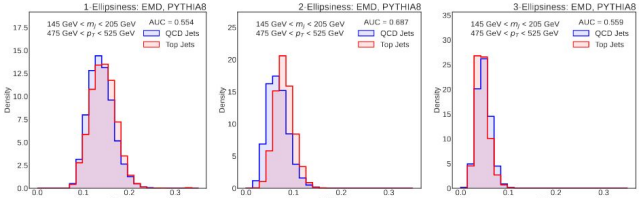


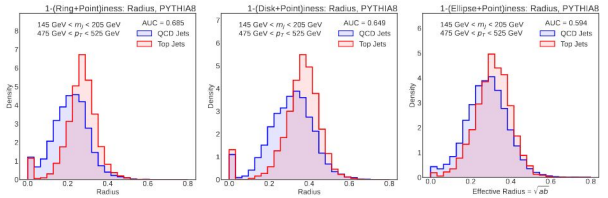


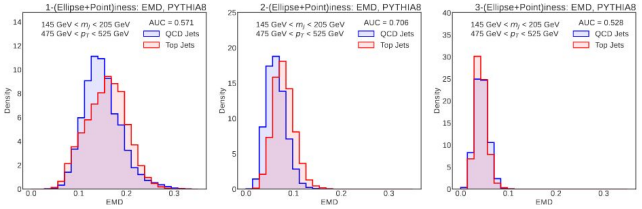


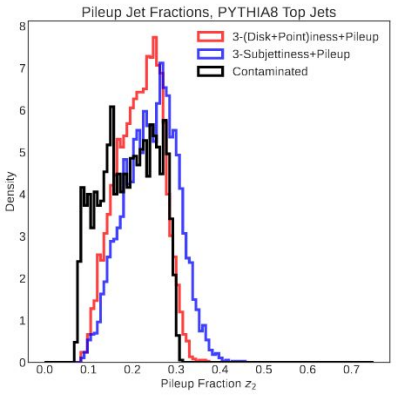


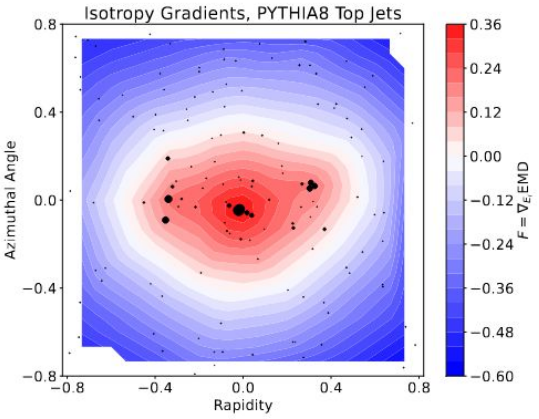












… **Lots** of extractable information!

Automatic Grooming with Shapes

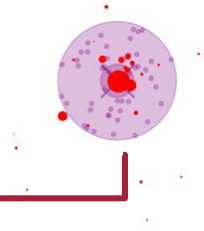
Use shapes to approximate events and extract masses – model pileup with a uniform background with floating weight!

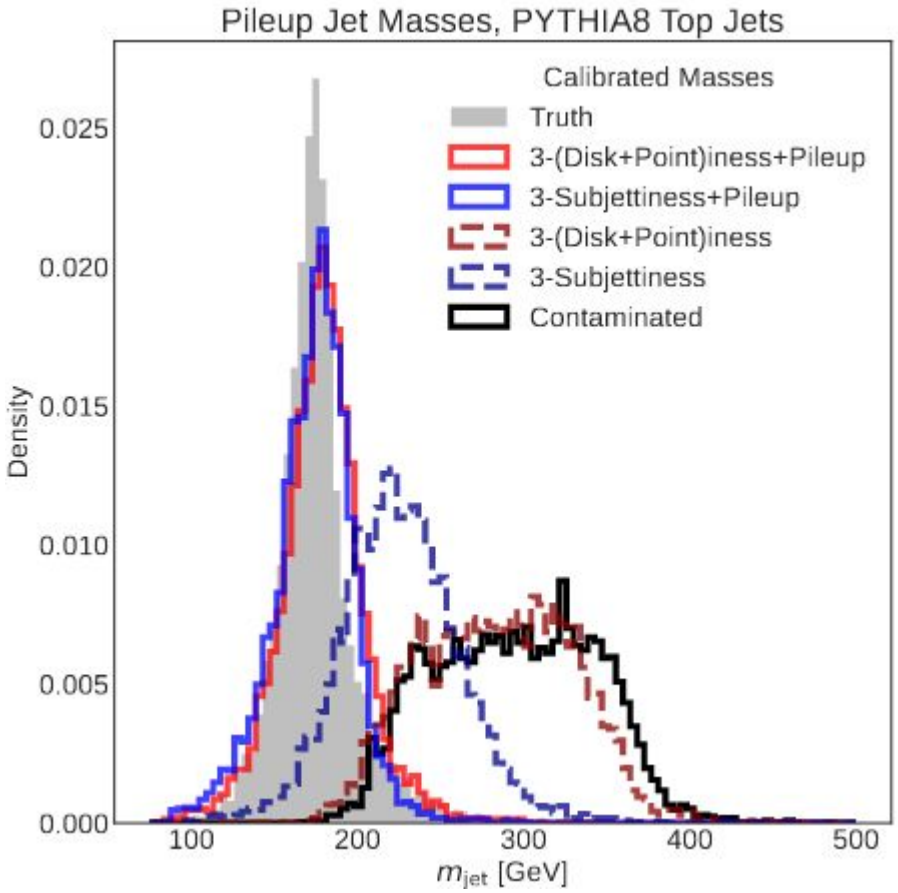
No external hyperparameters, unlike softdrop. Only need to assume pileup is uniform!

Contaminate top jets with 5-30% extra energy spread uniformly in an 0.8x0.8 plane

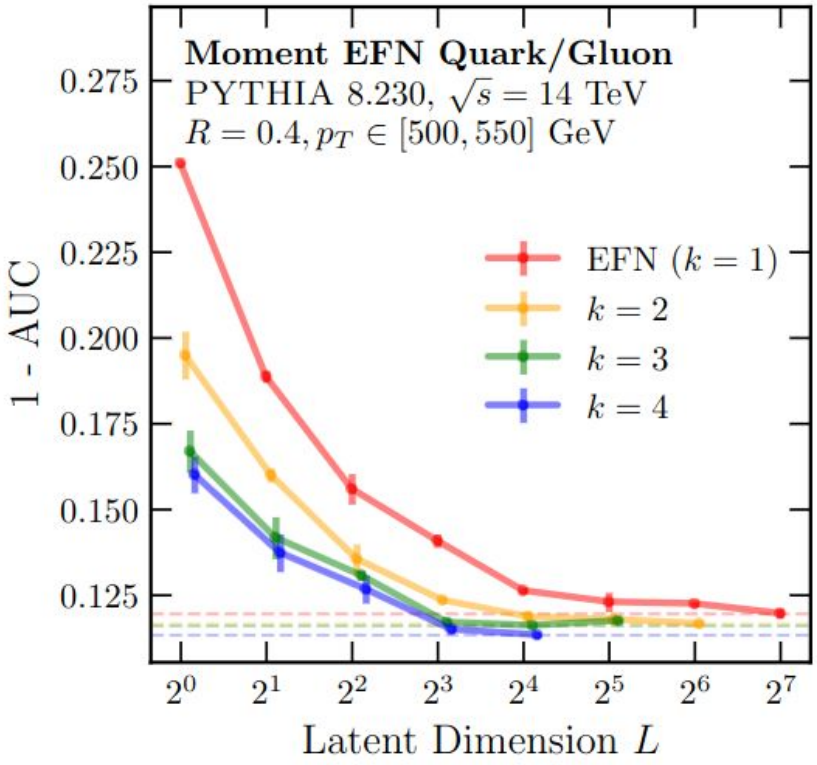
Consider 4 shapes:

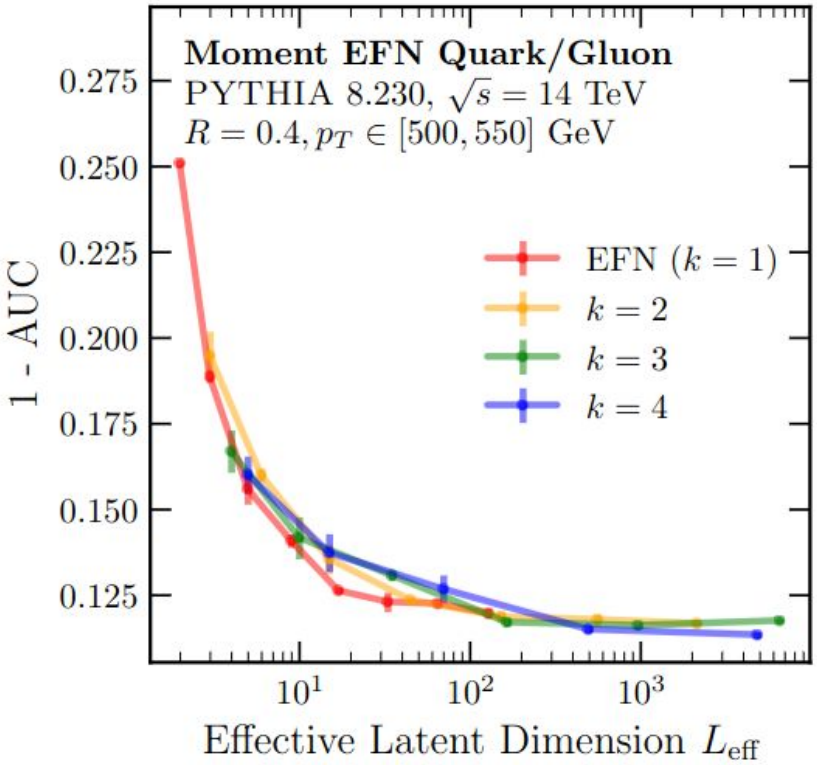
- 3-Subjettiness
- 3-Subjettiness + Pileup
- 3-(Disk+Point)iness
- 3-(Disk+Point)iness + Pileup

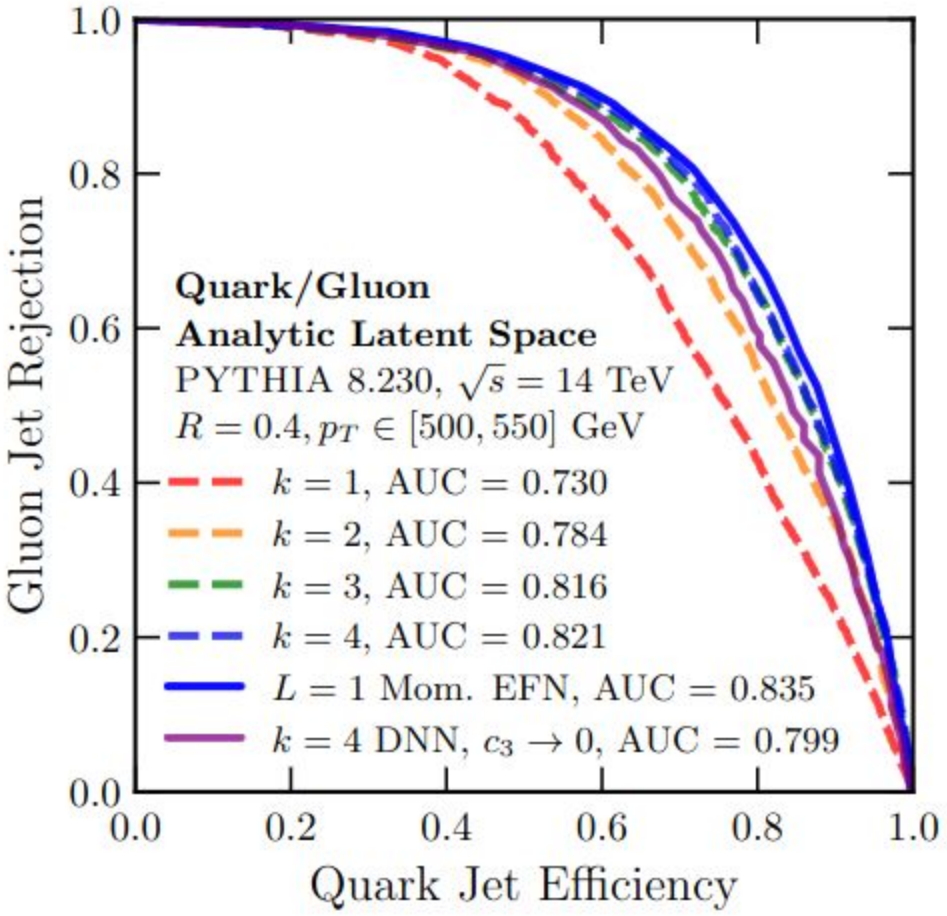




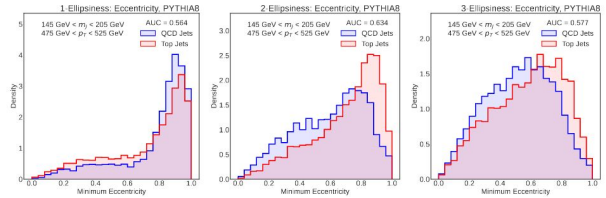
Can also consider ellipses instead of disks – only marginally better performance

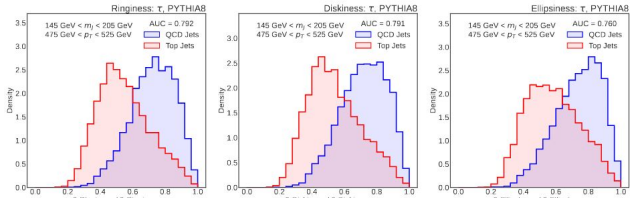

Moment EFN Quark/Gluon
PYTHIA 8.230, $\sqrt{s} = 14$ TeV
$R = 0.4, p_T \in [500, 550]$ GeV
EFN ($k = 1$)
$k = 2$
$k = 3$
$k = 4$
1 - AUC
0.275
0.250
0.225
0.200
0.175
0.150
0.125
2^0 2^1 2^2 2^3 2^4 2^5 2^6 2^7
Latent Dimension L

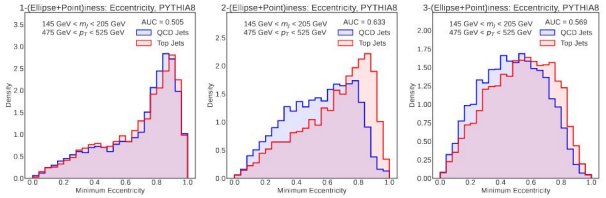

Moment EFN Quark/Gluon
PYTHIA 8.230, $\sqrt{s} = 14$ TeV
$R = 0.4, p_T \in [500, 550]$ GeV
EFN ($k = 1$)
$k = 2$
$k = 3$
$k = 4$
1 - AUC
0.275
0.250
0.225
0.200
0.175
0.150
0.125
10^1 10^2 10^3
Effective Latent Dimension L_{eff}

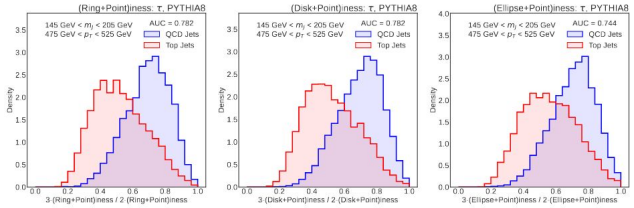

Quark/Gluon
Analytic Latent Space
PYTHIA 8.230, $\sqrt{s} = 14$ TeV
$R = 0.4, p_T \in [500, 550]$ GeV
$k = 1$, AUC = 0.730
$k = 2$, AUC = 0.784
$k = 3$, AUC = 0.816
$k = 4$, AUC = 0.821
$L = 1$ Mom. EFN, AUC = 0.835
$k = 4$ DNN, $c_3 \to 0$, AUC = 0.799
Gluon Jet Rejection
Quark Jet Efficiency
0.0
0.2
0.4
0.6
0.8
1.0

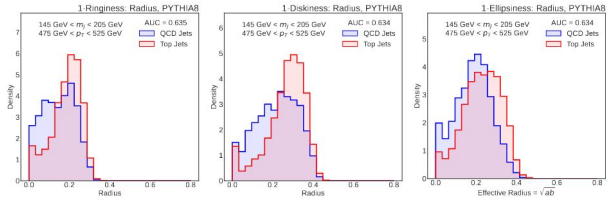
New IRC-Safe Observables

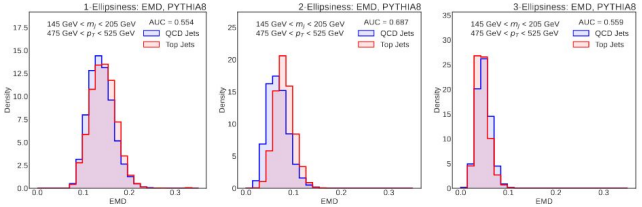


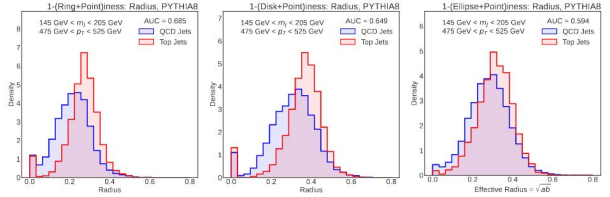


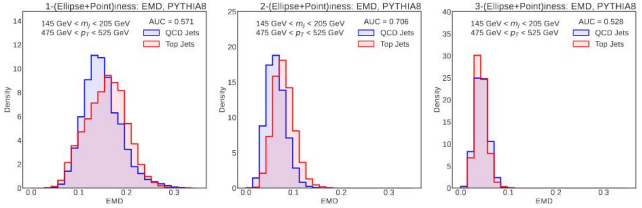


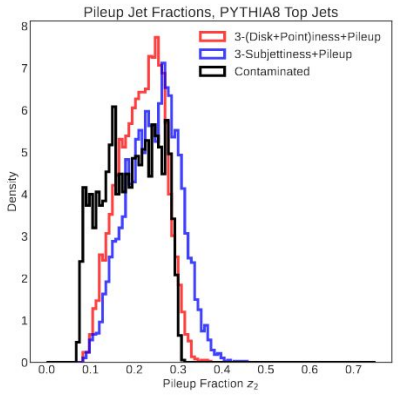


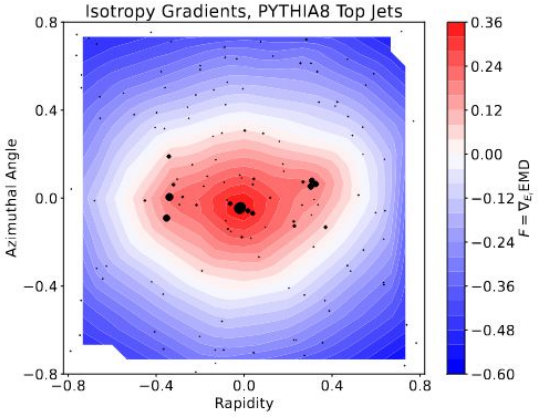












… **Lots** of extractable information!

Automatic Grooming with Shapes

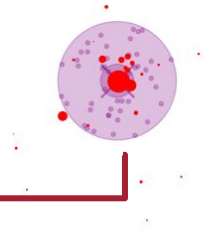
Use shapes to approximate events and extract masses – model pileup with a uniform background with floating weight!

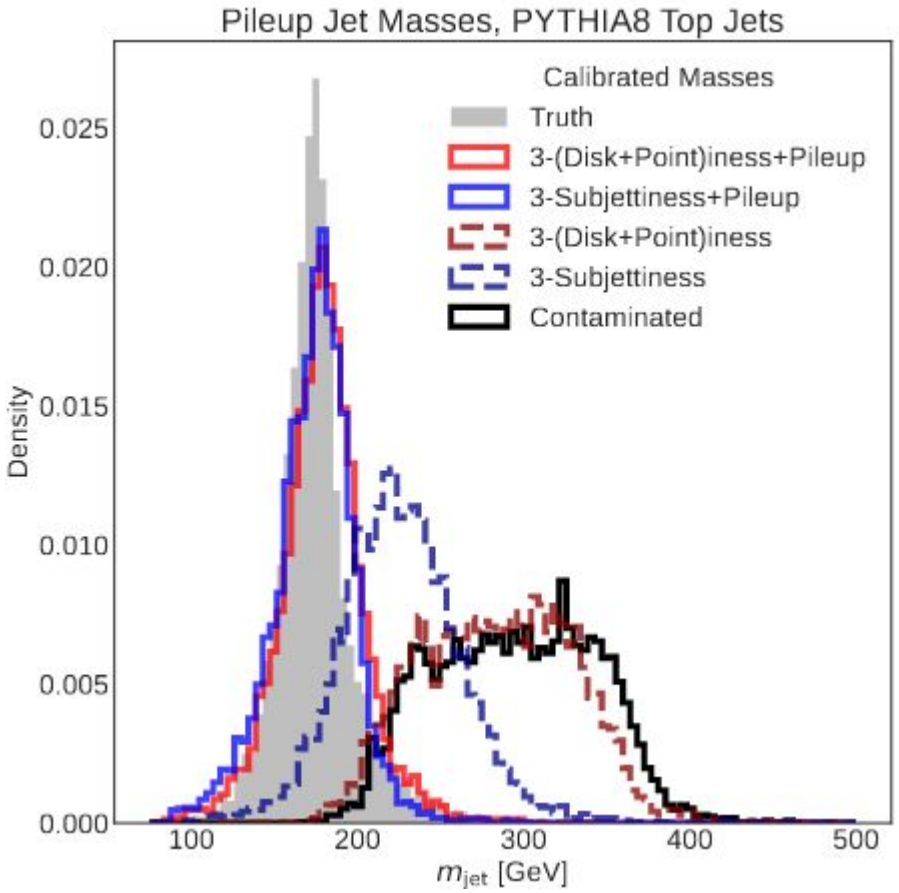
No external hyperparameters, unlike softdrop. Only need to assume pileup is uniform!

Contaminate top jets with 5-30% extra energy spread uniformly in an 0.8x0.8 plane

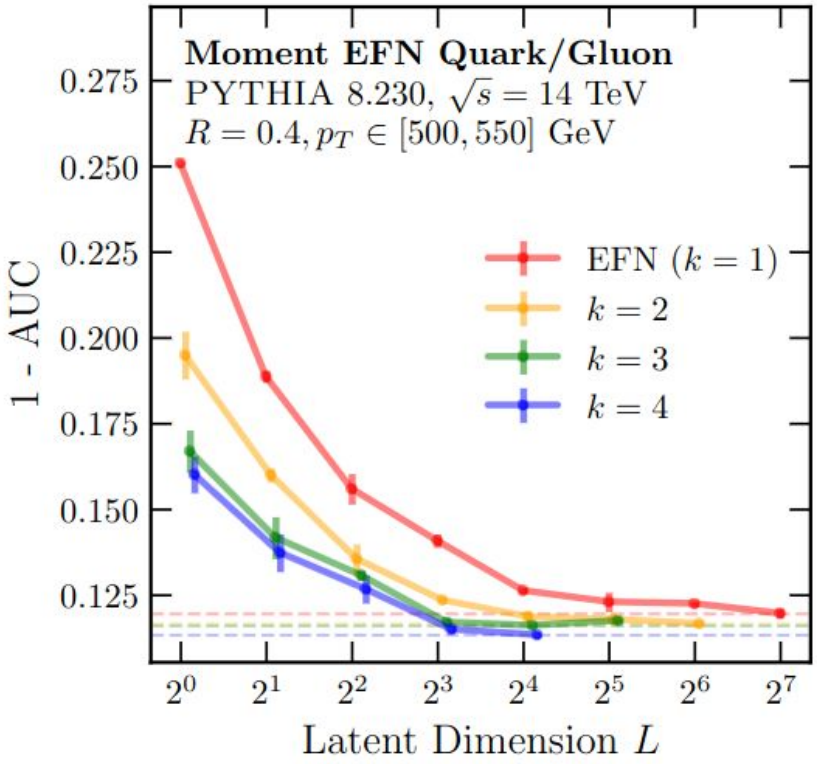
Consider 4 shapes:

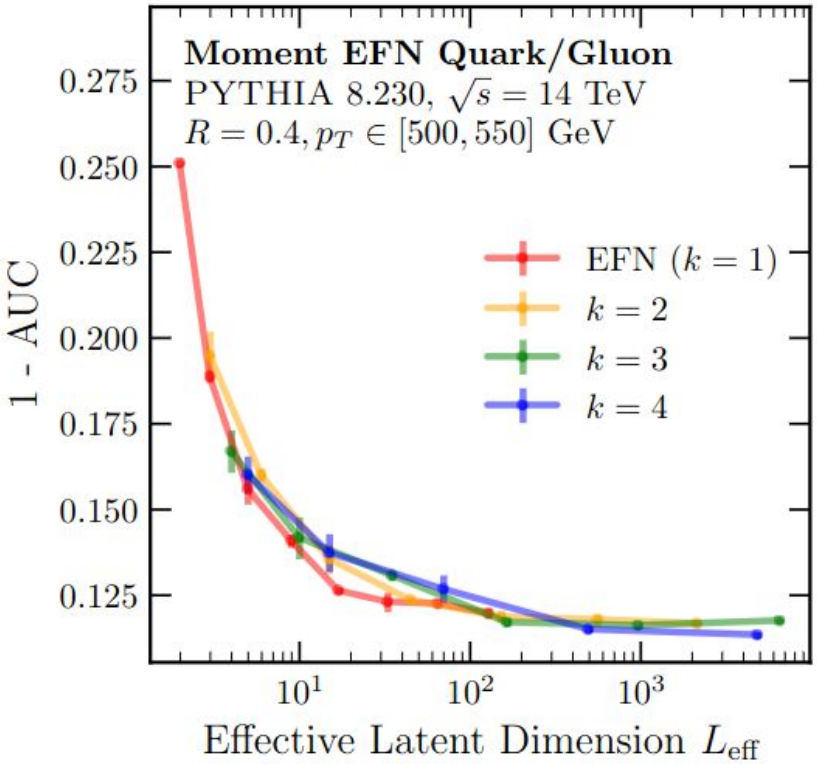
- 3-Subjettiness
- 3-Subjettiness + Pileup
- 3-(Disk+Point)iness
- 3-(Disk+Point)iness + Pileup

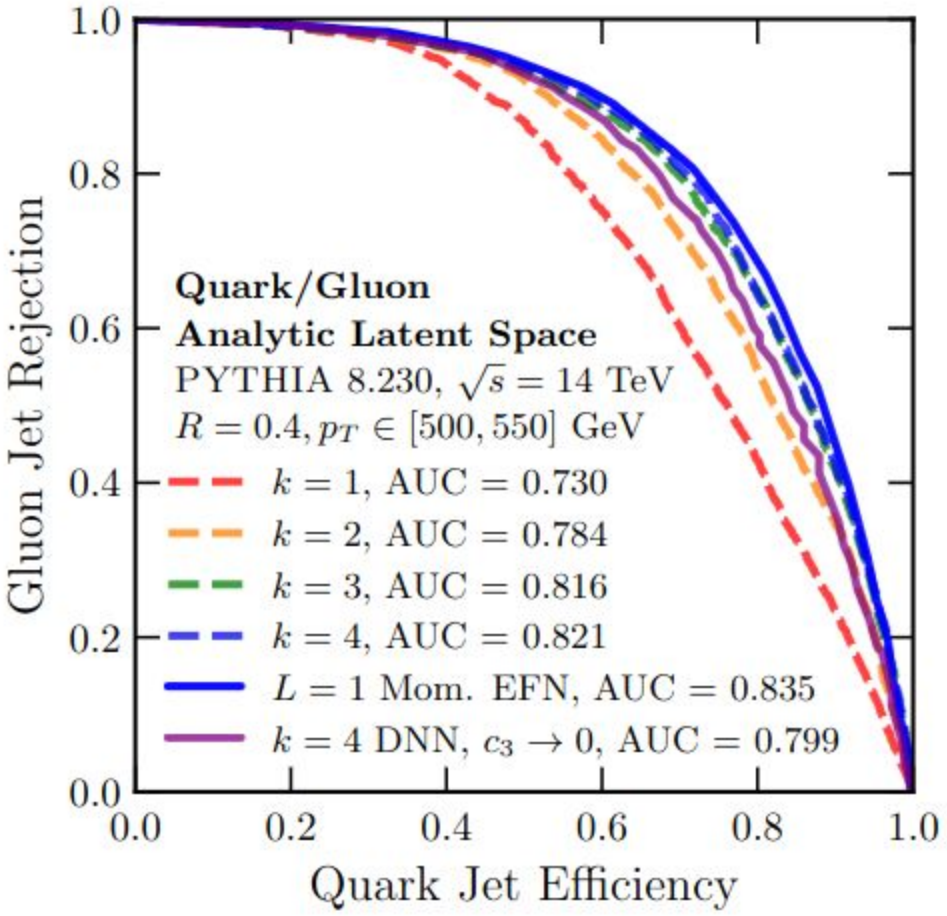




Can also consider ellipses instead of disks – only marginally better performance


Moment EFN Quark/Gluon
PYTHIA 8.230, $\sqrt{s} = 14$ TeV
$R = 0.4, p_T \in [500, 550]$ GeV
EFN ($k = 1$)
$k = 2$
$k = 3$
$k = 4$
1 - AUC
0.275
0.250
0.225
0.200
0.175
0.150
0.125
2^0 2^1 2^2 2^3 2^4 2^5 2^6 2^7
Latent Dimension L


Moment EFN Quark/Gluon
PYTHIA 8.230, $\sqrt{s} = 14$ TeV
$R = 0.4, p_T \in [500, 550]$ GeV
EFN ($k = 1$)
$k = 2$
$k = 3$
$k = 4$
1 - AUC
0.275
0.250
0.225
0.200
0.175
0.150
0.125
10^1 10^2 10^3
Effective Latent Dimension L_{eff}


Quark/Gluon
Analytic Latent Space
PYTHIA 8.230, $\sqrt{s} = 14$ TeV
$R = 0.4, p_T \in [500, 550]$ GeV
$k = 1$, AUC = 0.730
$k = 2$, AUC = 0.784
$k = 3$, AUC = 0.816
$k = 4$, AUC = 0.821
$L = 1$ Mom. EFN, AUC = 0.835
$k = 4$ DNN, $c_3 \to 0$, AUC = 0.799
Gluon Jet Rejection
Quark Jet Efficiency
0.0
0.2
0.4
0.6
0.8
1.0

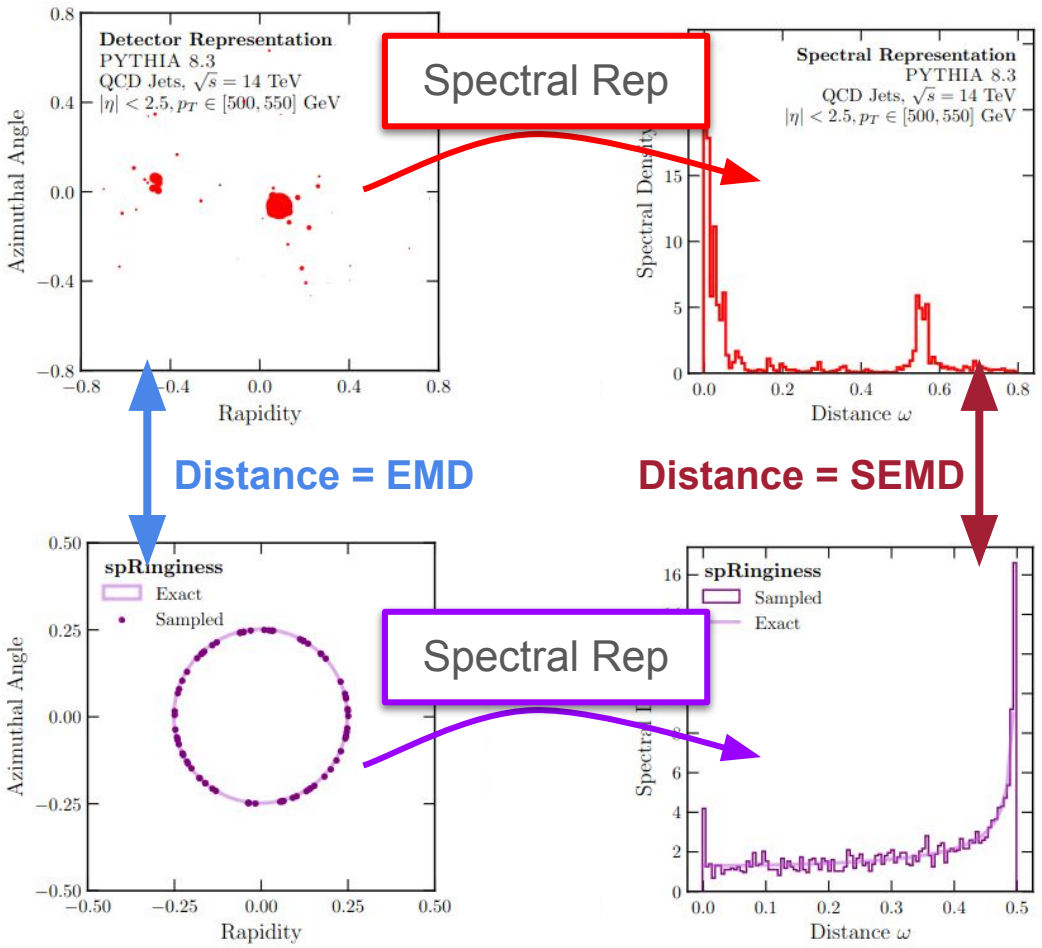
[1][Larkoski, Thaler, 2305.03751]

The Spectral EMD

Similar to the ordinary EMD, the **Spectral EMD (SEMD)**[1] is an IRC-saf metric between events or jets, computed on their **spectral representations**:

$$s(\omega) \equiv \sum_{i,j \in \mathcal{E}} E_i E_j \, \delta(\omega - \omega_{ij})$$

= list of energy-weighted pairwise distances



Then the *p*-SEMD is given by:

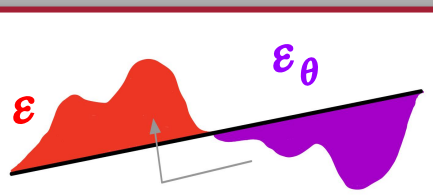
$$\mathrm{SEMD}_p(s_A, s_B) \equiv \int_0^{E_{\mathrm{tot}}^2} dE^2 \, \left| S_A^{-1}(E^2) - S_B^{-1}(E^2) \right|^p$$

Inverse of integral of $s(\omega)$

The SEMD automatically respects all isometries of the metric ω. In this case, the SEMD is invariant under rotations or translations of either event or jet

[1][Larkoski, Thaler, 2305.03751]
[Larkoski, **RG**, Thaler, 23XX.XXXX]

Technical Details …



For $p = 2$, possible
problem on the sp

For events A, B, the p **spectral EMD** is defined as (1D OT!):

EMD = Work done to move "dirt" optimally

$$\mathrm{SEMD}_{\beta,p}(s_A, s_B) \equiv \int_0^{E_{\mathrm{tot}}^2} dE^2 \left| S_A^{-1}(E^2) - S_B^{-1}(E^2) \right|^p$$

$$\mathrm{SEMD}_{\beta,p=2}(s_A, s_B) = \sum_{i<j\in\mathcal{E}_A} 2E_iE_j\omega_{ij}^2 + \sum_{i<j\in\mathcal{E}_B} 2E_iE_j\omega_{ij}^2$$
$$- 2 \sum_{n\in\mathcal{E}_A^2, l\in\mathcal{E}_B^2} \omega_n\omega_l \left(\min\left[S_A(\omega_n^+), S_B(\omega_l^+)\right] - \max\left[S_A(\omega_n^-), S_B(\omega_l^-)\right]\right)$$
$$\times \Theta\left(S_A(\omega_n^+) - S_B(\omega_l^-)\right) \Theta\left(S_B(\omega_l^+) - S_A(\omega_n^-)\right),$$

S = **cumulative spectral function**
± indicates whether or not to include ω in the sum

The trick: Sum over pairs n of particles within each event.

Looks like $O(N^4)$, but with clever sorting & indexing in 1D, reduces to ***O(N²)***!

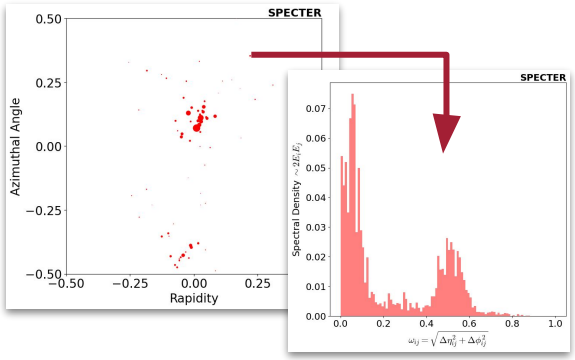
The **spectral density function**

$$s(\omega) = \sum_{i=1}^{N}\sum_{j=1}^{N} E_i\, E_j\, \delta\big(\omega - \omega(\hat{n}_i, \hat{n}_j)\big)$$

Pairwise Distances

Reduces events to 1D, while preserving all* information about the event, up to translations and rotations.

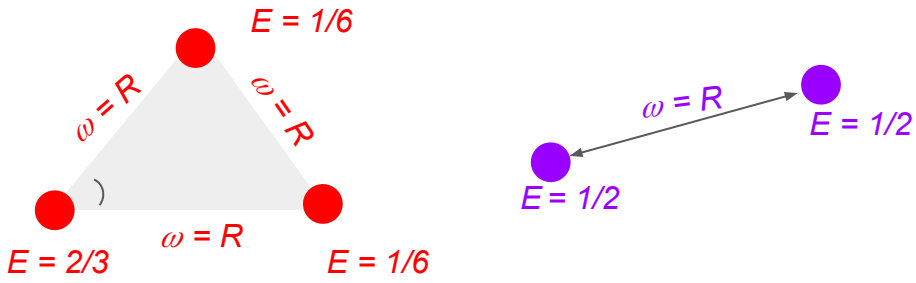
*up to measure 0, but important degeneracies, ask me about this later!



…EMDs, …

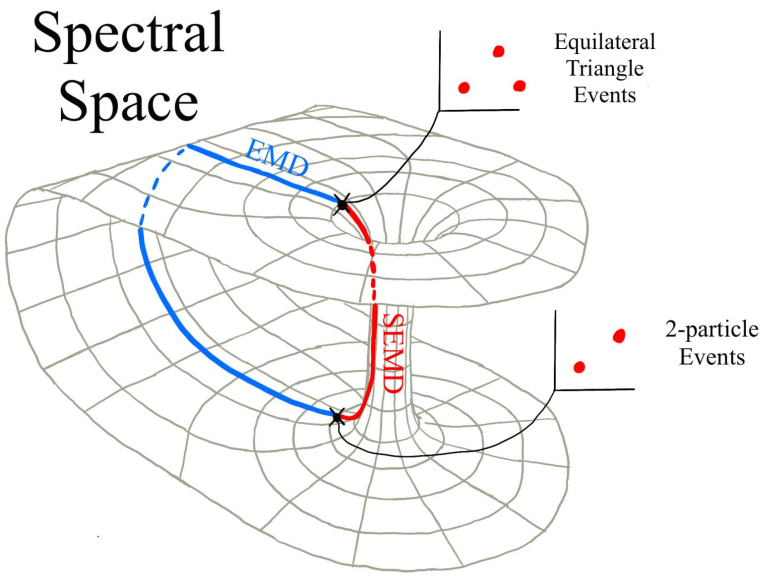
The EMD vs. The SEMD

The SEMD is *topologically different* from the ordinary EMD!



This three particle event looks very different from this two particle event, and thus they have a large EMD.

But their SEMD is zero! The spectral function only cares about pairwise distances and degenerate configurations can occur.



The space of events gets "pinched" at degenerate configurations when looking at only their spectral representations

This can come into play when events have 3 hard prongs, e.g. top jets!

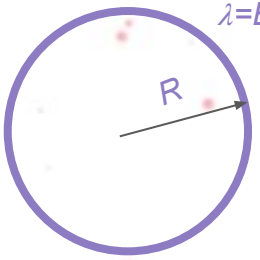
Full Example: How “ring-like” are jets?

SPECTER Our code framework for these calculations

SPECTER

Step 1: Define the shape with parameters

$\lambda = E_{tot} / 2\pi R$

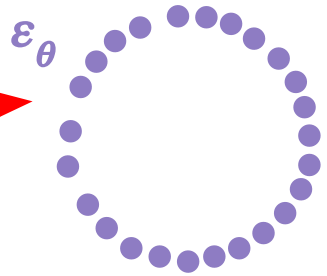



```
def sample_circle(params, N, seed):

    thetas = jax.random.uniform(seed, shape
    x = params["Radius"] * jnp.cos(thetas)
    y = params["Radius"] * jnp.sin(thetas)
```

Unlike ordinary EMD, not necessary to specify center / orientation!

Step 2: Sample from Parameterized Shapes

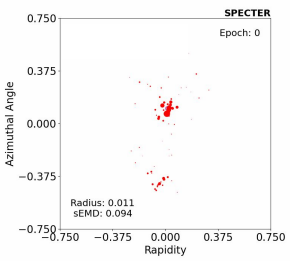


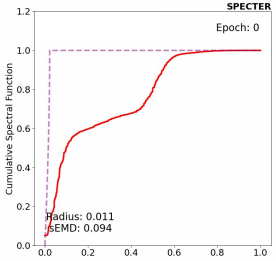
Step 3: Calculate the spectral metric between events and shapes

$$\mathrm{SEMD}_{\beta,p=2}(s_A, s_B) = \sum_{i<j\in\mathcal{E}_A} 2E_iE_j\omega_{ij}^2 + \sum_{i<j\in\mathcal{E}_B} 2E_iE_j\omega_{ij}^2$$
$$- 2 \sum_{n\in\mathcal{E}_A^2, l\in\mathcal{E}_B^2} \omega_n\omega_l \left(\min\left[S_A(\omega_n^+), S_B(\omega_l^+)\right] - \max\left[S_A(\omega_n^-), S_B(\omega_l^-)\right]\right)$$
$$\times \Theta\left(S_A(\omega_n^+) - S_B(\omega_l^-)\right)\Theta\left(S_B(\omega_l^+) - S_A(\omega_n^-)\right),$$

Key difference from previous work: We use the SEMD, *not* the EMD!

Step 4: Minimize w.r.t. parameters using grads

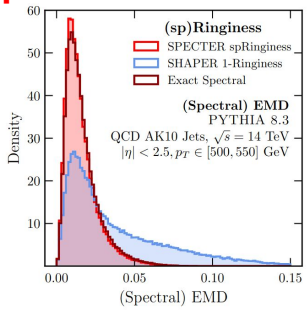


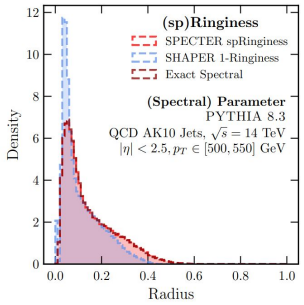


Pictured: Animation of optimizing for the radius R^{ω}

Pictured: 100k Jets, PYTHIA 8 QCD Jets

Step 5: Plots!





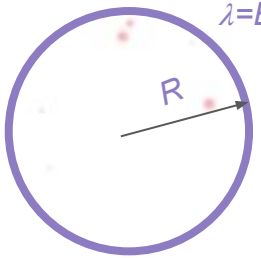
Full Example: How “ring-like” are jets?

SPECTER Our code framework for these calculations

SPECTER

Step 1: Define the shape with parameters

$\lambda = E_{tot} / 2\pi R$




```
def sampl
    theta
    x = p
    y = p
```

Unlike ordinary EMD, not necessary to s

Alternatively…

The spectral EMD, *and its optimization*, are often partially or ***completely solvable*** in closed form!

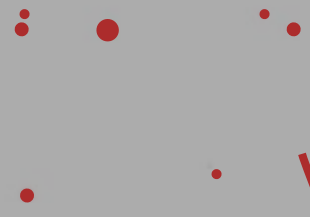
$$R_{\text{opt}} = \frac{2}{\pi} \sum_{\substack{n \in \mathcal{E}^2 \\ \omega_n < \omega_{n+1}}} \omega_n \left[\sin\left(\frac{\pi}{2E_{\text{tot}}^2} \sum_{\substack{n \le m \in \mathcal{E}^2 \\ \omega_m < \omega_{m+1}}} (2EE)_m \right) - \sin\left(\frac{\pi}{2E_{\text{tot}}^2} \sum_{\substack{n < m \in \mathcal{E}^2 \\ \omega_m < \omega_{m+1}}} (2EE)_m \right) \right]$$

$$\text{SEMD}_{\beta, p=2}(s, s_{\text{jet ring}}) = \sum_{i<j \in \mathcal{E}} 2E_i E_j \omega_{ij}^2 - 2E_{\text{tot}}^2 R_{\text{opt}}^2$$

For many shapes, we can completely short circuit having to perform expensive optimization over an optimal transport problem entirely!

Pictured: 100k Jets, PYTHIA 8 QCD Jets

Step 2: Sample from Parameterized Shapes



e spectral metric between

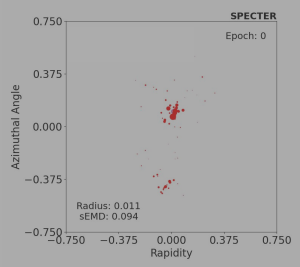
$$\sum_{\mathcal{E}_A} 2E_i E_j \omega_{ij}^2 + \sum_{i<j \in \mathcal{E}_B} 2E_i E_j \omega_{ij}^2$$

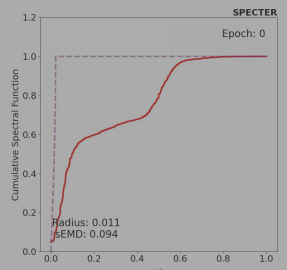
$$(\omega_n^+), S_B(\omega_l^+)] - \max\left[S_A(\omega_n^-), S_B(\omega_l^-)\right])$$

$$) - S_B(\omega_l^-)) \, \Theta\left(S_B(\omega_l^+) - S_A(\omega_n^-)\right) ,$$

Key difference from previous work: We use the SEMD, *not* the EMD!

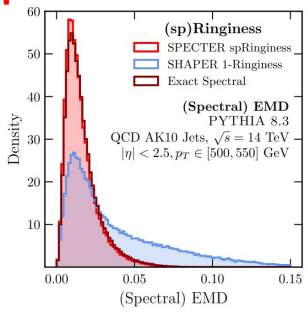
Step 4: Minimize w.r.t. parameters using grads

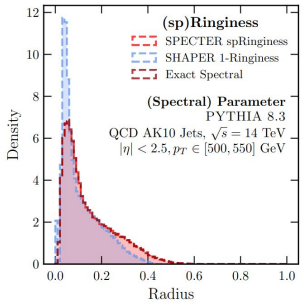




Pictured: Animation of optimizing for the radius R^{ω}

Step 5: Plots!





Hearing Jets (sp)Ring

Runtimes (NVIDIA A100 GPU):
SHAPER (EMD): ~ 36 hours / 100k events
Generalized *SPECTER*: ~55 seconds / 100k events
Closed Form *SPECTER*: ~ < 0.1 seconds / 100k events

The SEMD and EMD are qualitatively different, but give similar radii!
They probe the same event length scale